COLEGIO DE LA UNIVERSIDAD PONTIFICIA BOLIVARIANA
PROYECTO DE VIDA / TECNOLOGÍA E INFORMÁTICA

MANUAL DE PROGRAMADOR

JINHWAN CORPORATION
SISTEMA INTEGRAL DE GESTIÓN MARCIAL

INTEGRANTES / EQUIPO DE DESARROLLO:

Samuel Gomez Londoño

Estefania Londoño Sanchez

Luciana Rodríguez Hernández

Mariana Mora Chavarría

Estefani Andreina Montoya Soto

OCTUBRE DEL 2026
MEDELLÍN, COLOMBIA

ÍNDICE GENERAL

1. Introducción y Objetivo del Sistema
2. Librerías y Tecnologías Utilizadas
3. Estructura General del Sistema (Arquitectura MVC)
4. Configuración del Entorno y Base de Datos
   a. config/conexion.php
   b. config/ia.php
   c. Modelo Relacional de Base de Datos
5. Núcleo del Framework (Core)
   a. Router.php
   b. Controller.php
   c. Model.php
   d. Security.php
   e. Roles.php
6. Modelos de Datos (Models)
   a. Usuario.php
   b. Cronograma.php
   c. Ejercicio.php
   d. Certificado.php
   e. Teoria.php
   f. Sede.php y Grupo.php
   g. Evento.php y MultimediaGaleria.php
7. Controladores y Lógica de Negocio
   a. Módulo de Autenticación (AutenticacionController.php)
   b. Módulo de Administración (Estudiantes, Maestros, Ascensos, Sedes, Grupos, Teoría)
   c. Módulo de Maestro (Dashboard, Cronogramas, Ejercicios, Alumnos, IAController)
   d. Módulo de Estudiante (Dashboard, Estudio, Historial)
   e. Módulo de Usuario y Portal Web Público
8. Vistas y Componentes de Interfaz
9. Casos de Uso Críticos y Flujo de Datos
10. Guía de Instalación y Mantenimiento Técnico

1. Introducción

El presente Manual de Programador tiene como objetivo explicar en detalle la estructura, arquitectura interna, estándares de codificación y funcionamiento lógico del sistema de información Jinhwan Corporation. Este software fue diseñado y desarrollado como una plataforma integral de gestión marcial orientada a academias y dojangs de Taekwondo, permitiendo coordinar de manera eficiente la labor de directores, maestros y deportistas.

A lo largo de este documento técnico se documenta la arquitectura Modelo-Vista-Controlador (MVC), el motor de enrutamiento propio, los mecanismos de seguridad y control de acceso basado en roles (RBAC), el catálogo completo de modelos de datos, las capas controladoras por cada perfil de usuario y la integración con servicios externos como el asistente inteligente de planificación de clases.

Cada sección incluye explicaciones detalladas de los archivos fuente, descripción exhaustiva de sus métodos y funciones principales, junto con los recuadros delimitados para la inserción de las capturas de pantalla de código de los módulos desarrollados.

2. Librerías y Tecnologías Utilizadas

El sistema Jinhwan Corporation utiliza un conjunto homogéneo de herramientas y dependencias para garantizar alto rendimiento, mantenibilidad y una experiencia visual de primer nivel:

- **PHP 8.x (Backend):** Lenguaje principal del servidor. Implementa programación orientada a objetos (POO), tipado estricto, manejo seguro de sesiones nativas y una arquitectura limpia sin sobrecargas de frameworks pesados.
- **MySQL / MariaDB & MySQLi (Base de Datos):** Motor relacional para la persistencia transaccional. La comunicación se realiza mediante la extensión nativa MySQLi con consultas preparadas (Prepared Statements), previniendo eficazmente ataques de inyección SQL.
- **Tailwind CSS v4 (Diseño y Maquetación):** Framework de utilidades CSS de última generación. Permite construir una interfaz moderna, limpia, responsiva y adaptable a modo claro y oscuro sin comprometer la velocidad de carga.
- **TypeScript y JavaScript Moderno (Interactividad):** Utilizado para la lógica dinámica del lado del cliente: validaciones de formularios en tiempo real, peticiones asíncronas vía Fetch API, filtrado dinámico de tablas y paginación reactiva.
- **Vite (Empaquetador de Recursos):** Herramienta moderna de frontend para la compilación, minificación y optimización en caliente de los estilos CSS y scripts JavaScript.
- **SweetAlert2 (Alertas y Modales):** Librería de interfaces gráficas para notificaciones interactivas elegantes, alertas de confirmación de eliminación y retroalimentación de operaciones.
- **OpenRouter API (Inteligencia Artificial):** API REST de inferencia de modelos de lenguaje para el módulo del maestro (IAController), permitiendo generar planes de clase personalizados según la edad y cinturón de los deportistas.

3. Estructura General del Sistema

El sistema implementa el patrón de arquitectura de software Modelo - Vista - Controlador (MVC), asegurando una separación total entre la lógica de negocio, el acceso a datos y la capa de presentación visual:

- **Punto de Entrada Único (index.php):** Recibe todas las solicitudes HTTP dirigidas a la aplicación. Carga la configuración, inicializa la sesión, importa los archivos del núcleo (core) y delega la ejecución al ruteador general.
- **Enrutador Central (ruteador.php y core/Router.php):** Interpreta el método HTTP (GET/POST) y el URI solicitado, ejecutando el controlador y método correspondiente.
- **Controladores (controladores/):** Orquestan la lógica del negocio. Verifican los permisos de rol mediante Security.php, consultan o actualizan datos a través de los modelos y preparan los datos para la vista.
- **Modelos (modelos/):** Heredan de core/Model.php. Encapsulan la lógica de persistencia y ejecutan las sentencias SQL contra la base de datos.
- **Vistas (vistas/):** Plantillas PHP puras con HTML5 semántico y utilidades de Tailwind CSS. Presentan la información procesada al usuario final.

Árbol de Directorios del Proyecto

A continuación se detalla la organización jerárquica de carpetas del repositorio:

- **config/:** Archivos de configuración global: conexion.php (conexión a base de datos) e ia.php (parámetros de OpenAI / OpenRouter).
- **controladores/:** Controladores agrupados por contexto funcional: Administracion, Maestro, Estudiante, Autenticacion, Usuario y Web.
- **core/:** Núcleo del framework MVC: Router.php, Controller.php, Model.php, Security.php y Roles.php.
- **database/:** Scripts de base de datos SQL (esquema if0_42216592_jinhwa_corporation.sql) y respaldos.
- **helpers/:** Funciones auxiliares transversales (ej. url_helper.php).
- **modelos/:** Clases de entidad y acceso a datos SQL.
- **public/:** Raíz pública web con hojas de estilo compiladas, scripts JS e imágenes.
- **vistas/:** Archivos de vista organizados por rol, además de la subcarpeta layouts/ para cabeceras, menús y pies de página.

Archivo de Entrada: index.php

El archivo index.php es el despachador inicial. Su responsabilidad es verificar la sesión PHP, incluir los cargadores del núcleo y activar el ruteador del sistema.

index.php

```php
<?php

require_once __DIR__ . '/config/conexion.php';
require_once __DIR__ . '/helpers/url_helper.php';

require_once __DIR__ . '/core/Roles.php';
require_once __DIR__ . '/core/Security.php';
require_once __DIR__ . '/core/Model.php';
require_once __DIR__ . '/core/Controller.php';
require_once __DIR__ . '/core/Router.php';

require_once __DIR__ . '/modelos/Sede.php';
require_once __DIR__ . '/modelos/Nivel.php';
require_once __DIR__ . '/modelos/Teoria.php';
require_once __DIR__ . '/modelos/Usuario.php';
require_once __DIR__ . '/modelos/MultimediaGaleria.php';
require_once __DIR__ . '/modelos/Evento.php';
require_once __DIR__ . '/modelos/Categoria.php';

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

require_once __DIR__ . '/ruteador.php';
```

4. Configuración del Entorno y Base de Datos

a. Archivo de Conexión: config/conexion.php

El módulo conexion.php define la función getDBConnection() y expone la variable global de conexión $conn. Soporta tanto el entorno de desarrollo local (Laragon con MySQL en localhost) como el entorno de producción remoto en InfinityFree.

- **Manejo de Errores:** Valida el estado de la conexión mediante mysqli_connect_error(). En caso de fallo crítico, detiene la ejecución informando el error de forma segura.
- **Juego de Caracteres UTF-8:** Ejecuta mysqli_set_charset($conn, 'utf8mb4') para garantizar compatibilidad total con tildes, caracteres especiales y emojis.

config > conexion.php

```php
<?php

class Database {
    private static $instance = null;
    private $connection;

    private $host = 'sql113.infinityfree.com';
    private $user = 'if0_42216592';
    private $pass = 'IfK0M0n94NKpHQq';
    private $name = 'if0_42216592_jinhwa_corporation';

    private function __construct() {
        $this->connection = new mysqli($this->host, $this->user, $this->pass, $this->name);

        if ($this->connection->connect_error) {
            die("Connection failed: " . $this->connection->connect_error);
        }

        $this->connection->set_charset("utf8mb4");
        $this->connection->query("SET NAMES 'utf8mb4'");
        $this->connection->query("SET CHARACTER SET utf8mb4");
    }

    public static function getInstance() {
        if (!self::$instance) {
            self::$instance = new Database();
        }
        return self::$instance;
    }

    public function getConnection() {
        return $this->connection;
    }
}
```

b. Configuración del Asistente IA: config/ia.php

Este archivo centraliza las credenciales, endpoint y modelo de lenguaje utilizado por el módulo inteligente de maestros. Permite modificar las claves de API y el modelo sin alterar el código de los controladores.

config > ia.php

```php
<?php

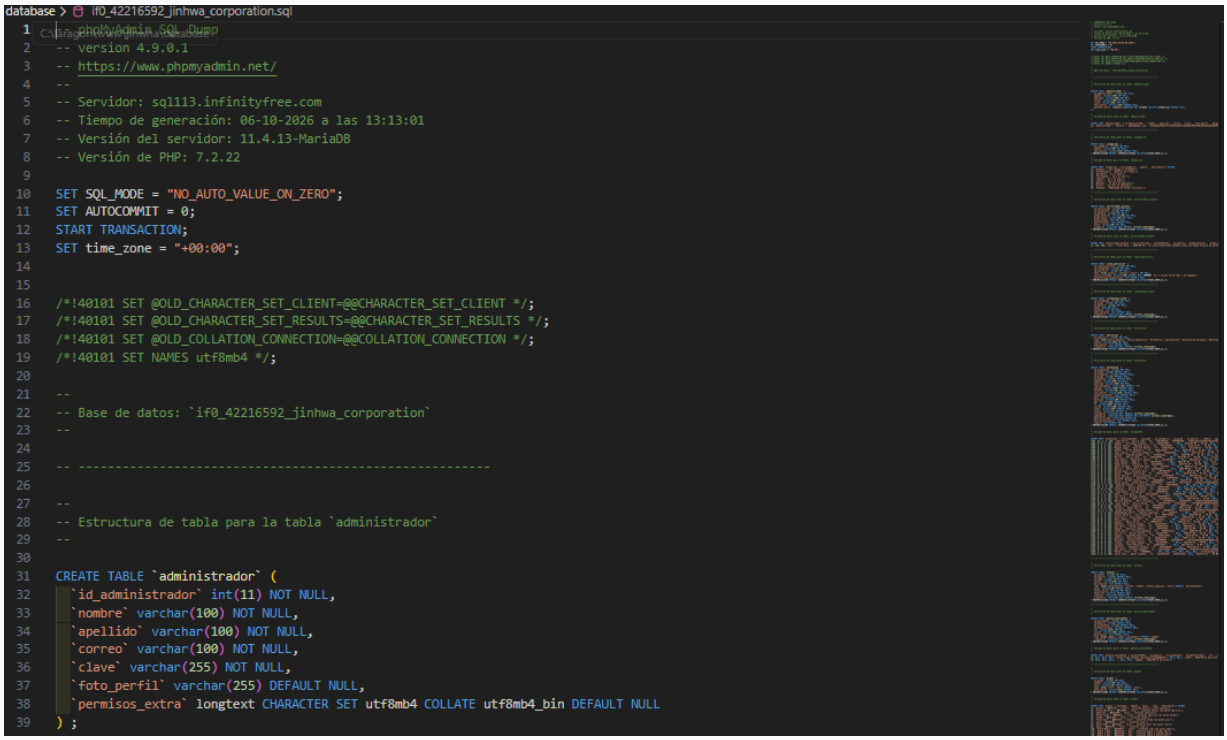
/**
 * Configuración de la IA — NVIDIA NIM API
 * Modelos disponibles con la API Key actual:
 *   - openai/gpt-oss-20b (Ultra rápido, respuestas en ~2s, excelente en español y JSON)
 *   - meta/llama-3.2-90b-vision-instruct
 *   - deepseek-ai/deepseek-v4.1-flash
 */

define('NVIDIA_API_KEY',        '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');
define('NVIDIA_BASE_URL',       'https://integrate.api.nvidia.com/v1/chat/completions');
define('NVIDIA_MODEL',          'openai/gpt-oss-20b');
define('NVIDIA_FALLBACK_MODEL', 'meta/llama-3.2-90b-vision-instruct');
```

c. Estructura y Tablas de la Base de Datos

El sistema cuenta con un esquema de base de datos relacional de 16 tablas principales normalizadas:

- **administrador:** Almacena los usuarios con privilegios globales (id, nombre, email, password, fecha_registro).
- **maestro:** Instructores de la academia (id, nombre, email, password, telefono, especialidad, id_sede, activo).
- **estudiante:** Deportistas registrados (id, nombre, email, password, fecha_nacimiento, nombre_acudiente, telefono_acudiente, id_grado, id_sede, activo).
- **grados:** Niveles y cinturones de Taekwondo (id, nombre, color, tipo_grado: Kup / Dan, orden).
- **sedes:** Dojangs o sucursales físicas (id, nombre, direccion, telefono, ciudad).
- **grupos:** Divisiones y horarios de entrenamiento (id, nombre, horario, id_sede, id_maestro).
- **ejercicios:** Biblioteca de técnicas marciales (id, nombre, descripcion, id_categoria, id_nivel, repeticiones_sugeridas).
- **cronogramas_clase:** Sesiones planificadas (id, fecha, titulo, objetivo, id_grupo, id_maestro).
- **clase_ejercicios:** Tabla intermedia que asocia los ejercicios asignados a cada cronograma de clase.
- **certificados_ascenso:** Diplomas oficiales emitidos tras aprobar examen de ascenso de cinturón.
- **historial_grados:** Registro cronológico de los grados alcanzados por cada deportista.
- **teorias y tipos_teoria:** Contenidos pedagógicos marciales (terminología coreana, historia y principios del Taekwondo).
- **eventos y galeria_multimedia:** Publicaciones en calendario y fotografías de torneos y actividades.



```
database > if0_42216592_jinhwa_corporation.sql
-- phpMyAdmin SQL Dump
-- version 4.9.0.1
-- https://www.phpmyadmin.net/
--
-- Servidor: sql113.infinityfree.com
-- Tiempo de generación: 06-10-2026 a las 13:13:01
-- Versión del servidor: 11.4.13-MariaDB
-- Versión de PHP: 7.2.22

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
SET AUTOCOMMIT = 0;
START TRANSACTION;
SET time_zone = "+00:00";


/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Base de datos: `if0_42216592_jinhwa_corporation`
--

-- --------------------------------------------------------


--
-- Estructura de tabla para la tabla `administrador`
--

CREATE TABLE `administrador` (
  `id_administrador` int(11) NOT NULL,
  `nombre` varchar(100) NOT NULL,
  `apellido` varchar(100) NOT NULL,
  `correo` varchar(100) NOT NULL,
  `clave` varchar(255) NOT NULL,
  `foto_perfil` varchar(255) DEFAULT NULL,
  `permisos_extra` longtext CHARACTER SET utf8mb4 COLLATE utf8mb4_bin DEFAULT NULL
) ;
```

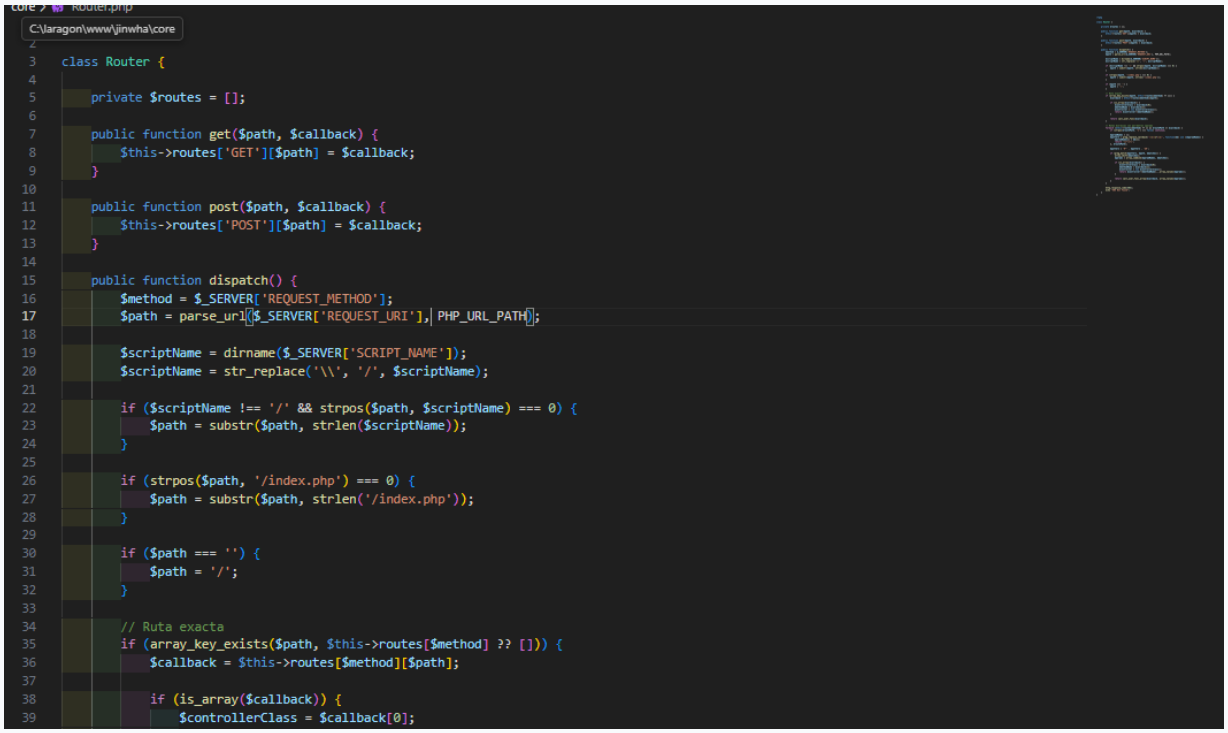
5. Núcleo del Framework (Core)

La carpeta core/ contiene las clases fundamentales que gobiernan la ejecución de la arquitectura MVC:

a. core/Router.php

Implementa el motor de enrutamiento web. Registra las rutas asociadas a verbos HTTP (GET y POST) y ejecuta la acción indicada cuando coincide con el URI pedido.

- **Método get($ruta, $handler):** Registra una ruta atendida bajo el método GET.
- **Método post($ruta, $handler):** Registra una ruta atendida bajo el método POST.
- **Método dispatch():** Extrae la ruta solicitada desde $_SERVER['REQUEST_URI'], remueve query parameters y ejecuta el método de la clase controladora. Si la ruta no existe, responde con código 404.



b. core/Controller.php

Es la clase base de la que heredan todos los controladores del sistema. Provee métodos utilitarios de respuesta y renderizado:

- **Método renderView($vista, $datos = []):** Usa extract($datos) para transformar los arreglos asociativos en variables PHP individuales y carga la vista solicitada dentro del layout maestro.
- **Método json($datos, $status = 200):** Envía encabezados application/json y serializa los datos con json_encode para respuestas asíncronas vía AJAX/Fetch.

- **Método redirect($url):** Redirecciona al cliente de forma segura utilizando encabezados HTTP Location.

core > Controller.php

```php
<?php

require_once __DIR__ . '/../helpers/url_helper.php';

class Controller {

    protected function view($view, $data = []) {
        extract($data);

        $viewPath = __DIR__ . "/../vistas/$view.php";

        if (file_exists($viewPath)) {
            require $viewPath;
        } else {
            die("View $view not found");
        }
    }

    protected function redirect($url) {
        header("Location: " . base_url($url));
        exit;
    }
}
```

c. core/Model.php

Clase abstracta base para la capa de acceso a datos. En su constructor inicializa la propiedad protegida $this->db utilizando la conexión centralizada proveniente de config/conexion.php.

config > conexion.php

```php
<?php

class Database {
    private static $instance = null;
    private $connection;

    private $host = 'sql113.infinityfree.com';
    private $user = 'if0_42216592';
    private $pass = 'IfK0M0n94NKpHQq';
    private $name = 'if0_42216592_jinhwa_corporation';

    private function __construct() {
        $this->connection = new mysqli($this->host, $this->user, $this->pass, $this->name);

        if ($this->connection->connect_error) {
            die("Connection failed: " . $this->connection->connect_error);
        }

        $this->connection->set_charset("utf8mb4");
        $this->connection->query("SET NAMES 'utf8mb4'");
        $this->connection->query("SET CHARACTER SET utf8mb4");
    }

    public static function getInstance() {
        if (!self::$instance) {
            self::$instance = new Database();
        }
        return self::$instance;
    }

    public function getConnection() {
        return $this->connection;
    }
}
```

d. core/Security.php

Componente de seguridad perimetral y control de acceso. Provee métodos estáticos esenciales:

- **verifyAdmin():** Comprueba que exista una sesión activa y que el rol corresponda a 'Administracion'. De lo contrario, redirige al login.
- **verifyMaestro():** Valida los privilegios del rol 'Maestros'.
- **verifyEstudiante():** Valida los privilegios del rol 'Deportistas'.
- **hashPassword($password):** Genera un hash criptográfico mediante password_hash($password, PASSWORD_BCRYPT).
- **verifyPassword($password, $hash):** Verifica la autenticidad de una contraseña frente a su hash con password_verify().
- **sanitize($data):** Sanitiza cadenas de entrada eliminando etiquetas HTML y caracteres maliciosos.

core > Security.php

```php
<?php

class Security {

    public static function initSession() {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    public static function verifySession() {
        self::initSession();

        if (!isset($_SESSION['id'])) {
            self::redirectLogin('no_session');
        }

        $timeout = 1800;
        if (isset($_SESSION['last_activity'])) {
            $inactivo = time() - $_SESSION['last_activity'];
            if ($inactivo > $timeout) {
                self::logout();
                self::redirectLogin('timeout');
            }
        }

        $_SESSION['last_activity'] = time();

        if (isset($_SESSION['user_agent'])) {
            if ($_SESSION['user_agent'] !== $_SERVER['HTTP_USER_AGENT']) {
                self::logout();
                self::redirectLogin('security');
            }
        }

        if (!isset($_SESSION['last_regeneration'])) {
            $_SESSION['last_regeneration'] = time();
        } else {
            $tiempo_desde_regeneracion = time() - $_SESSION['last_regeneration'];
```

e. core/Roles.php

Centraliza los identificadores de roles del sistema mediante constantes de clase:

- **Roles::ADMINISTRACION:** Identificador 'Administracion' para el personal directivo.
- **Roles::MAESTROS:** Identificador 'Maestros' para instructores y profesores.
- **Roles::ESTUDIANTES:** Identificador 'Deportistas' para los alumnos de la academia.

core > Roles.php

```php
<?php

class Roles {
    const ADMINISTRADOR = 'Administracion';
    const MAESTRO = 'Maestros';
    const PROFESOR = 'Profesores';
    const MONITOR = 'Monitores';
    const ESTUDIANTE = 'Deportistas';

    public static function esAdmin($rol_id) {
        return $rol_id == self::ADMINISTRADOR;
    }

    public static function esMaestro($rol_id) {
        return $rol_id == self::MAESTRO;
    }
}
```

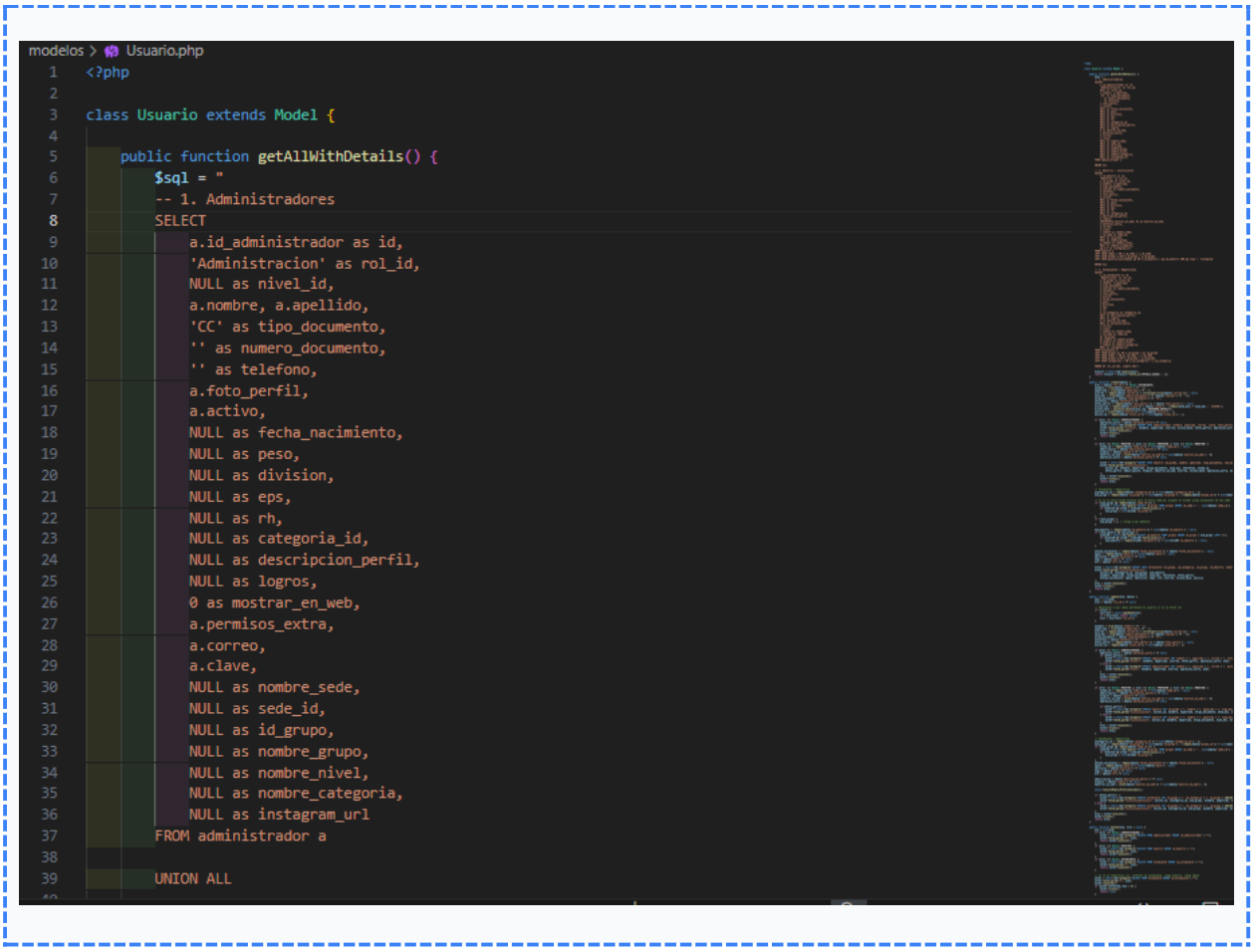
6. Modelos de Datos (Models)

Los modelos representan las entidades de la academia y centralizan las consultas SQL preparadas:

a. modelos/Usuario.php

Gestiona el ciclo de vida de los tres tipos de usuarios. Debido a la arquitectura de negocio, consulta de forma discriminada las tablas administrador, maestro y estudiante.

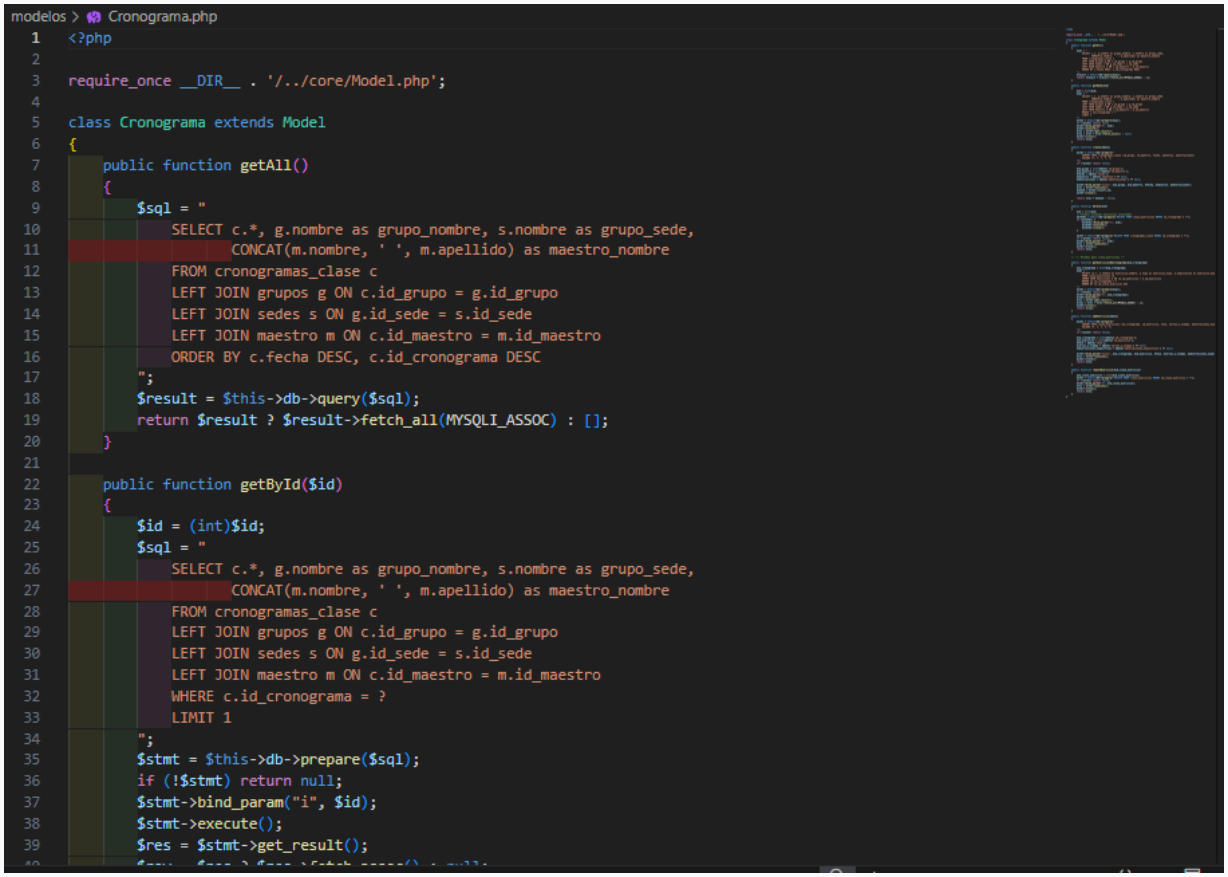
- **autenticar($email, $password):** Busca las credenciales del usuario en las tres tablas, verifica el hash de la contraseña y retorna la entidad con su rol correspondiente.
- **cambiarPassword($id, $rol, $nuevaPassword):** Actualiza la contraseña cifrada en la tabla específica del rol.
- **actualizarPerfil($id, $rol, $datos):** Modifica datos de contacto, foto o información personal.



b. modelos/Cronograma.php

Administra la programación de clases para los dojangs.

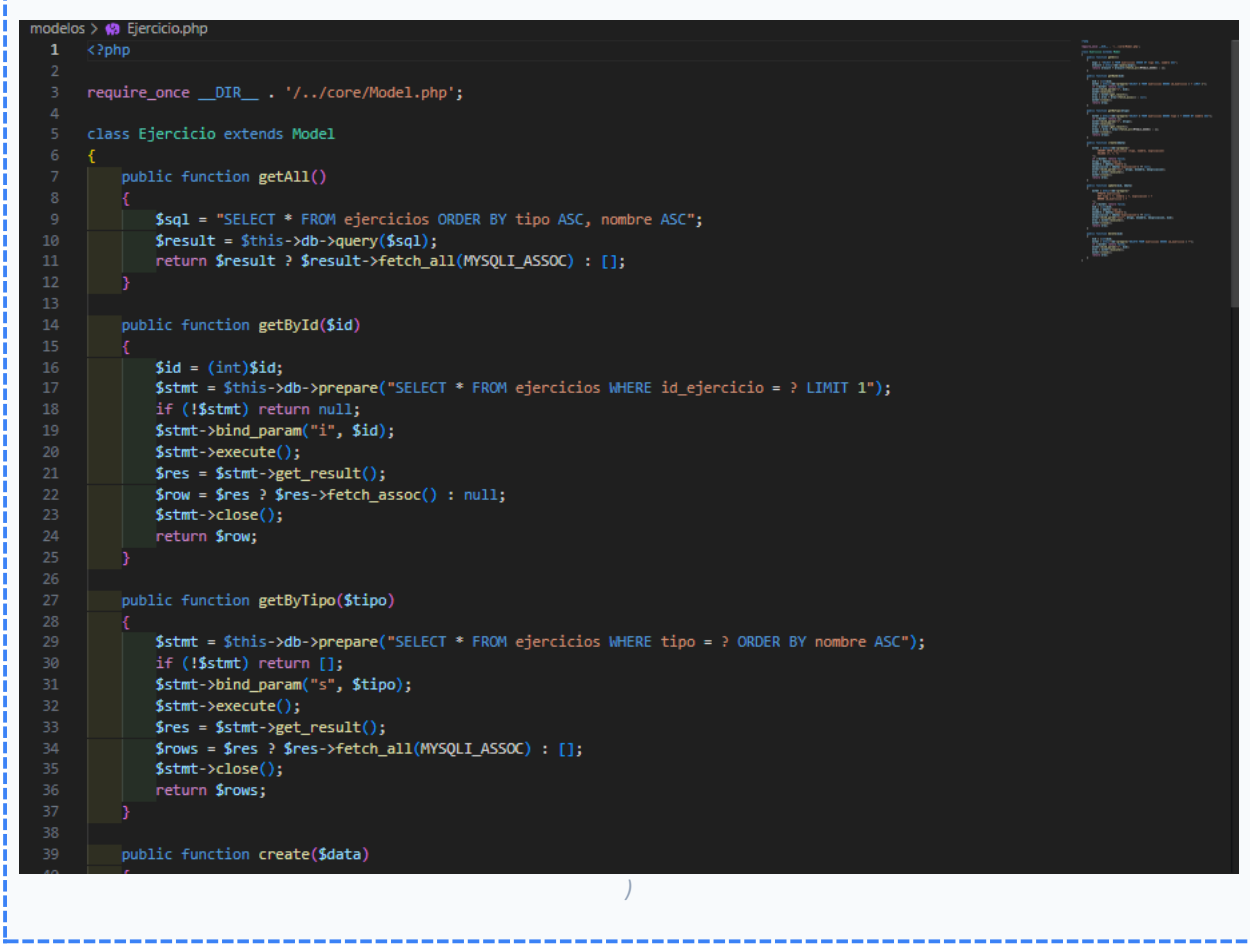
- **crearCronograma($datos):** Inserta un nuevo plan de clase en la tabla cronogramas_clase.
- **asignarEjercicios($idCronograma, $ejercicios):** Inserta las relaciones en clase_ejercicios indicando series y repeticiones.
- **obtenerPorGrupo($idGrupo):** Retorna el historial y próximas sesiones de un grupo específico.



c. modelos/Ejercicio.php

Contiene el catálogo técnico de artes marciales:

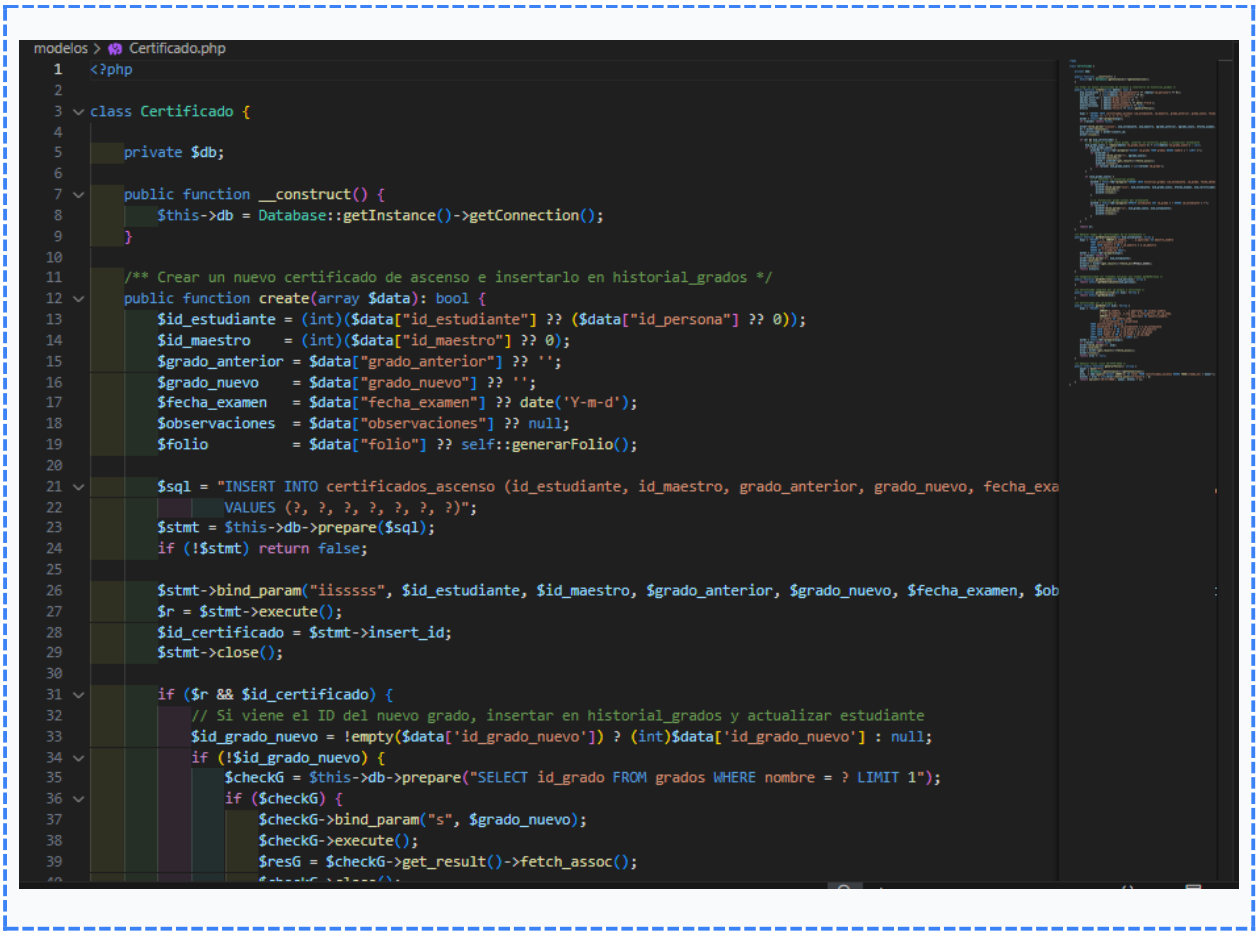
- **listarTodos($filtros = []):** Obtiene ejercicios con filtros opcionales de categoría (formas, combate, patadas) o nivel de dificultad.
- **crear($datos):** Registra una nueva técnica o ejercicio con sus directrices metodológicas.
- **actualizar($id, $datos) y eliminar($id):** Operaciones de mantenimiento técnico.



d. modelos/Certificado.php

Controla la expedición y validación de diplomas de ascenso marcial.

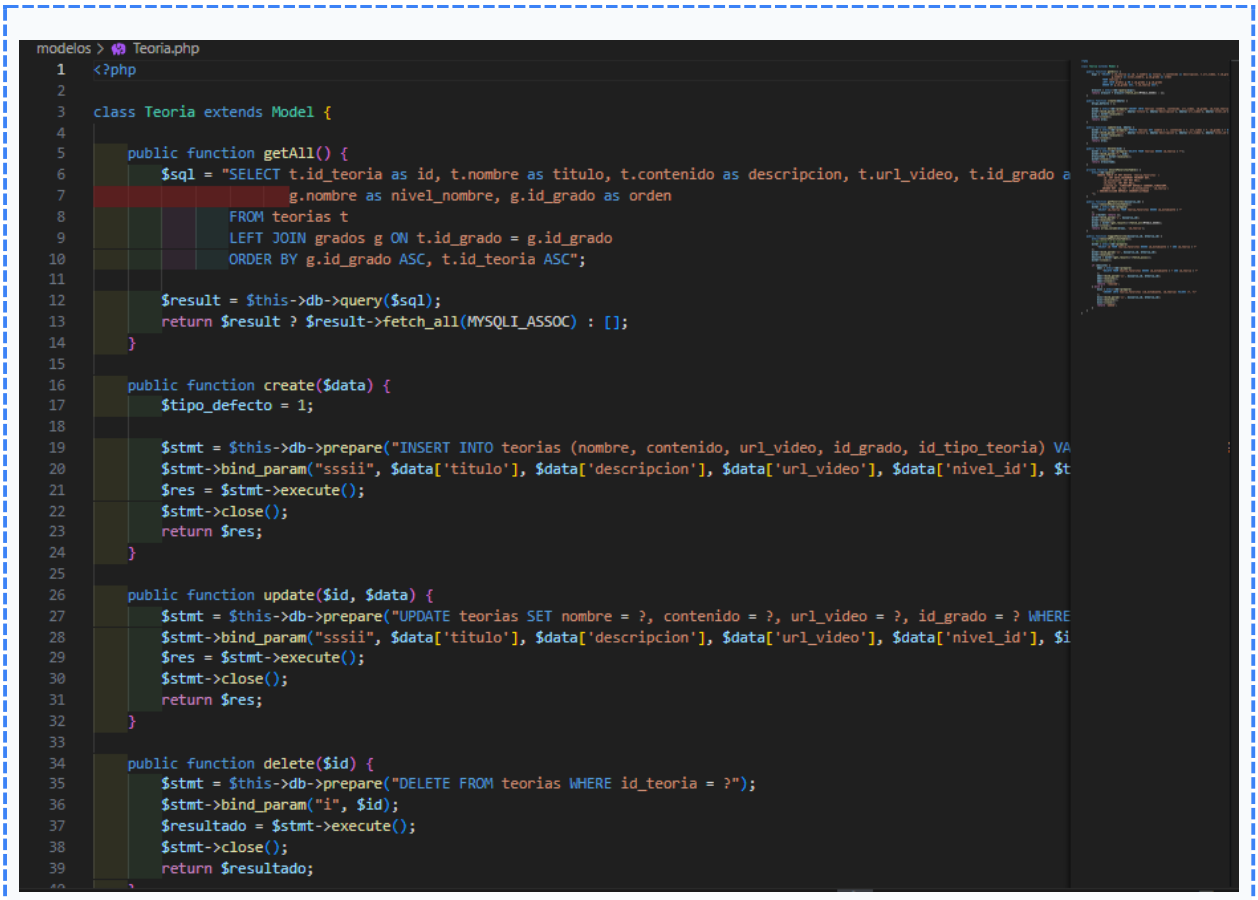
- **emitirCertificado($idEstudiante, $idGrado, $idMaestro, $folio):** Registra la emisión oficial del certificado.
- **obtenerPorEstudiante($idEstudiante):** Recupera la lista de certificaciones oficiales logradas por el alumno.



e. modelos/Teoria.php

Provee acceso a guías pedagógicas, historia marcial y terminología coreana según el cinturón del estudiante.

- **obtenerPorGrado($idGrado):** Consulta los módulos teóricos exigidos para el examen del grado seleccionado.
- **marcarCompletado($idEstudiante, $idTeoria):** Registra el avance del alumno en el estudio del material teórico.



f. modelos/Sede.php y modelos/Grupo.php

Modelan la estructura física y organizativa de la corporación:

- **Sede::listarActivas():** Retorna las escuelas y sucursales disponibles.
- **Grupo::obtenerPorSede($idSede):** Retorna los horarios y grupos de entrenamiento vinculados.

modelos > Sede.php

```php
<?php

class Sede extends Model {

    public function getAll() {
        $sql = "SELECT s.id_sede as id, s.id_sede, s.nombre, s.direccion, s.telefono, s.horario, s.activo,
                       COUNT(DISTINCT e.id_estudiante) as numero_estudiantes,
                       COUNT(DISTINCT g.id_grupo) as numero_grupos
                FROM sedes s
                LEFT JOIN grupos g ON s.id_sede = g.id_sede
                LEFT JOIN estudiante e ON g.id_grupo = e.id_grupo AND e.activo = 1
                GROUP BY s.id_sede, s.nombre, s.direccion, s.telefono, s.horario, s.activo
                ORDER BY s.id_sede ASC";

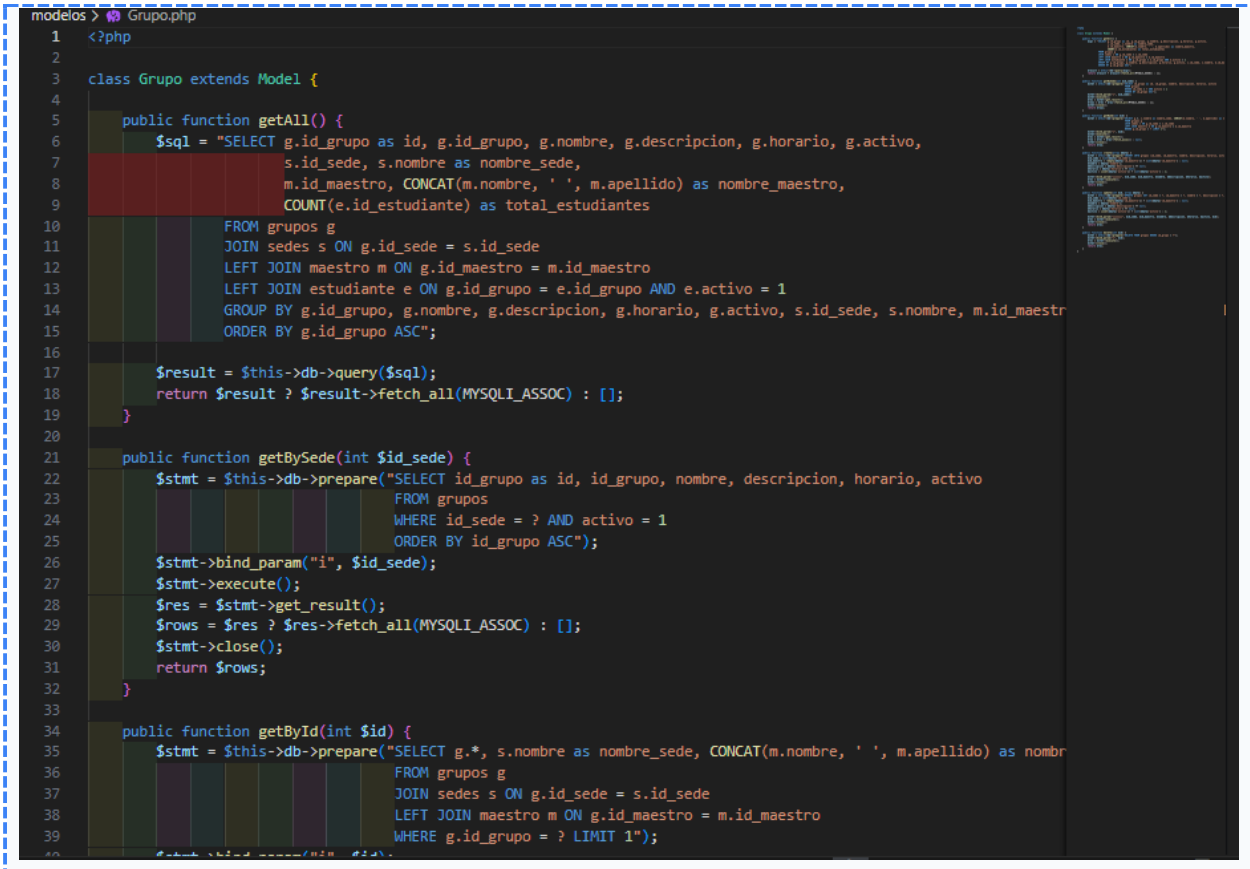
        $result = $this->db->query($sql);
        return $result ? $result->fetch_all(MYSQLI_ASSOC) : [];
    }

    public function create($data) {
        $horario = $data['horario'] ?? null;
        $telefono = $data['telefono'] ?? null;
        $direccion = $data['direccion'] ?? null;
        $email = $data['email'] ?? null;

        $stmt = $this->db->prepare("INSERT INTO sedes (nombre, direccion, telefono, email, horario) VALUES (?, ?, ?
        $stmt->bind_param("sssss", $data['nombre'], $direccion, $telefono, $email, $horario);
        $res = $stmt->execute();
        $stmt->close();
        return $res;
    }

    public function update($id, $data) {
        $horario = $data['horario'] ?? null;
        $telefono = $data['telefono'] ?? null;
        $direccion = $data['direccion'] ?? null;
        $email = $data['email'] ?? null;

        $stmt = $this->db->prepare("UPDATE sedes SET nombre = ?, direccion = ?, telefono = ?, email = ?, horario =
        $stmt->bind_param("ssssssi", $data['nombre'], $direccion, $telefono, $email, $horario, $id);
```



7. Controladores y Lógica de Negocio

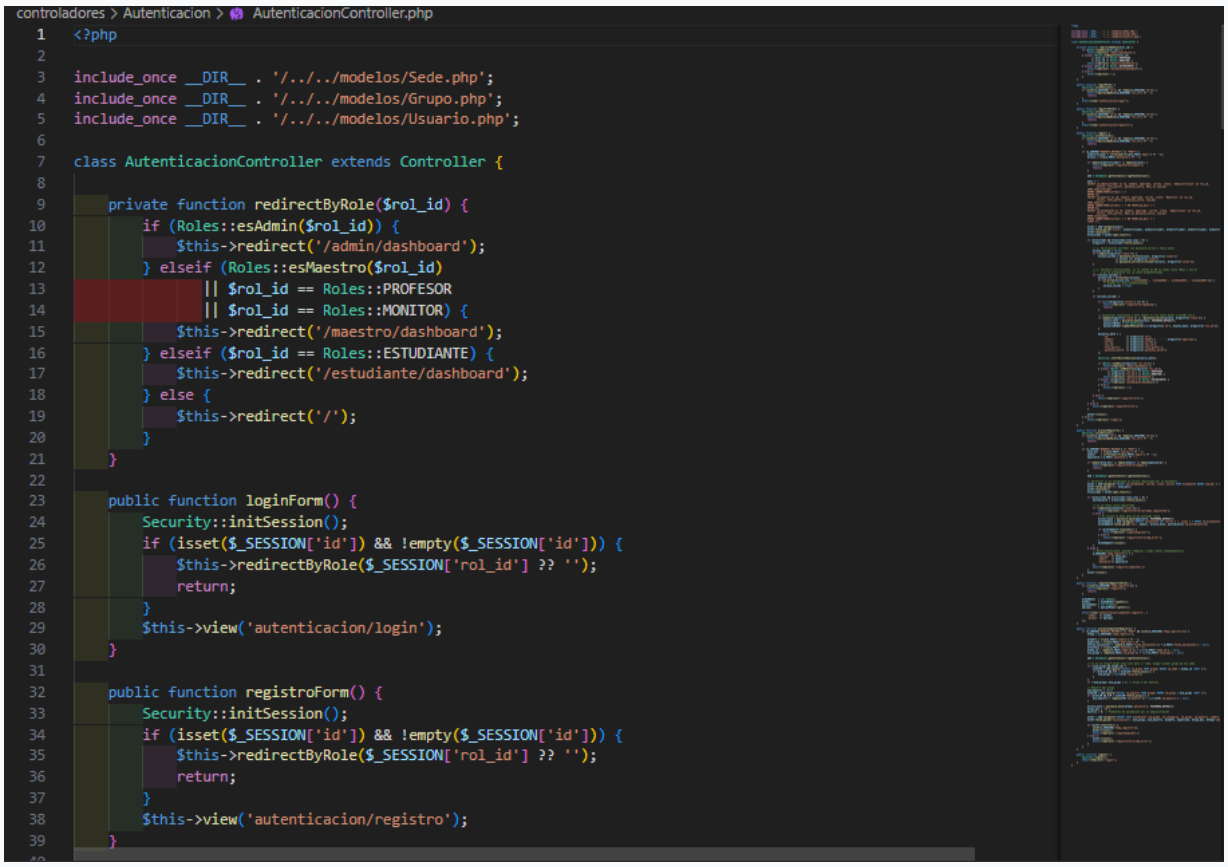
Los controladores organizan las acciones del usuario, validan las peticiones y orquestan la comunicación entre modelos y vistas.

a. Módulo de Autenticación (controladores/Autenticacion/AutenticacionController.php)

Controla el ciclo de vida de la sesión del usuario:

- **login():** Muestra el formulario de inicio de sesión.
- **autenticar():** Recibe email y password por POST. Llama a Usuario::autenticar(), almacena el id y rol en $_SESSION y redirige al dashboard correspondiente (admin, maestro o estudiante).
- **logout():** Destruye la sesión, elimina cookies de autenticación y redirige a la página principal.

• **Qué debe capturarse:** Captura del flujo de login, verificación de credenciales y despacho condicional por rol.



b. Módulo de Administración (controladores/Administracion/)

Controladores protegidos por Security::verifyAdmin():

- **DashboardController.php:** Calcula métricas clave: total de estudiantes activos, maestros vinculados, sedes y ascensos del mes.
- **EstudiantesController.php:** CRUD completo de alumnos con soporte de paginación, filtros por sede y cinturón, y exportación de expedientes.
- **MaestrosController.php:** Gestión de profesores, asignación a dojangs y reseteo administrativo de credenciales.
- **AscensosController.php:** Revisión y aprobación de postulaciones de examen técnico, culminando en la generación del diploma.
- **SedesController.php y GruposController.php:** Administración de sedes físicas, cupos y horarios.
- **TeoriaController.php:** Carga y edición de módulos teóricos y material de estudio.

• **Qué debe capturarse:** Captura de EstudiantesController.php mostrando los métodos index(), crear(), guardar() y paginar().

(Haga clic aquí y pegue [Ct

controladores > Administracion > EstudiantesController.php

```php
<?php

include_once __DIR__ . '/../../modelos/Usuario.php';
include_once __DIR__ . '/../../modelos/Sede.php';
include_once __DIR__ . '/../../modelos/Grupo.php';
include_once __DIR__ . '/../../modelos/Nivel.php';
include_once __DIR__ . '/../../modelos/Categoria.php';
include_once __DIR__ . '/../../modelos/MultimediaGaleria.php';

class AdminEstudiantesController extends Controller {

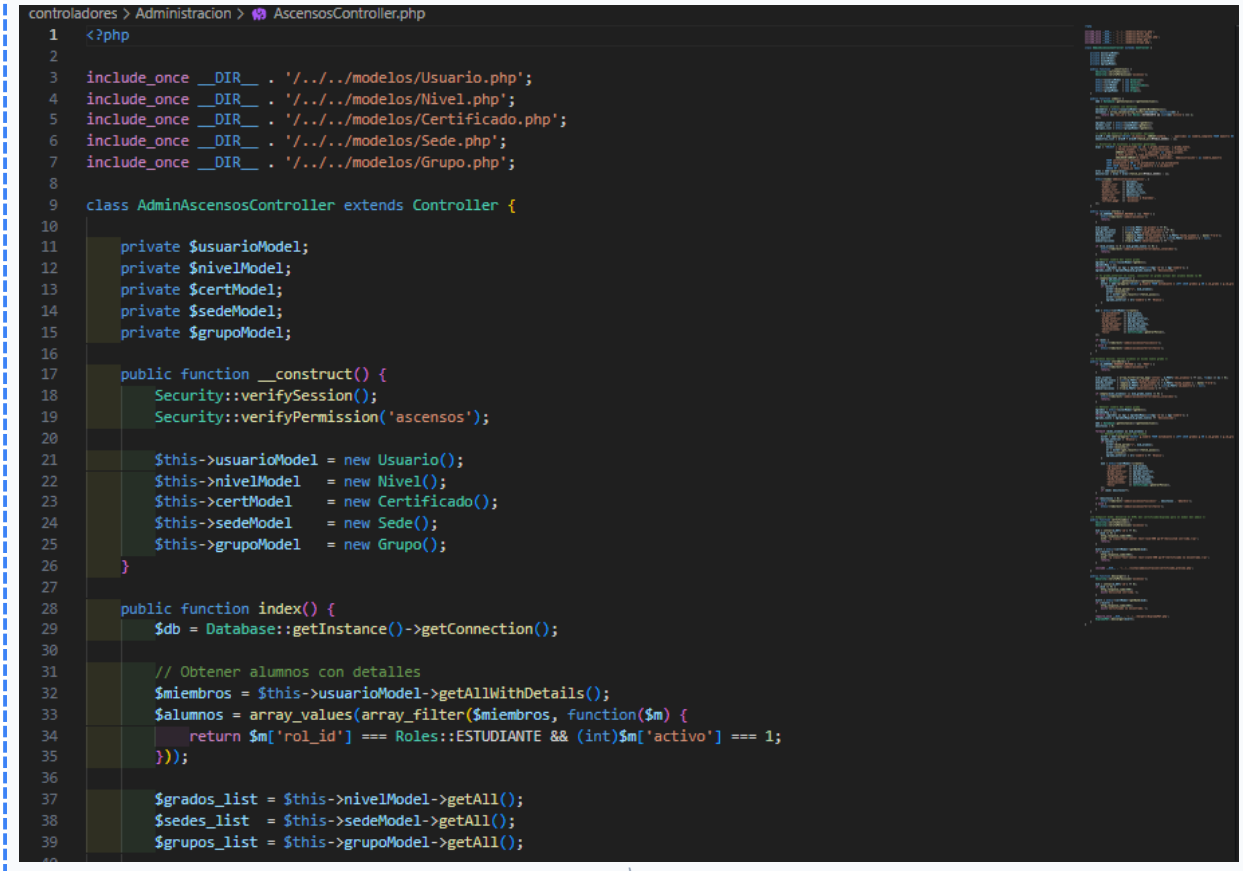
    private $usuarioModel;
    private $sedeModel;
    private $grupoModel;
    private $nivelModel;
    private $categoriaModel;

    public function __construct() {
        Security::verifySession();
        Security::verifyAdmin();

        $this->usuarioModel   = new Usuario();
        $this->sedeModel      = new Sede();
        $this->grupoModel     = new Grupo();
        $this->nivelModel     = new Nivel();
        $this->categoriaModel = new Categoria();
    }

    public function index() {
        $db = Database::getInstance()->getConnection();

        // Solo estudiantes
        $sql = "SELECT e.id_estudiante as id, e.nombre, e.apellido, e.correo, e.telefono,
                       e.num_doc as numero_documento, e.tipo_documento, e.fecha_nacimiento,
                       e.foto_perfil, e.activo,
                       e.peso, e.division, e.eps, e.rh,
                       g.nombre as grupo_nombre, g.id_grupo,
                       gr.nombre as grado_nombre, gr.id_grado,
                       c.nombre as categoria_nombre,
```

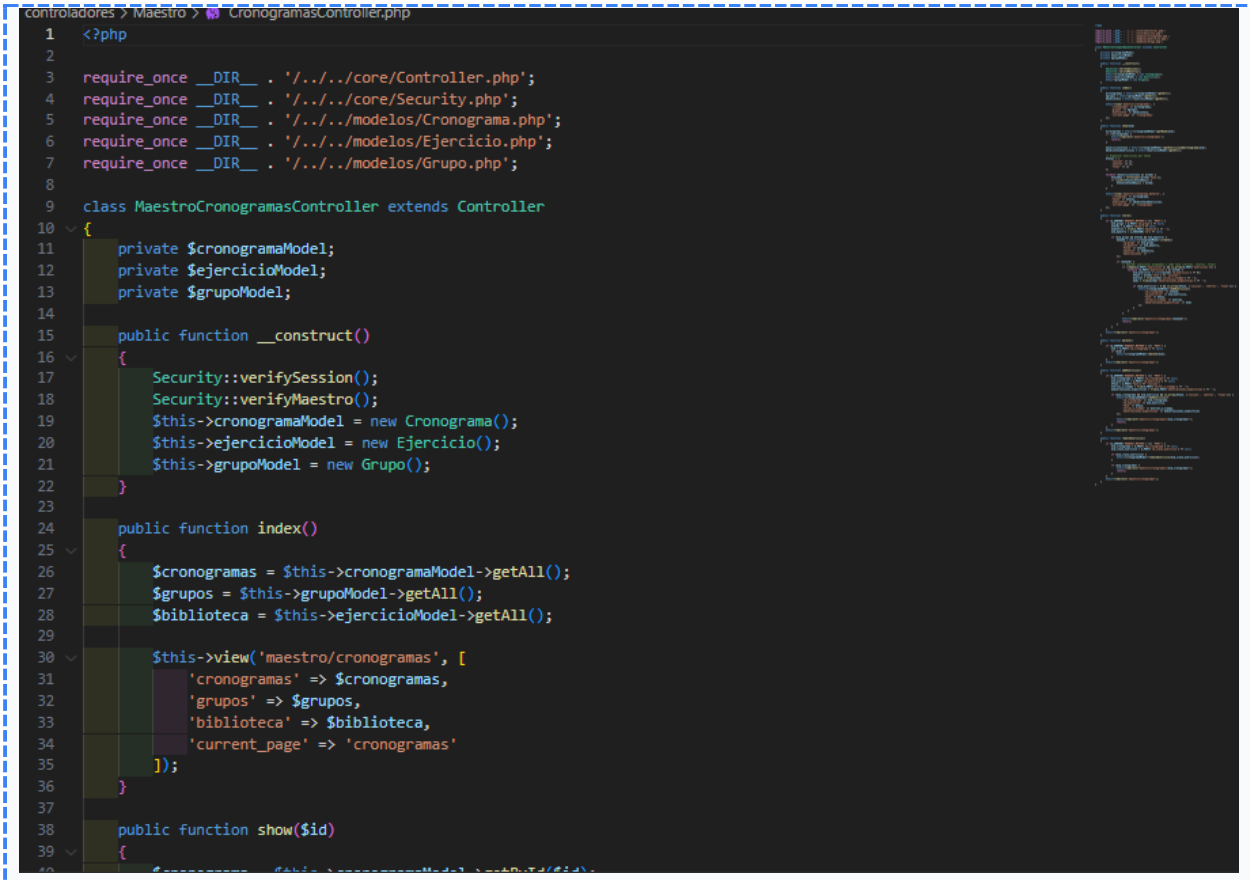


c. Módulo de Maestro (controladores/Maestro/)

Controladores protegidos por Security::verifyMaestro():

- **DashboardController.php:** Panel del instructor con sus próximas clases asignadas y accesos directos.
- **CronogramasController.php:** Diseño interactivo de sesiones: asignación de técnicas, series, tiempos y orden de ejecución.
- **EjerciciosController.php:** Creación y clasificación de ejercicios técnicos para la biblioteca de la academia.
- **AlumnosController.php:** Seguimiento individual de asistencia, rendimiento técnico y preparación para examen.
- **SolicitudesAscensoController.php:** Postulación formal de un deportista preparado para ser evaluado.
- **IAController.php:** Módulo de Inteligencia Artificial que se comunica con la API de OpenRouter para generar recomendaciones de entrenamiento adaptadas a la categoría del estudiante.

controladores > Maestro > CronogramasController.php



```php
<?php

require_once __DIR__ . '/../../core/Controller.php';
require_once __DIR__ . '/../../core/Security.php';
require_once __DIR__ . '/../../modelos/Cronograma.php';
require_once __DIR__ . '/../../modelos/Ejercicio.php';
require_once __DIR__ . '/../../modelos/Grupo.php';

class MaestroCronogramasController extends Controller
{
    private $cronogramaModel;
    private $ejercicioModel;
    private $grupoModel;

    public function __construct()
    {
        Security::verifySession();
        Security::verifyMaestro();
        $this->cronogramaModel = new Cronograma();
        $this->ejercicioModel = new Ejercicio();
        $this->grupoModel = new Grupo();
    }

    public function index()
    {
        $cronogramas = $this->cronogramaModel->getAll();
        $grupos = $this->grupoModel->getAll();
        $biblioteca = $this->ejercicioModel->getAll();

        $this->view('maestro/cronogramas', [
            'cronogramas' => $cronogramas,
            'grupos' => $grupos,
            'biblioteca' => $biblioteca,
            'current_page' => 'cronogramas'
        ]);
    }

    public function show($id)
    {
```

C:\laragon\www\jinwha\controladores\Maestro\IAController.php

```php
<?php

require_once __DIR__ . '/../../core/Controller.php';
require_once __DIR__ . '/../../core/Security.php';
require_once __DIR__ . '/../../config/ia.php';
require_once __DIR__ . '/../../modelos/Ejercicio.php';
require_once __DIR__ . '/../../modelos/Grupo.php';

class MaestroIAController extends Controller
{
    private $ejercicioModel;
    private $grupoModel;

    public function __construct()
    {
        Security::verifySession();
        Security::verifyMaestro();
        $this->ejercicioModel = new Ejercicio();
        $this->grupoModel = new Grupo();
    }

    /**
     * Endpoint POST /maestro/ia/chat
     * Procesa mensajes del profesor y consulta NVIDIA NIM
     */
    public function chat()
    {
        header('Content-Type: application/json; charset=utf-8');

        if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
            echo json_encode(['success' => false, 'error' => 'Método no permitido']);
            exit;
        }

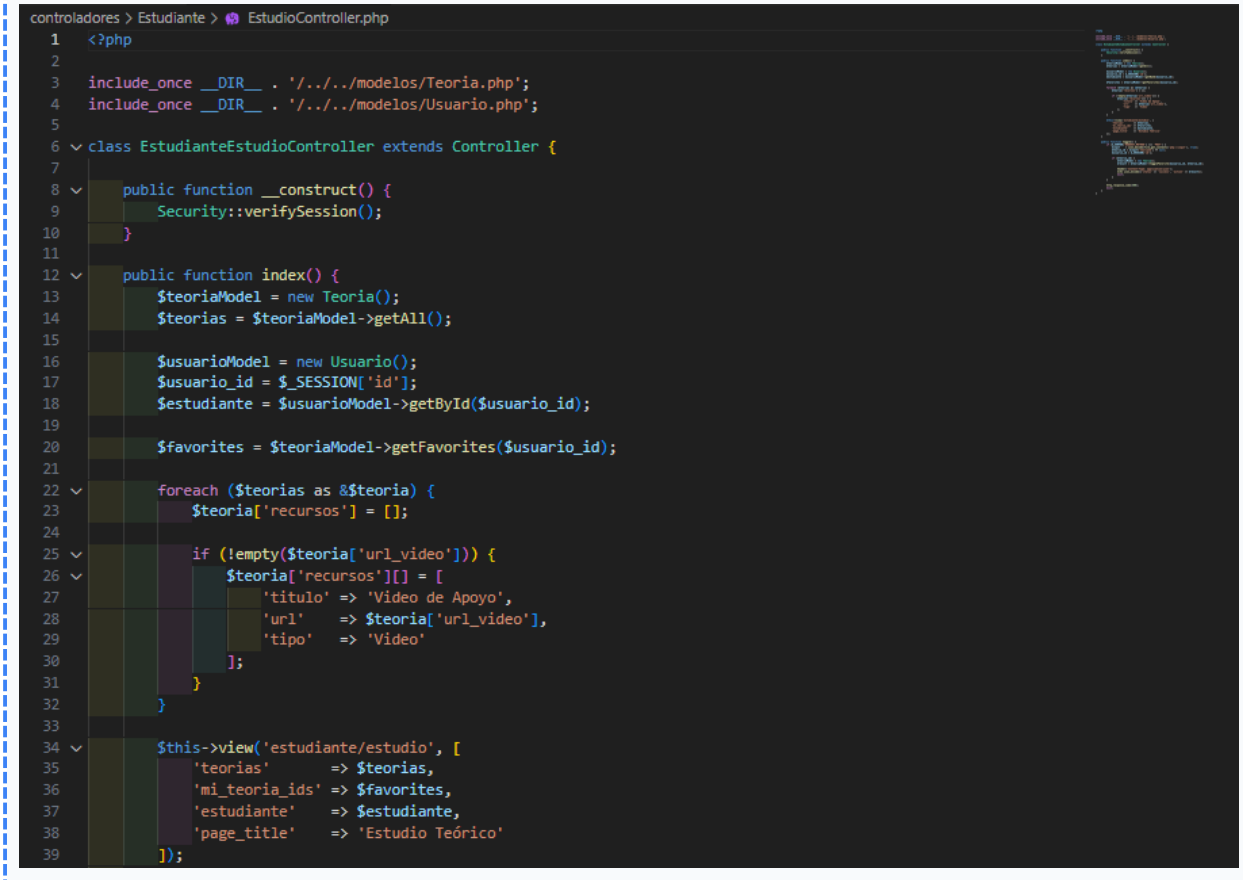
        $rawInput = file_get_contents('php://input');
        $input = json_decode($rawInput, true);

        $mensaje = trim($input['mensaje'] ?? '');
        $contexto = trim($input['contexto'] ?? 'cronogramas'); // 'cronogramas' | 'ejercicios'
```

d. Módulo de Estudiante (controladores/Estudiante/)

Controladores para deportistas protegidos por Security::verifyEstudiante():

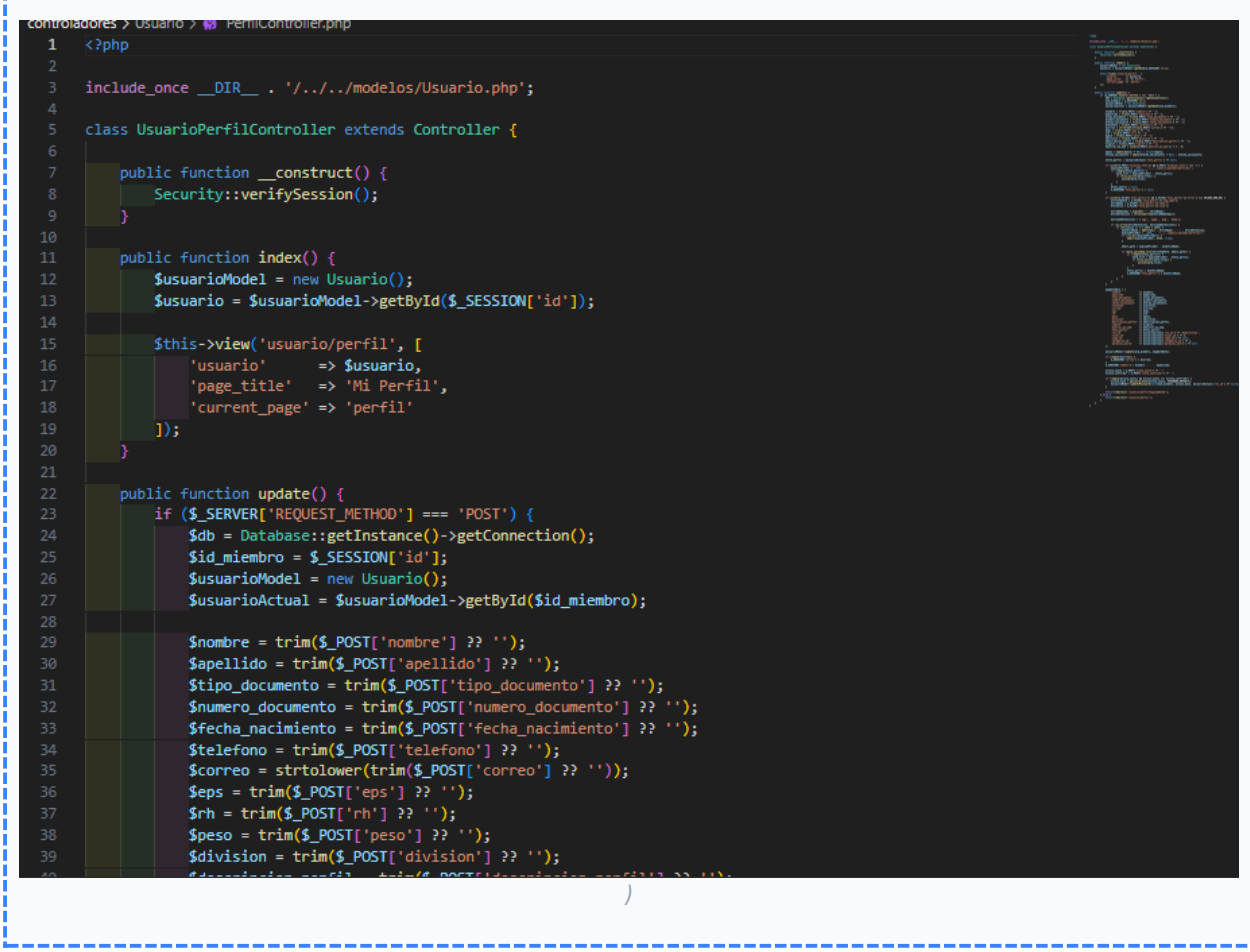
- **DashboardController.php:** Muestra el grado actual del alumno, color de cinturón, progreso de asistencias y calendario.
- **EstudioController.php:** Acceso interactivo a las guías teóricas, terminología en coreano y videos correspondientes a su cinturón.
- **HistorialController.php:** Visualización de exámenes aprobados y descarga de certificados digitales oficiales.



e. Módulo de Usuario y Portal Web Público

Controladores transversales y accesibles públicamente:

- **Usuario/PerfilController.php:** Permite a cualquier usuario autenticado cambiar su foto de perfil, actualizar teléfono y modificar su contraseña de forma segura.
- **Web/InicioController.php, SedesController.php, etc.:** Portal institucional público con información de la academia, dojangs, catálogo de maestros y formulario de contacto.

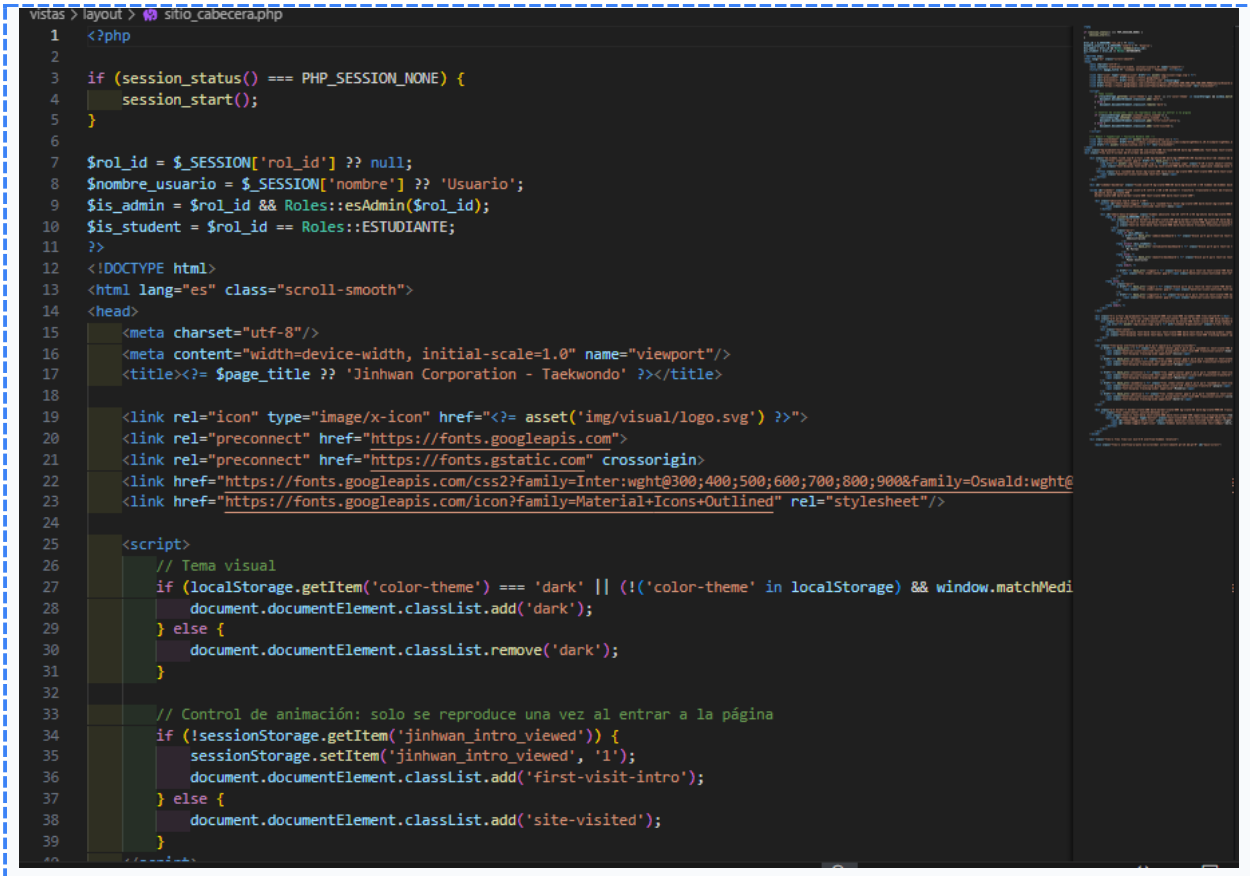


8. Vistas y Componentes del Frontend

Las vistas del sistema están estructuradas de forma modular en la carpeta vistas/:

- **Layouts Maestros (vistas/layouts/):** header.php (metadatos, fuentes, Tailwind CSS), navbar.php (barra de navegación superior adaptativa), sidebar.php (menú lateral dinámico según rol) y footer.php (scripts y cierre de etiquetas).
- **Vistas por Rol:** vistas/administracion/, vistas/maestro/, vistas/estudiante/ y vistas/web/. Cada vista consume las variables inyectadas por el controlador sin mezclar lógica de negocio.
- **Interactividad y Alertas:** Uso de SweetAlert2 para mensajes de confirmación (ej. al eliminar un ejercicio o aprobar un ascenso) y peticiones Fetch API para evitar recargas completas de pantalla.

vistas > layout > sitio_cabecera.php



```php
<?php

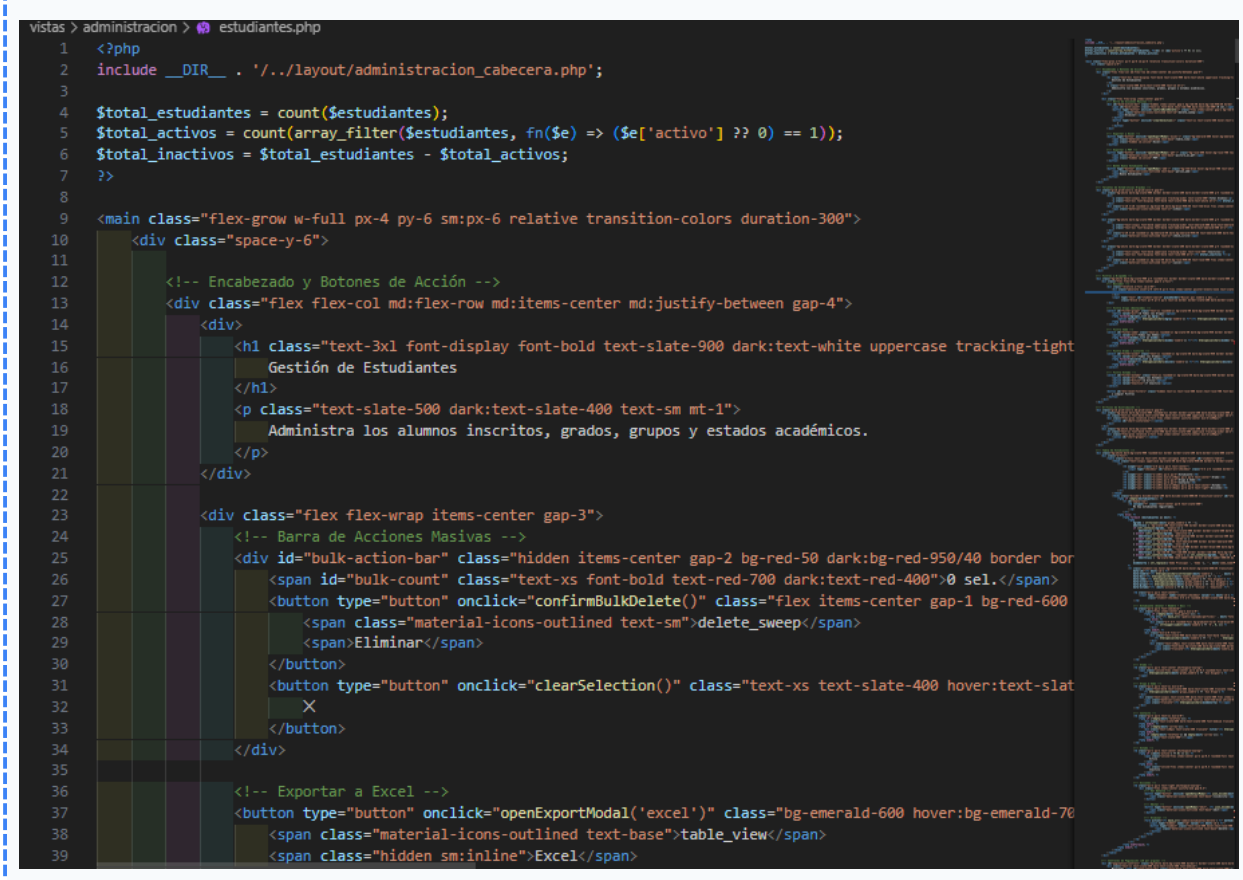
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

$rol_id = $_SESSION['rol_id'] ?? null;
$nombre_usuario = $_SESSION['nombre'] ?? 'Usuario';
$is_admin = $rol_id && Roles::esAdmin($rol_id);
$is_student = $rol_id == Roles::ESTUDIANTE;
?>
<!DOCTYPE html>
<html lang="es" class="scroll-smooth">
<head>
    <meta charset="utf-8"/>
    <meta content="width=device-width, initial-scale=1.0" name="viewport"/>
    <title><?= $page_title ?? 'Jinhwan Corporation - Taekwondo' ?></title>

    <link rel="icon" type="image/x-icon" href="<?= asset('img/visual/logo.svg') ?>">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&family=Oswald:wght@
    <link href="https://fonts.googleapis.com/icon?family=Material+Icons+Outlined" rel="stylesheet"/>

    <script>
        // Tema visual
        if (localStorage.getItem('color-theme') === 'dark' || (!('color-theme' in localStorage) && window.matchMedi
            document.documentElement.classList.add('dark');
        } else {
            document.documentElement.classList.remove('dark');
        }

        // Control de animación: solo se reproduce una vez al entrar a la página
        if (!sessionStorage.getItem('jinhwan_intro_viewed')) {
            sessionStorage.setItem('jinhwan_intro_viewed', '1');
            document.documentElement.classList.add('first-visit-intro');
        } else {
            document.documentElement.classList.add('site-visited');
        }
```

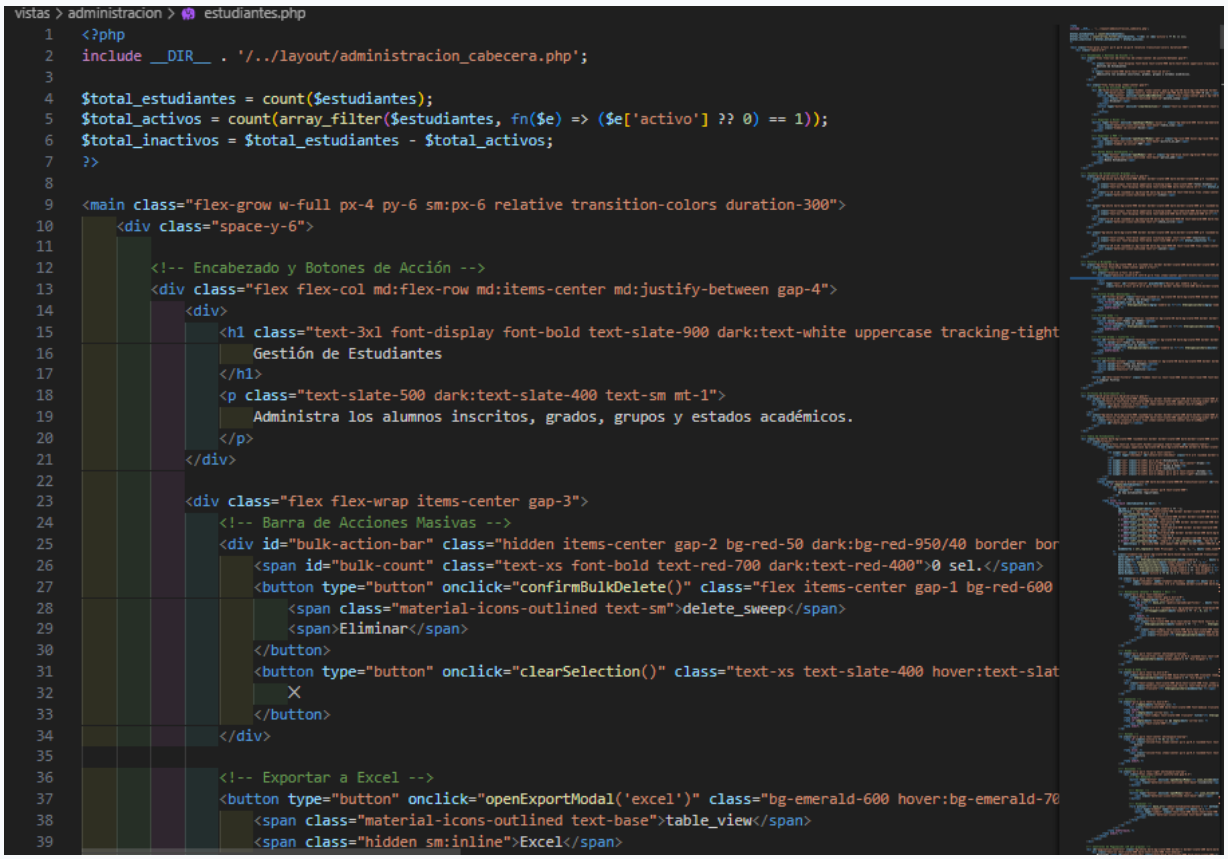


9. Casos de Uso Críticos y Flujo de Datos

Caso de Uso 1: Autenticación, Verificación Criptográfica y Redirección

Flujo de ejecución paso a paso:

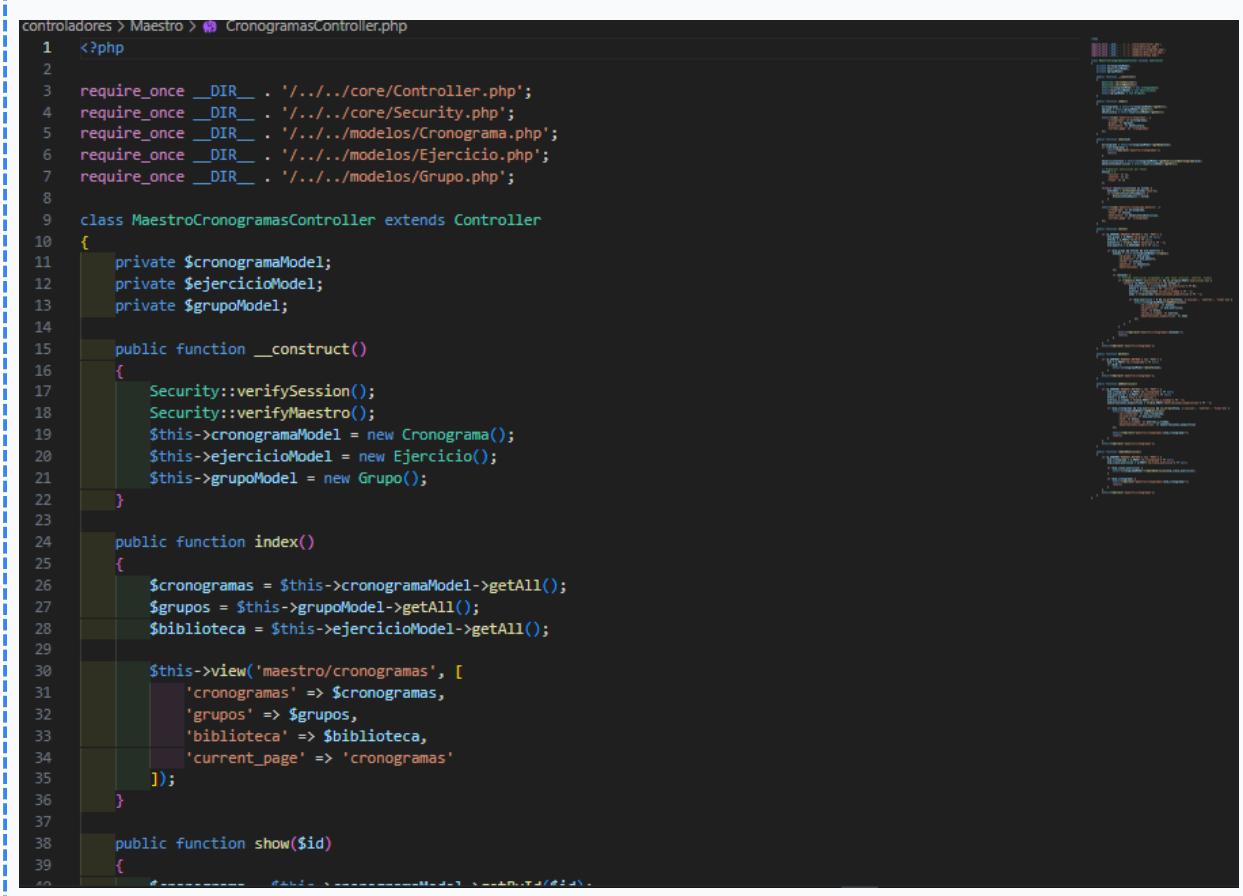
- **Paso 1:** El usuario ingresa su correo y contraseña en /inicio o /login.
- **Paso 2:** AutenticacionController::autenticar() captura los datos por POST y sanitiza las entradas.
- **Paso 3:** Usuario::autenticar() busca al usuario en las tablas administrador, maestro y estudiante mediante consultas preparadas.
- **Paso 4:** Security::verifyPassword() compara la contraseña en texto plano contra el hash Bcrypt almacenado.
- **Paso 5:** Si es válido, se inicializa la sesión con el ID, nombre y rol del usuario, redirigiendo a su dashboard correspondiente.



Caso de Uso 2: Planificación de Clase y Asignación de Ejercicios Técnicos

Flujo de ejecución paso a paso:

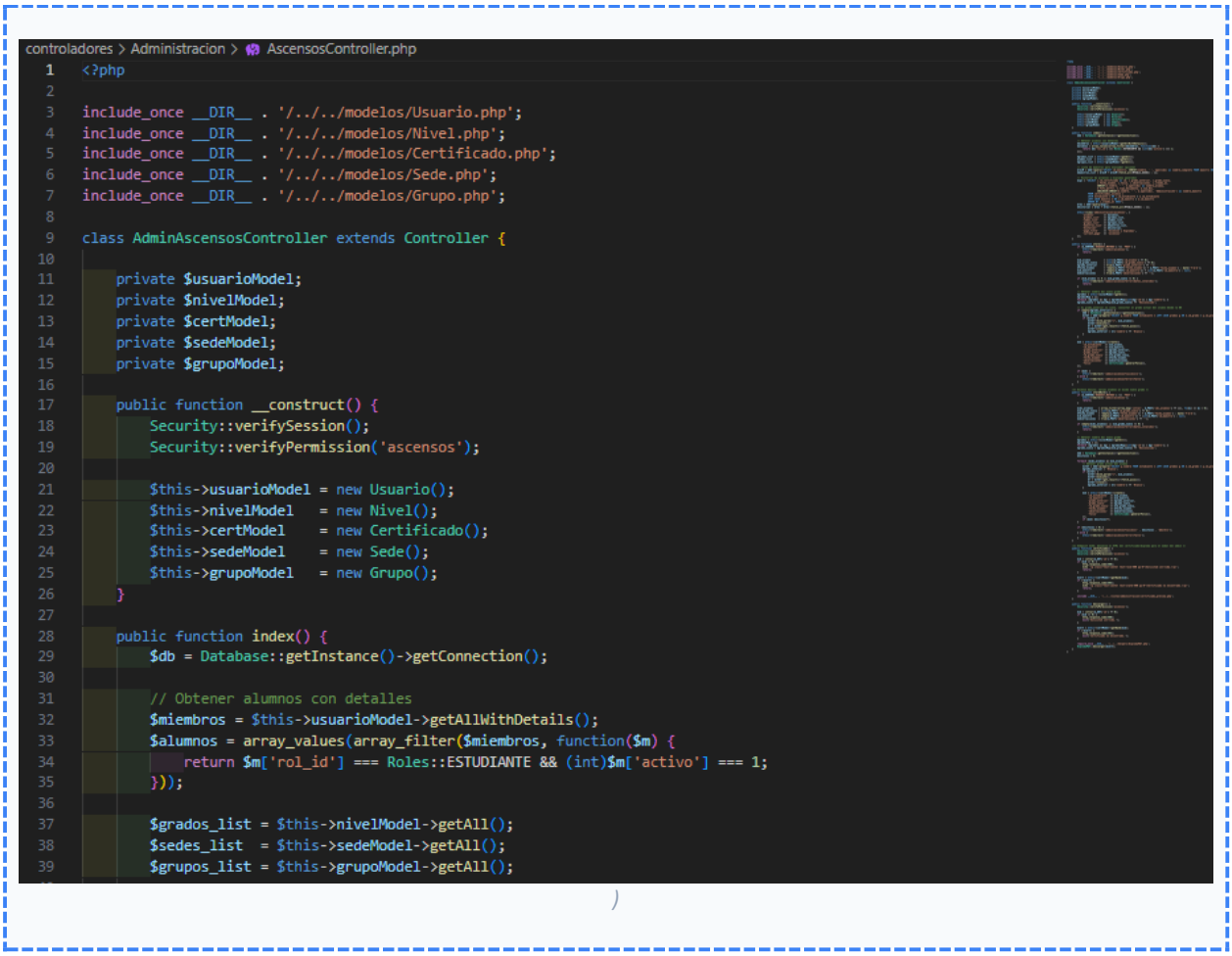
- **Paso 1:** El maestro ingresa al módulo de cronogramas y selecciona fecha, grupo y objetivo pedagógico.
- **Paso 2:** CronogramasController::crear() registra la cabecera de la sesión en cronogramas_clase.
- **Paso 3:** El maestro selecciona técnicas del catálogo de Ejercicio.php (Poomsae, Kyorugi, técnicas de pateo).
- **Paso 4:** Se persisten los ejercicios en la tabla clase_ejercicios vinculados con la sesión.



Caso de Uso 3: Postulación de Examen, Aprobación y Emisión de Certificado

Flujo de ejecución paso a paso:

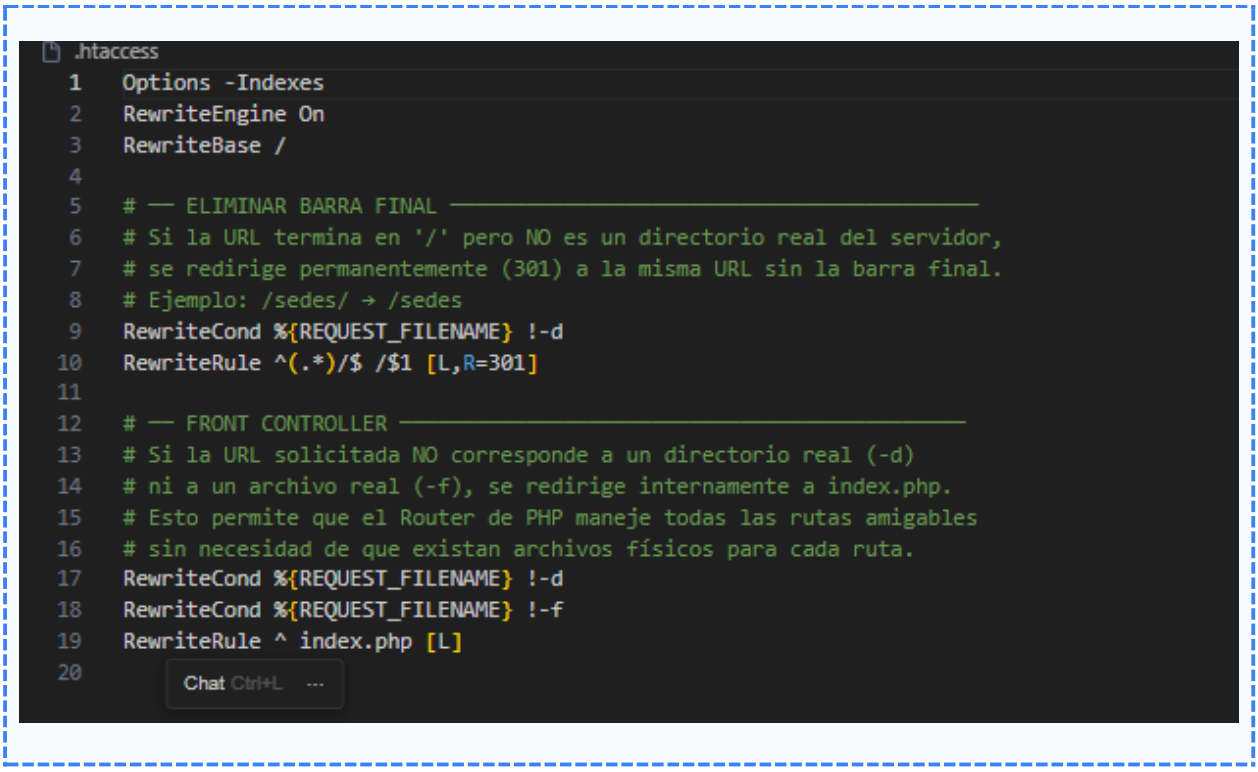
- **Paso 1:** El maestro verifica que el alumno cumpla con las asistencias y técnicas requeridas y emite la postulación.
- **Paso 2:** El administrador visualiza la postulación en AscensosController.php.
- **Paso 3:** Al aprobar el examen, el sistema actualiza id_grado en la tabla estudiante, registra el ascenso en historial_grados y crea el diploma en certificados_ascenso.
- **Paso 4:** El estudiante puede visualizar inmediatamente su nuevo cinturón en su Dashboard y descargar su certificado digital.



10. Guía de Instalación y Mantenimiento Técnico

Para desplegar el sistema en un nuevo entorno de desarrollo o producción, siga las siguientes instrucciones:

- **Requisitos de Software:** Servidor web Apache con módulo mod_rewrite habilitado, PHP versión 8.1 o superior (con extensiones mysqli, pdo, curl, mbstring), MySQL versión 5.7 o MariaDB 10.4+.
- **Clonación y Ubicación:** Copiar los archivos en el directorio web (ej. c:\laragon\www\jinwha en Windows o /var/www/html/jinwha en Linux).
- **Importación de Base de Datos:** Crear una base de datos con cotejamiento utf8mb4_unicode_ci e importar el script database/if0_42216592_jinhwa_corporation.sql.
- **Ajuste de Credenciales:** Editar config/conexion.php especificando host, usuario, contraseña y nombre de la base de datos.
- **Configuración del Virtual Host / Apache:** Asegurar que las directivas AllowOverride All estén activas en httpd.conf para permitir que el archivo .htaccess redirija el tráfico a index.php.

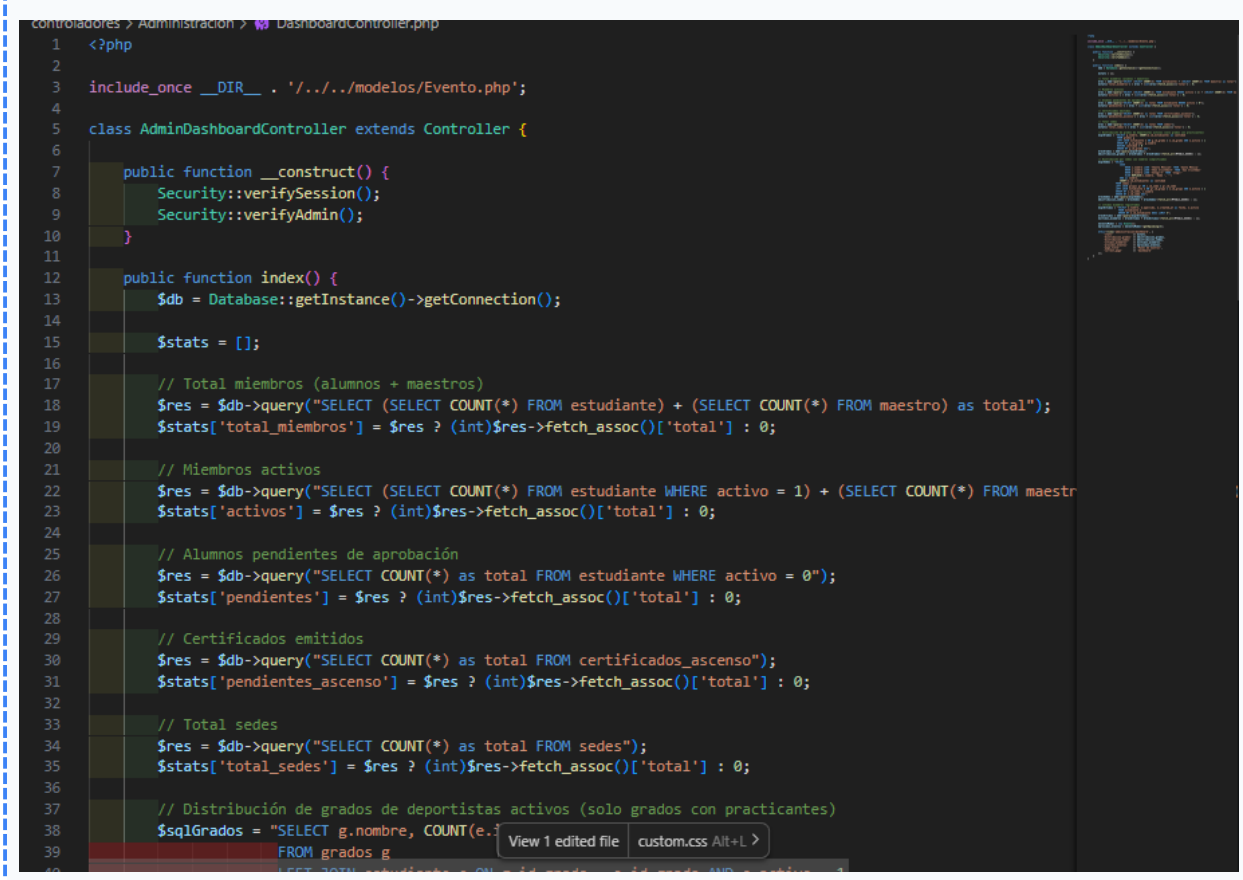


11. Controladores Restantes del Módulo de Administración

En este capítulo se documentan todos los controladores de administración que complementan las operaciones directivas del dojang y que no habían sido capturados previamente:

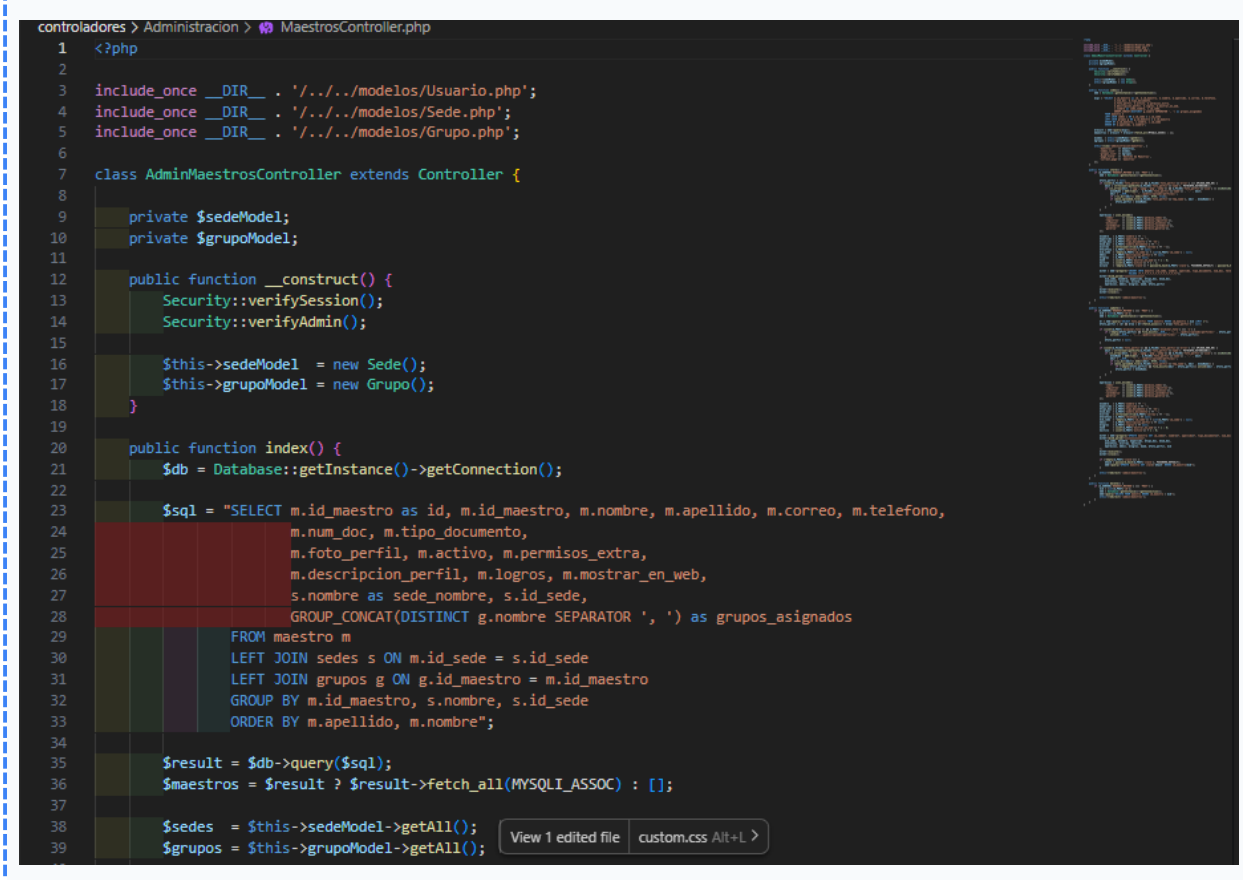
11.1 DashboardController.php (Administración)

- **Propósito:** Calcula y presenta las estadísticas consolidadas de la academia (total alumnos activos, profesores vinculados y alertas).



11.2 MaestrosController.php

- **Propósito:** Gestión integral del cuerpo docente: altas, asignación a sedes físicas, especialidad técnica y credenciales.




```php
<?php

include_once __DIR__ . '/../../modelos/Usuario.php';
include_once __DIR__ . '/../../modelos/Sede.php';
include_once __DIR__ . '/../../modelos/Grupo.php';

class AdminMaestrosController extends Controller {

    private $sedeModel;
    private $grupoModel;

    public function __construct() {
        Security::verifySession();
        Security::verifyAdmin();

        $this->sedeModel  = new Sede();
        $this->grupoModel = new Grupo();
    }

    public function index() {
        $db = Database::getInstance()->getConnection();

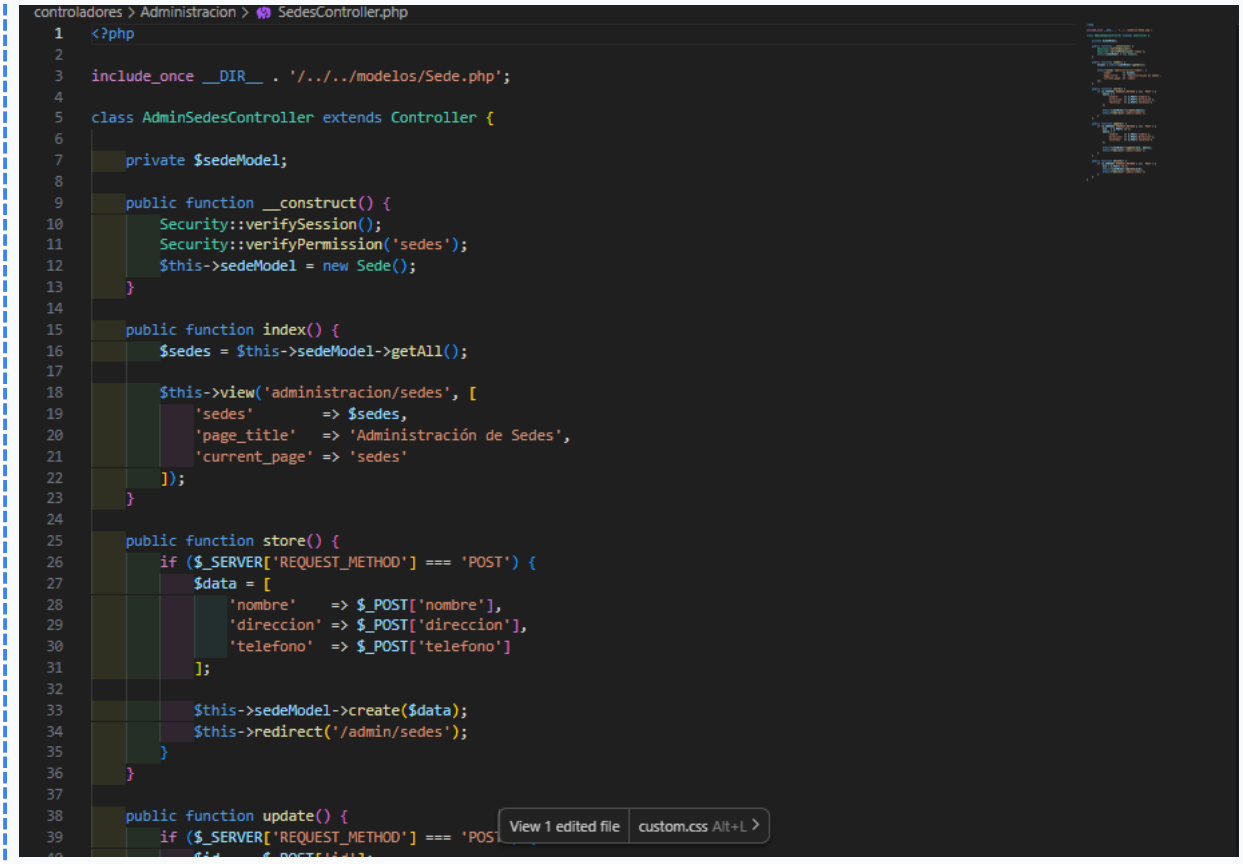
        $sql = "SELECT m.id_maestro as id, m.id_maestro, m.nombre, m.apellido, m.correo, m.telefono,
                       m.num_doc, m.tipo_documento,
                       m.foto_perfil, m.activo, m.permisos_extra,
                       m.descripcion_perfil, m.logros, m.mostrar_en_web,
                       s.nombre as sede_nombre, s.id_sede,
                       GROUP_CONCAT(DISTINCT g.nombre SEPARATOR ', ') as grupos_asignados
                FROM maestro m
                LEFT JOIN sedes s ON m.id_sede = s.id_sede
                LEFT JOIN grupos g ON g.id_maestro = m.id_maestro
                GROUP BY m.id_maestro, s.nombre, s.id_sede
                ORDER BY m.apellido, m.nombre";

        $result = $db->query($sql);
        $maestros = $result ? $result->fetch_all(MYSQLI_ASSOC) : [];

        $sedes  = $this->sedeModel->getAll();
        $grupos = $this->grupoModel->getAll();
```

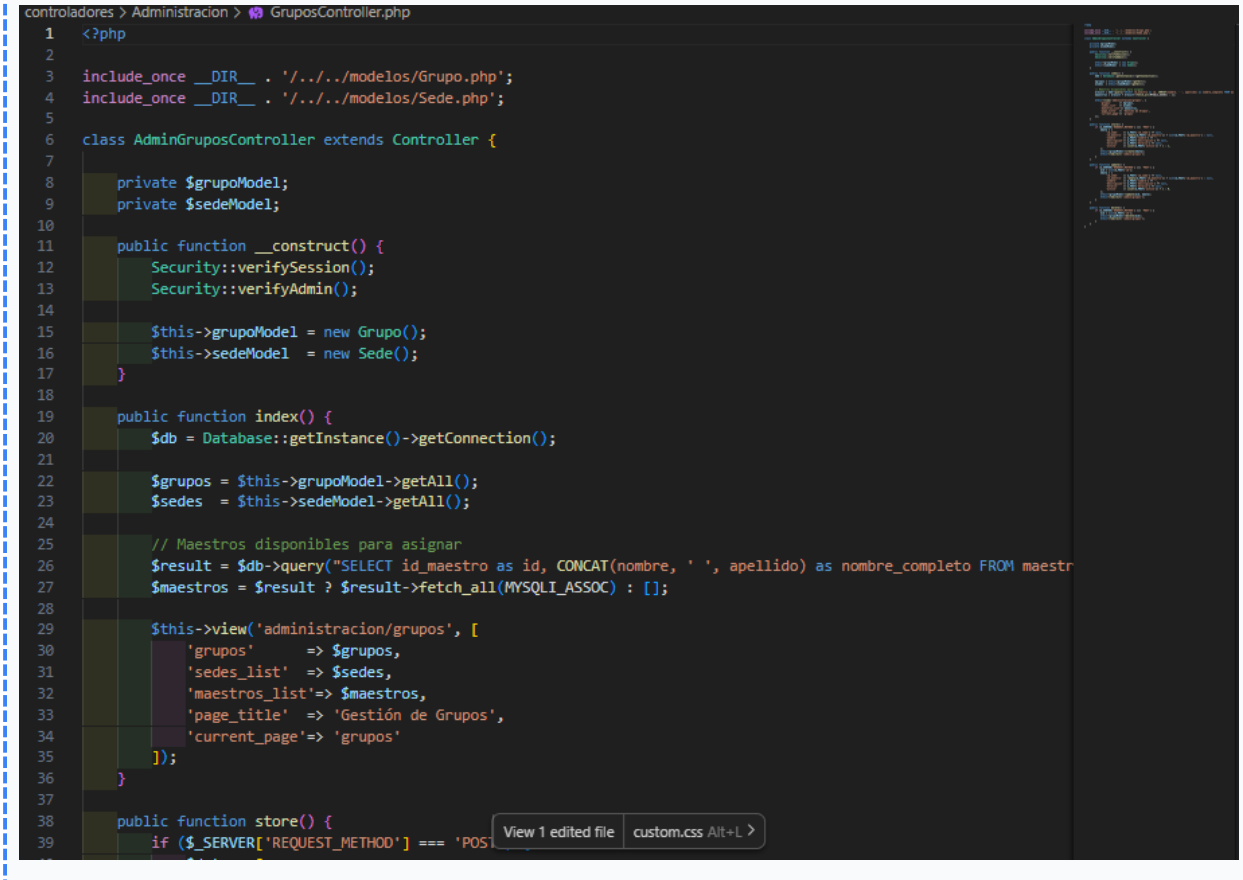
11.3 SedesController.php

- **Propósito:** Administración de academias y sucursales físicas (dirección, teléfonos de contacto y ciudad).



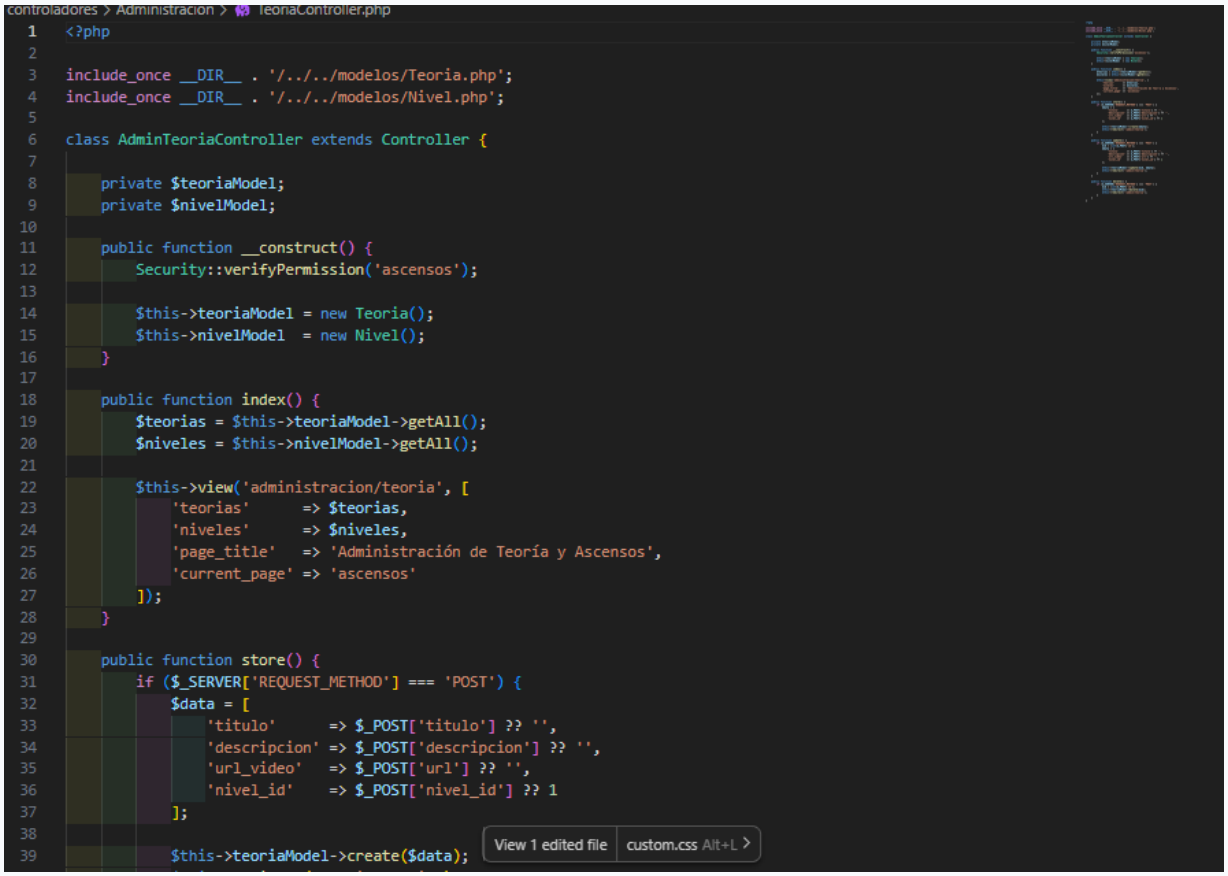
11.4 GruposController.php

- **Propósito:** Estructuración de horarios y grupos de entrenamiento, vinculando la sede y el profesor encargado.



11.5 TeoriaController.php

- **Propósito:** Gestión de los módulos pedagógicos marciales (historia, terminología en coreano y principios filosóficos).




```php
<?php

include_once __DIR__ . '/../../modelos/Teoria.php';
include_once __DIR__ . '/../../modelos/Nivel.php';

class AdminTeoriaController extends Controller {

    private $teoriaModel;
    private $nivelModel;

    public function __construct() {
        Security::verifyPermission('ascensos');

        $this->teoriaModel = new Teoria();
        $this->nivelModel  = new Nivel();
    }

    public function index() {
        $teorias = $this->teoriaModel->getAll();
        $niveles = $this->nivelModel->getAll();

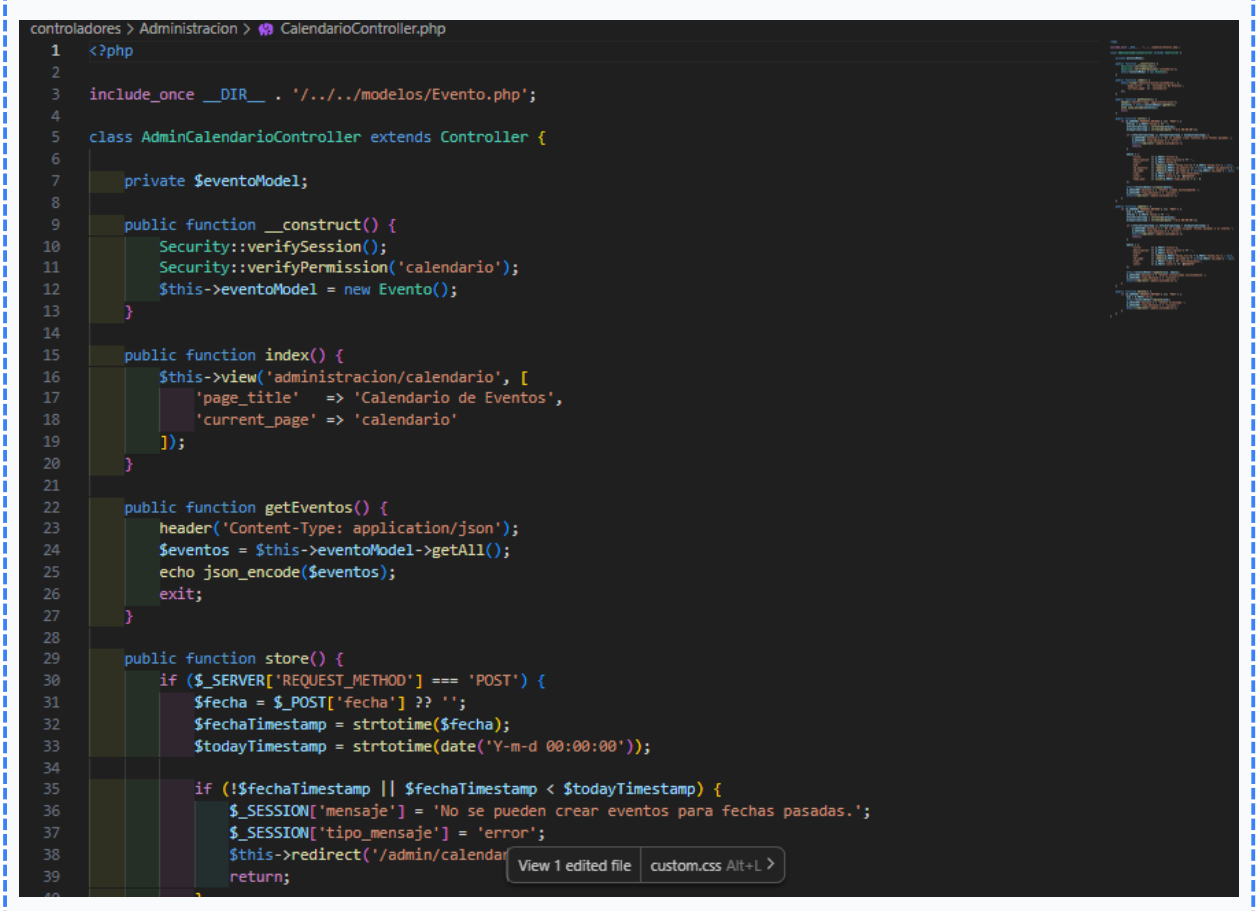
        $this->view('administracion/teoria', [
            'teorias'      => $teorias,
            'niveles'      => $niveles,
            'page_title'   => 'Administración de Teoría y Ascensos',
            'current_page' => 'ascensos'
        ]);
    }

    public function store() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $data = [
                'titulo'      => $_POST['titulo'] ?? '',
                'descripcion' => $_POST['descripcion'] ?? '',
                'url_video'   => $_POST['url'] ?? '',
                'nivel_id'    => $_POST['nivel_id'] ?? 1
            ];

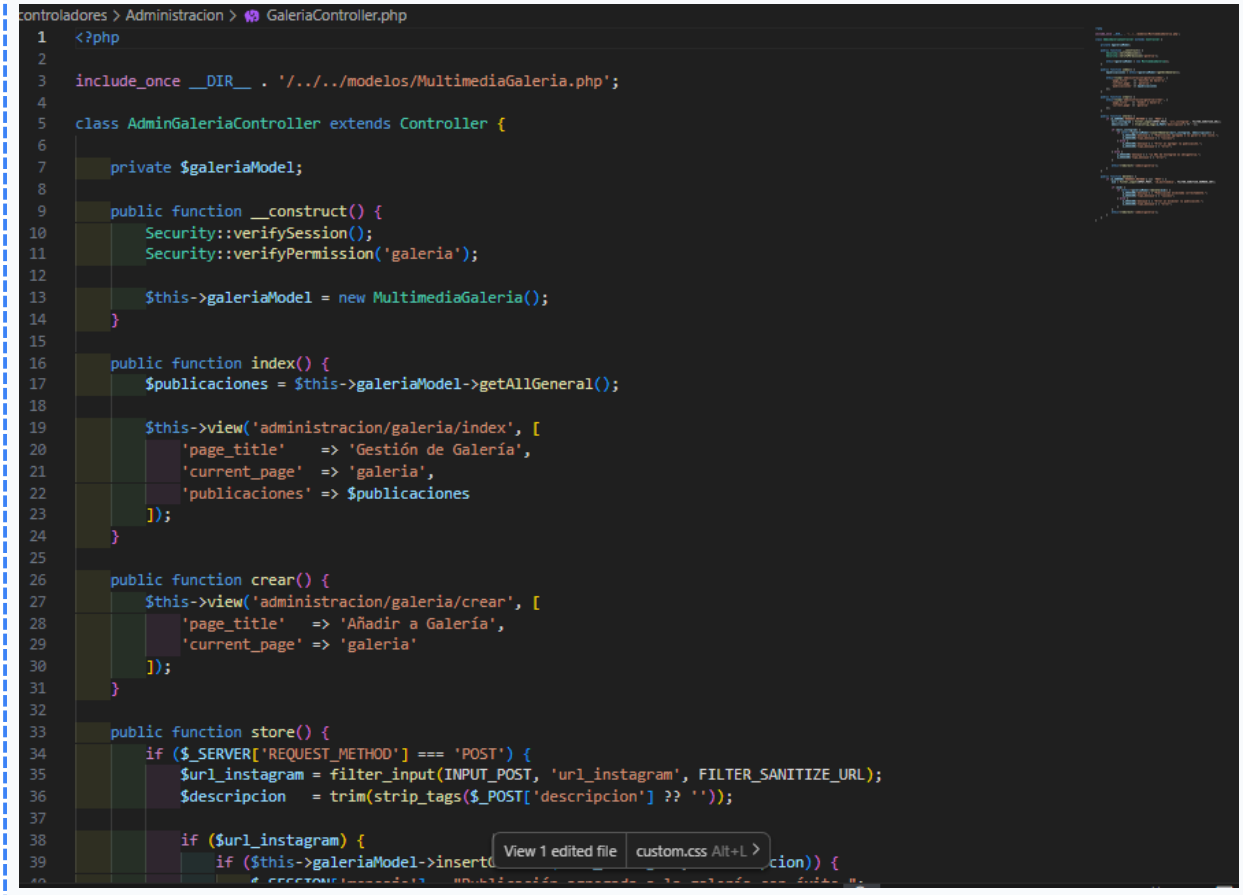
            $this->teoriaModel->create($data);
```

11.6 CalendarioController.php y GaleriaController.php

- **CalendarioController.php:** Publicación y mantenimiento de fechas de torneos, seminarios y exámenes en el calendario institucional.

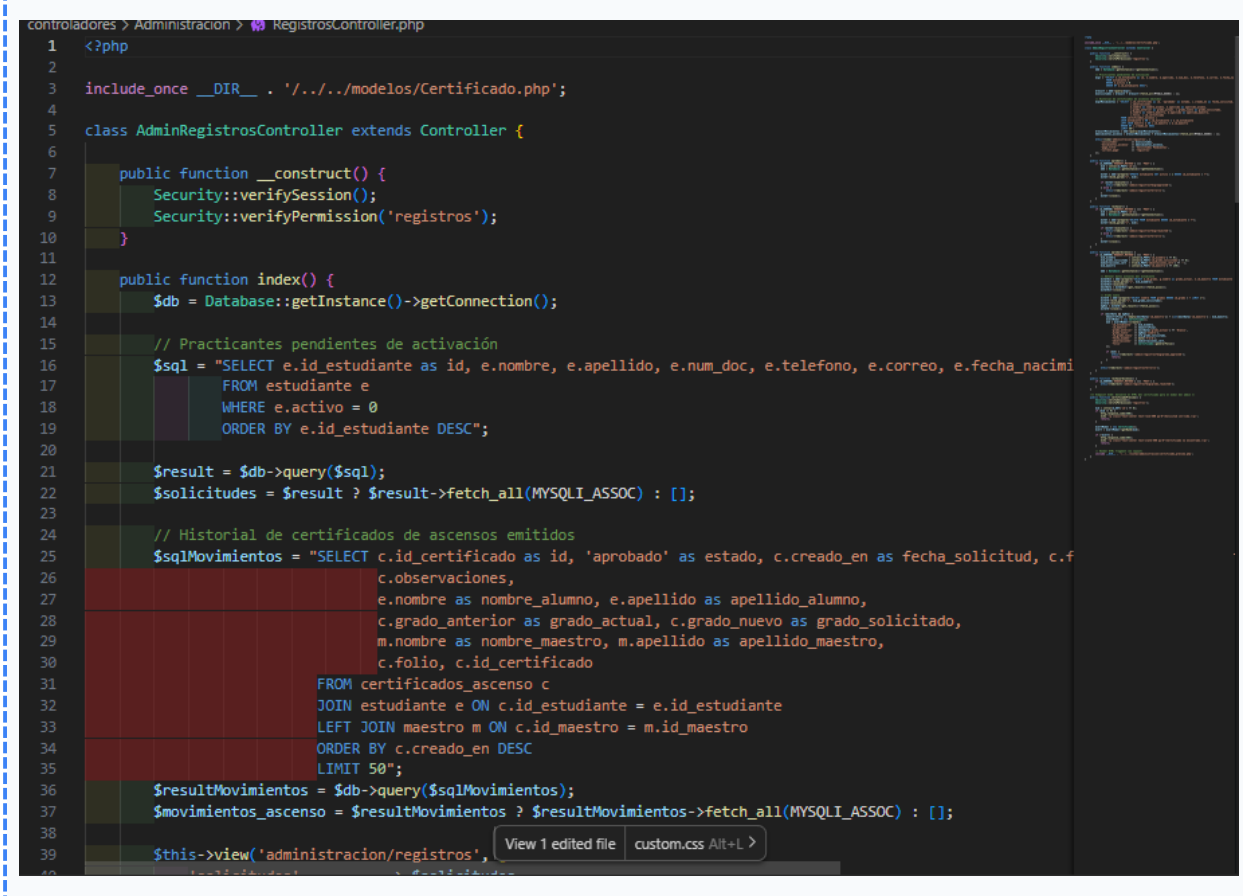


- **GaleriaController.php:** Gestión de fotografías de actividades con validación de extensiones de archivo.



11.7 RegistrosController.php y ReportesController.php

- **RegistrosController.php:** Bandeja de moderación para admitir o rechazar solicitudes de inscripción enviadas desde la web.




```php
<?php

include_once __DIR__ . '/../../modelos/Certificado.php';

class AdminRegistrosController extends Controller {

    public function __construct() {
        Security::verifySession();
        Security::verifyPermission('registros');
    }

    public function index() {
        $db = Database::getInstance()->getConnection();

        // Practicantes pendientes de activación
        $sql = "SELECT e.id_estudiante as id, e.nombre, e.apellido, e.num_doc, e.telefono, e.correo, e.fecha_nacimi
                FROM estudiante e
                WHERE e.activo = 0
                ORDER BY e.id_estudiante DESC";

        $result = $db->query($sql);
        $solicitudes = $result ? $result->fetch_all(MYSQLI_ASSOC) : [];

        // Historial de certificados de ascensos emitidos
        $sqlMovimientos = "SELECT c.id_certificado as id, 'aprobado' as estado, c.creado_en as fecha_solicitud, c.f
                                  c.observaciones,
                                  e.nombre as nombre_alumno, e.apellido as apellido_alumno,
                                  c.grado_anterior as grado_actual, c.grado_nuevo as grado_solicitado,
                                  m.nombre as nombre_maestro, m.apellido as apellido_maestro,
                                  c.folio, c.id_certificado
                           FROM certificados_ascenso c
                           JOIN estudiante e ON c.id_estudiante = e.id_estudiante
                           LEFT JOIN maestro m ON c.id_maestro = m.id_maestro
                           ORDER BY c.creado_en DESC
                           LIMIT 50";
        $resultMovimientos = $db->query($sqlMovimientos);
        $movimientos_ascenso = $resultMovimientos ? $resultMovimientos->fetch_all(MYSQLI_ASSOC) : [];

        $this->view('administracion/registros',
```

- **ReportesController.php:** Generación de resúmenes operativos, asistencias y listados para la junta directiva.

controladores > Administracion > ReportesController.php

```php
<?php

include_once __DIR__ . '/../../modelos/Usuario.php';
include_once __DIR__ . '/../../modelos/Sede.php';
include_once __DIR__ . '/../../modelos/Nivel.php';

class AdminReportesController extends Controller {

    private $usuarioModel;
    private $sedeModel;
    private $nivelModel;

    public function __construct() {
        Security::verifySession();
        Security::verifyPermission('reportes');

        $this->usuarioModel = new Usuario();
        $this->sedeModel    = new Sede();
        $this->nivelModel   = new Nivel();
    }

    public function index() {
        $miembros = $this->usuarioModel->getAllWithDetails();
        $sedes    = $this->sedeModel->getAll();
        $niveles  = $this->nivelModel->getAll();

        $this->view('administracion/reportes', [
            'miembros'     => $miembros,
            'sedes_list'   => $sedes,
            'niveles_list' => $niveles,
            'page_title'   => 'Reportes y Consultas',
            'current_page' => 'reportes'
        ]);
    }
}

```

12. Controladores Restantes del Módulo del Maestro

Módulos de instrucción técnica y seguimiento pedagógico a cargo de los profesores:

12.1 DashboardController.php (Maestro)

- **Propósito:** Panel del instructor que expone las clases programadas para el día y accesos directos.

controladores > Maestro > DashboardController.php

```php
<?php

include_once __DIR__ . '/../../modelos/Evento.php';

class MaestroDashboardController extends Controller {

    public function __construct() {
        Security::verifySession();
        Security::verifyMaestro();
    }

    public function index() {
        $db = Database::getInstance()->getConnection();
        $id_maestro = (int)$_SESSION['id'];

        $stats = [];

        // Total alumnos activos
        $res = $db->query("SELECT COUNT(*) as total FROM estudiante WHERE activo = 1");
        $stats['total_alumnos'] = $res ? (int)$res->fetch_assoc()['total'] : 0;

        // Certificados emitidos por este maestro
        $stmt = $db->prepare("SELECT COUNT(*) as total FROM certificados_ascenso WHERE id_maestro = ?");
        $stmt->bind_param("i", $id_maestro);
        $stmt->execute();
        $stats['solicitudes_aprobadas'] = (int)$stmt->get_result()->fetch_assoc()['total'];
        $stmt->close();

        // Solicitudes pendientes de activación en el sistema
        $resPend = $db->query("SELECT COUNT(*) as total FROM estudiante WHERE activo = 0");
        $stats['solicitudes_pendientes'] = $resPend ? (int)$resPend->fetch_assoc()['total'] : 0;

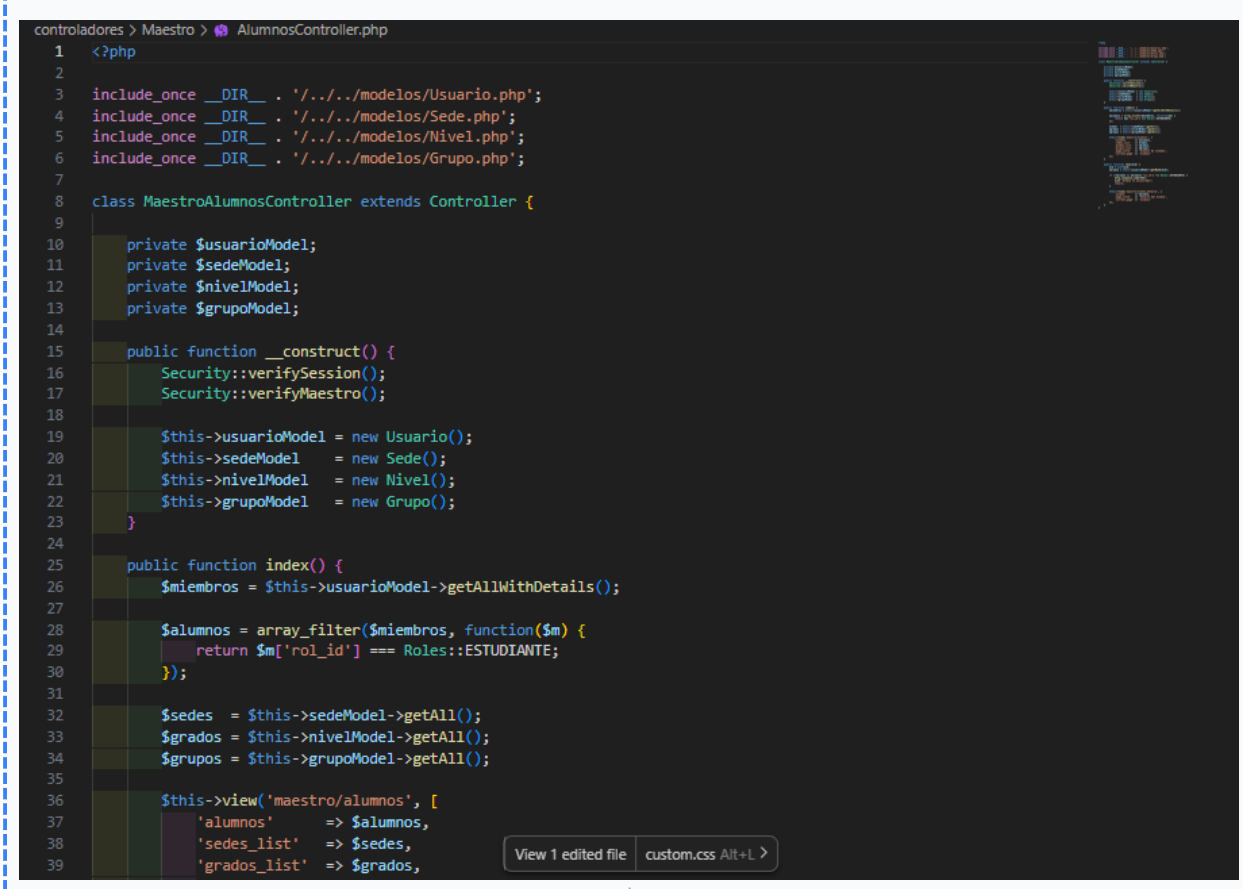
        // Distribución de grados de alumnos
        $sqlGrados = "SELECT g.nombre, COUNT(e.id_estudiante) as cantidad
                      FROM grados g
                      LEFT JOIN estudiante e ON g.id_grado = e.id_grado AND e.activo = 1
                      GROUP BY g.id_grado, g.nombre
                      ORDER BY g.id_grado ASC";
```

View 1 edited file | custom.css Alt+L

(Haga clic aquí y pegue [Ctrl + V] la captura del editor de código)

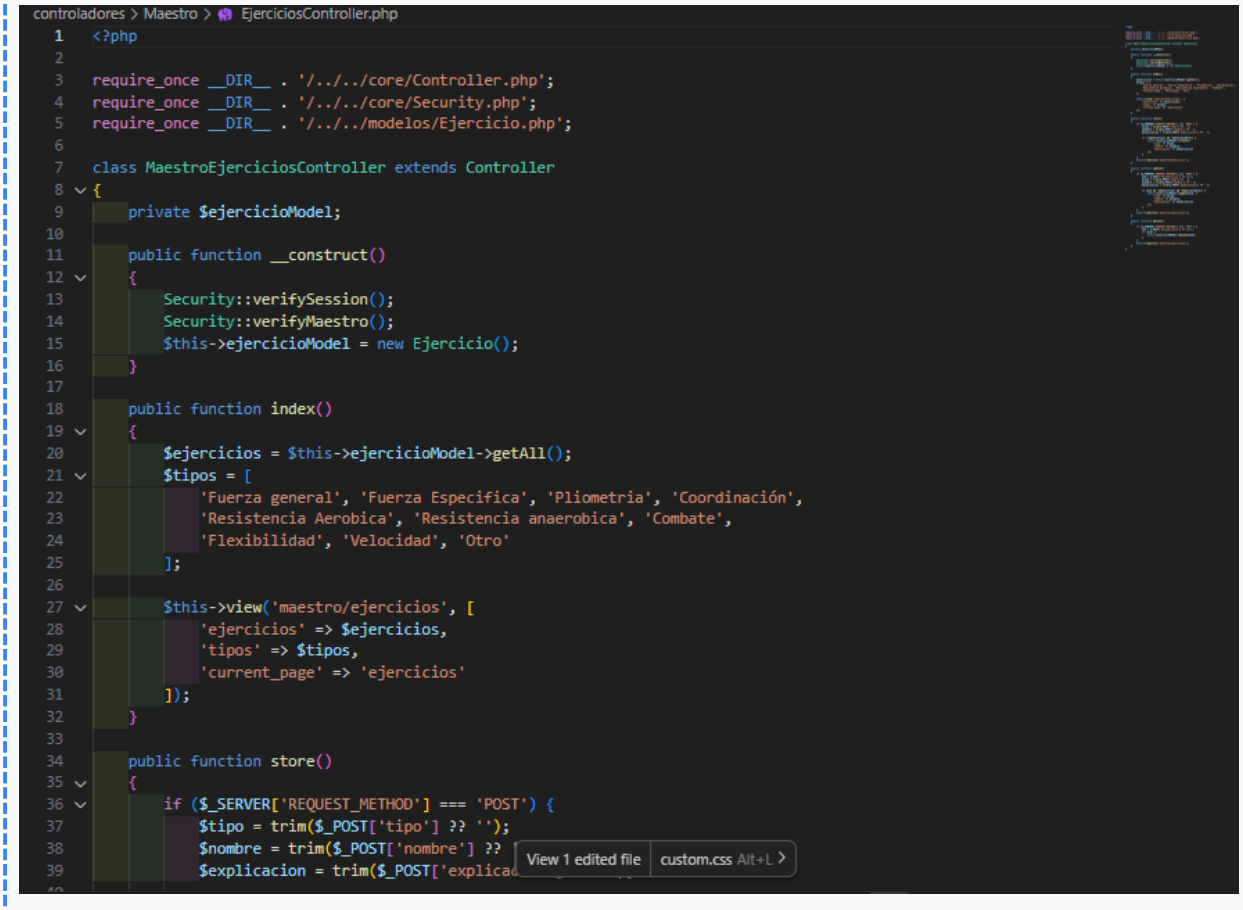
12.2 AlumnosController.php

- **Propósito:** Listado y seguimiento del rendimiento técnico y asistencia de los deportistas a su cargo.



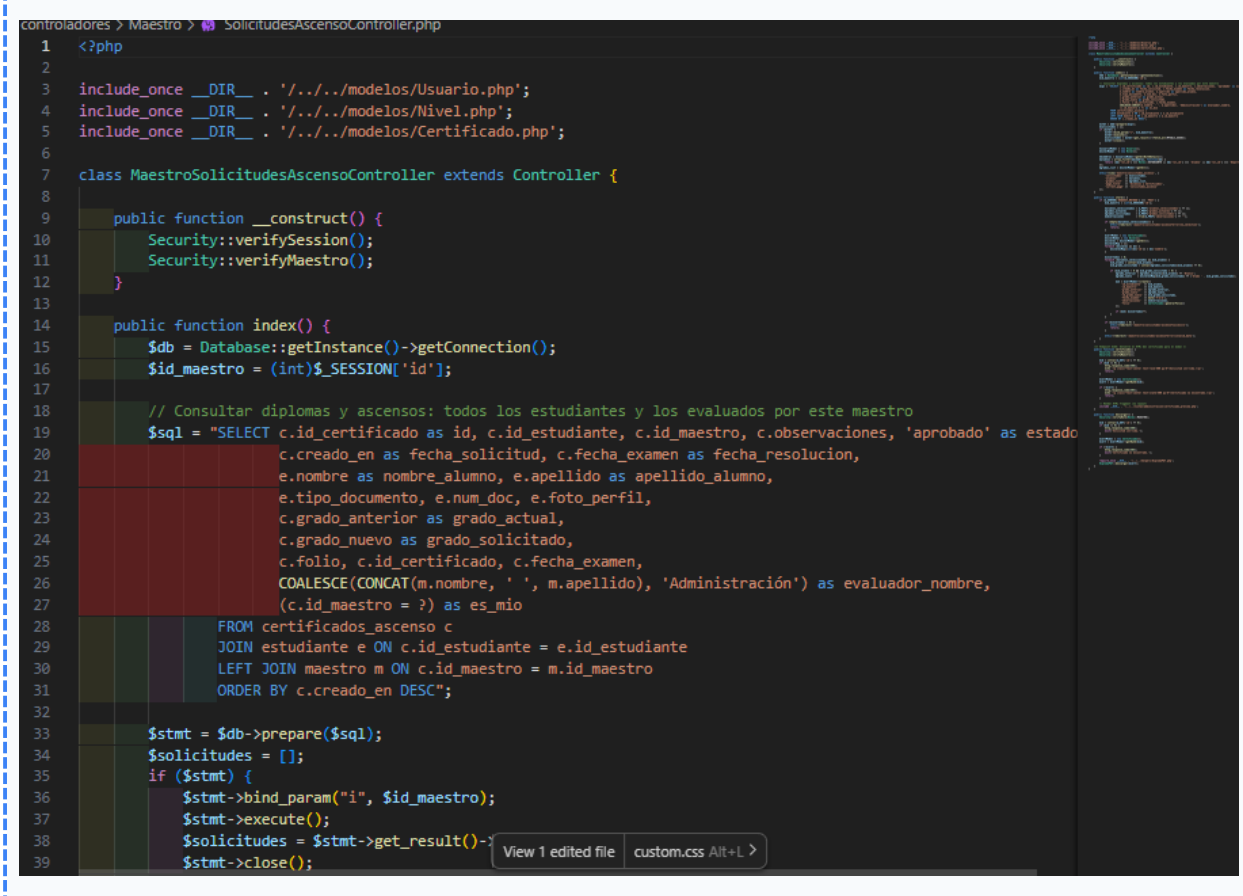
12.3 EjerciciosController.php

- **Propósito:** Biblioteca técnica de artes marciales: creación y categorización de ejercicios (Poomsae, Kyorugi, pateo).



12.4 SolicitudesAscensoController.php

- **Propósito:** Postulación formal de estudiantes que cumplen con los requisitos de tiempo y madurez para presentar examen de cinturón.



13. Controladores Restantes de Estudiante, Usuario y Portal Web

13.1 Estudiante/DashboardController.php

- **Propósito:** Presenta al alumno su cinturón actual, porcentaje de asistencias y próximos eventos institucionales.

controladores > Estudiante > DashboardController.php

```php
<?php

include_once __DIR__ . '/../../modelos/Usuario.php';
include_once __DIR__ . '/../../modelos/Evento.php';

class EstudianteDashboardController extends Controller {

    public function __construct() {
        Security::verifySession();

        $rol = $_SESSION['rol_id'] ?? null;
        if (Roles::esAdmin($rol)) {
            $this->redirect('/admin/dashboard');
        }
        if (Roles::esMaestro($rol) || $rol === Roles::PROFESOR || $rol === Roles::MONITOR) {
            $this->redirect('/maestro/dashboard');
        }
    }

    public function index() {
        $usuarioModel = new Usuario();
        $eventoModel  = new Evento();

        $estudiante = $usuarioModel->getById($_SESSION['id']);

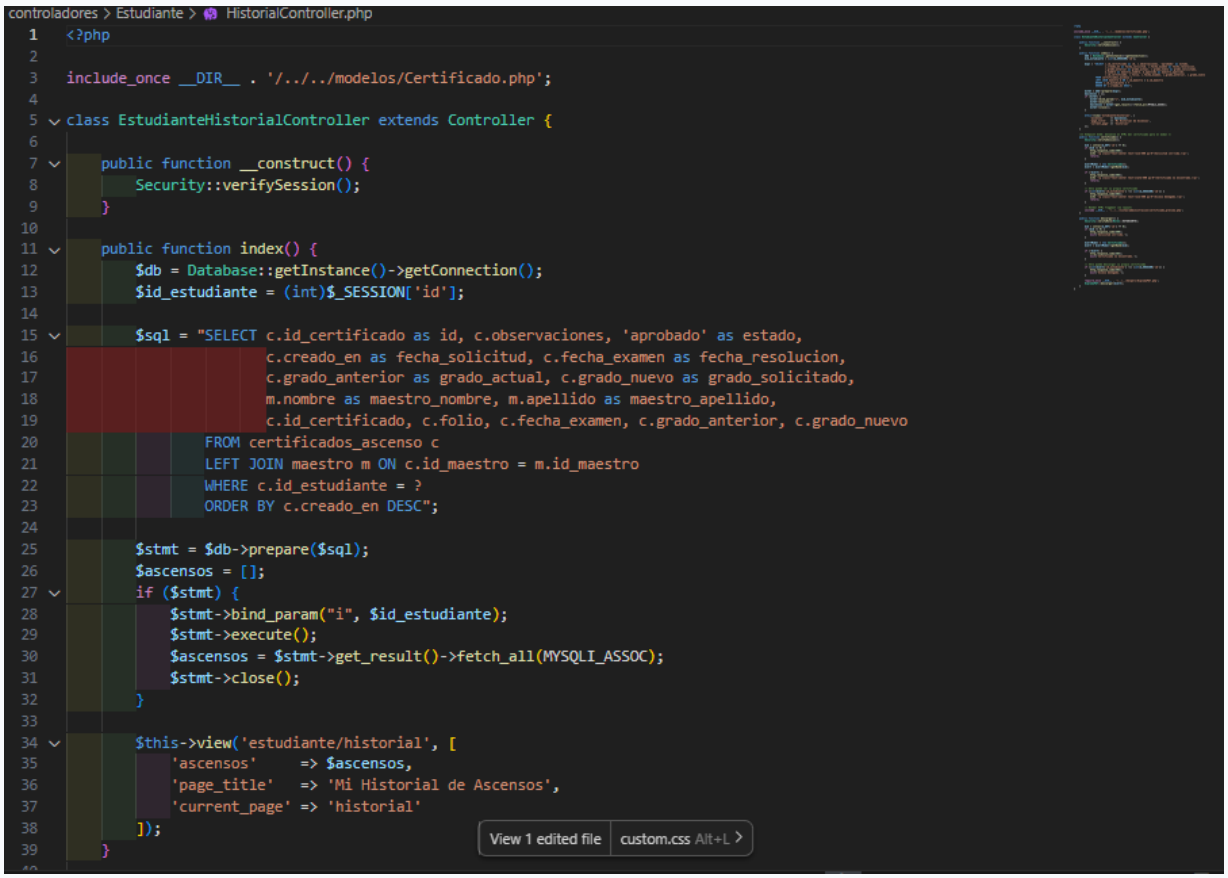
        $proximos_eventos = $eventoModel->getUpcoming(3);

        $this->view('estudiante/dashboard', [
            'estudiante'       => $estudiante,
            'proximos_eventos' => $proximos_eventos,
            'page_title'       => 'Portal del Alumno'
        ]);
    }
}

```

13.2 Estudiante/HistorialController.php

- **Propósito:** Línea de tiempo con el histórico de ascensos de grado y visualización del certificado oficial.



13.3 Usuario/CalendarioController.php

- **Propósito:** Muestra el calendario mensual sincronizado con las actividades de la sede del usuario.



13.4 Controladores del Portal Web Público (Web/)

- **Web/InicioController.php:** Landing page institucional y portal de bienvenida al público general.

controladores > Web > InicioController.php

```php
<?php

class WebInicioController extends Controller {

    public function index() {
        $this->view('web/inicio', [
            'page_title' => 'Jinnwhan Organization - Taekwondo'
        ]);
    }

    public function portal() {
        $this->view('web/portal', [
            'page_title' => 'Bienvenido - Jinhwa Corporation'
        ]);
    }
}

```

- **Web/SedesController.php y Web/GruposController.php:** Directorio público donde los interesados consultan dojangs y horarios disponibles.

controladores > Web > SedesController.php

```php
<?php

include_once __DIR__ . '/../../modelos/Sede.php';

class WebSedesController extends Controller {

    public function index() {
        $sedeModel = new Sede();
        $sedes     = $sedeModel->getAll();

        $this->view('web/sedes', [
            'sedes'      => $sedes,
            'page_title' => 'Nuestras Sedes'
        ]);
    }
}

```

controladores > Web > GruposController.php

```php
<?php

include_once __DIR__ . '/../../modelos/Grupo.php';

class WebGruposController extends Controller {

    public function index() {
        $grupoModel = new Grupo();
        $grupos     = $grupoModel->getAll();

        // Filtrar solo los grupos activos
        $gruposActivos = array_filter($grupos, fn($g) => ($g['activo'] ?? 0) == 1);

        $this->view('web/grupos', [
            'grupos'     => $gruposActivos,
            'page_title' => 'Grupos de Entrenamiento'
        ]);
    }
}

```

- **Web/MiembrosController.php y Web/PaginaController.php:** Información institucional, filosofía del Taekwondo y equipo docente.

controladores > Web > PaginaController.php

```php
<?php

class WebPaginaController extends Controller {

    public function nosotros() {
        $this->view('web/nosotros', ['page_title' => 'Nosotros']);
    }
}

```

controladores > Web > MiembrosController.php

```php
<?php

include_once __DIR__ . '/../../modelos/Usuario.php';

class WebMiembrosController extends Controller {

    public function index() {
        $usuarioModel = new Usuario();
        $miembros = $usuarioModel->getPublicProfiles();

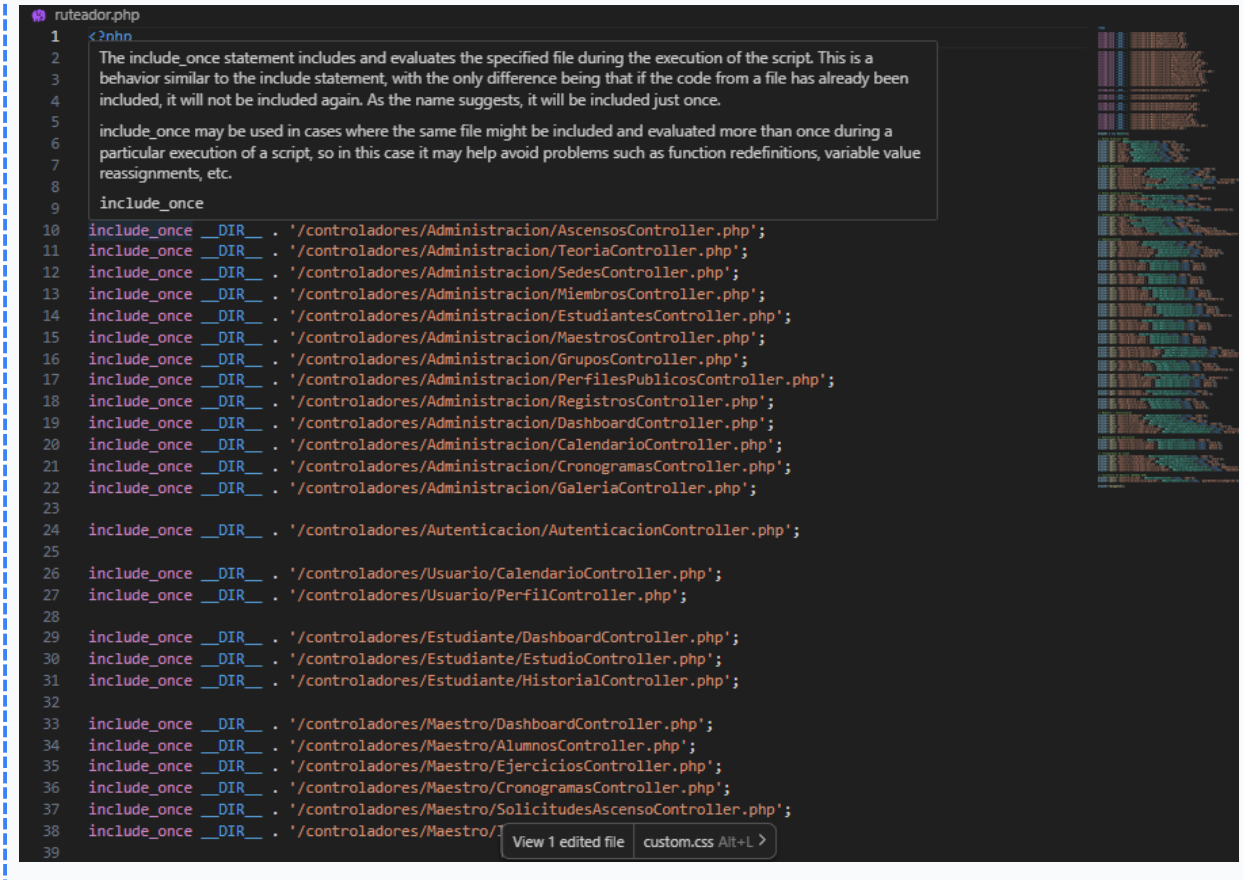
        $this->view('web/miembros', [
            'page_title' => 'Nuestros Miembros',
            'miembros'   => $miembros
        ]);
    }
}

```

14. Motor de Enrutamiento y Funciones Auxiliares (ruteador.php y helpers/)

14.1 Archivo Central de Enrutamiento: ruteador.php

- **Propósito:** Centraliza la definición de todas las rutas HTTP del sistema (Web, Estudiante, Maestro y Administración).



14.2 Funciones Auxiliares: helpers/url_helper.php

- **Propósito:** Funciones globales para construir URLs canónicas y enlaces a recursos estáticos (CSS, JS, imágenes).

helpers > url_helper.php

```php
<?php

if (!function_exists('base_url')) {
    function base_url($path = '') {

        $scriptName = $_SERVER['SCRIPT_NAME'];
        $base = dirname($scriptName);

        $base = str_replace('\\', '/', $base);

        if ($base !== '/') {
            $base = rtrim($base, '/');
        } else {

            $base = '';
        }

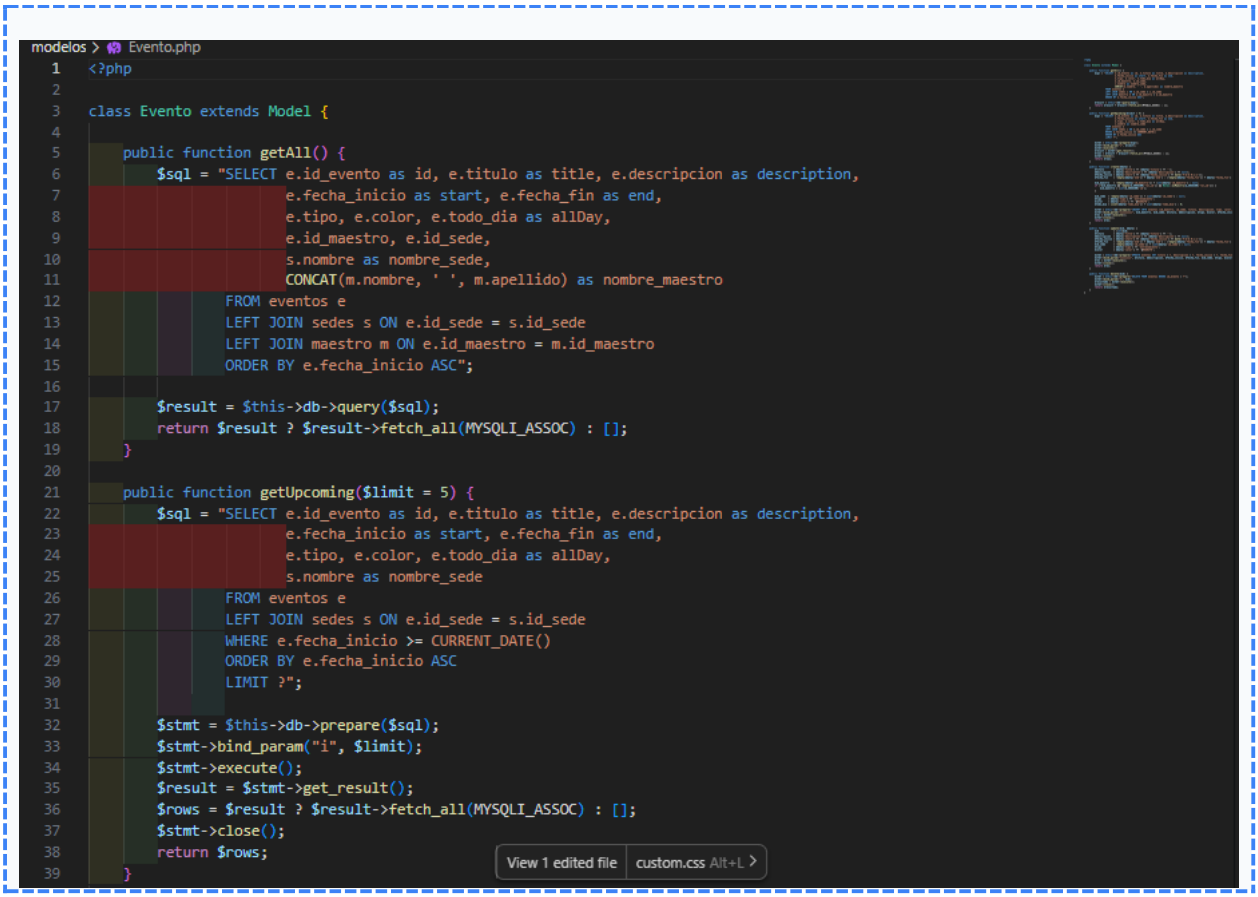
        return $base . '/' . ltrim($path, '/');
    }
}
  Ctrl+L for Agent
if (!function_exists('asset')) {
    function asset($path) {
        return base_url('public/' . ltrim($path, '/'));
    }
}

```

15. Modelos de Entidad Complementarios (modelos/)

15.1 Evento.php y MultimediaGaleria.php

- **Propósito:** Persistencia de torneos, seminarios y álbumes fotográficos de la corporación marcial.




```php
<?php

class Evento extends Model {

    public function getAll() {
        $sql = "SELECT e.id_evento as id, e.titulo as title, e.descripcion as description,
                       e.fecha_inicio as start, e.fecha_fin as end,
                       e.tipo, e.color, e.todo_dia as allDay,
                       e.id_maestro, e.id_sede,
                       s.nombre as nombre_sede,
                       CONCAT(m.nombre, ' ', m.apellido) as nombre_maestro
                FROM eventos e
                LEFT JOIN sedes s ON e.id_sede = s.id_sede
                LEFT JOIN maestro m ON e.id_maestro = m.id_maestro
                ORDER BY e.fecha_inicio ASC";

        $result = $this->db->query($sql);
        return $result ? $result->fetch_all(MYSQLI_ASSOC) : [];
    }

    public function getUpcoming($limit = 5) {
        $sql = "SELECT e.id_evento as id, e.titulo as title, e.descripcion as description,
                       e.fecha_inicio as start, e.fecha_fin as end,
                       e.tipo, e.color, e.todo_dia as allDay,
                       s.nombre as nombre_sede
                FROM eventos e
                LEFT JOIN sedes s ON e.id_sede = s.id_sede
                WHERE e.fecha_inicio >= CURRENT_DATE()
                ORDER BY e.fecha_inicio ASC
                LIMIT ?";

        $stmt = $this->db->prepare($sql);
        $stmt->bind_param("i", $limit);
        $stmt->execute();
        $result = $stmt->get_result();
        $rows = $result ? $result->fetch_all(MYSQLI_ASSOC) : [];
        $stmt->close();
        return $rows;
    }
```

modelos > MultimediaGaleria.php

```php
<?php

class MultimediaGaleria extends Model {

    private function ensureColumns() {
        $checkE = $this->db->query("SHOW COLUMNS FROM galeria_multimedia LIKE 'id_estudiante'");
        if ($checkE && $checkE->num_rows === 0) {
            $this->db->query("ALTER TABLE galeria_multimedia ADD COLUMN id_estudiante INT(11) DEFAULT NULL AFTER id
        }
        $checkA = $this->db->query("SHOW COLUMNS FROM galeria_multimedia LIKE 'id_administrador'");
        if ($checkA && $checkA->num_rows === 0) {
            $this->db->query("ALTER TABLE galeria_multimedia ADD COLUMN id_administrador INT(11) DEFAULT NULL AFTER
        }
    }

    public function getByMaestroId($id_maestro) {
        $stmt = $this->db->prepare("SELECT * FROM galeria_multimedia WHERE id_maestro = ? AND tipo = 'instagram' LI
        $stmt->bind_param("i", $id_maestro);
        $stmt->execute();
        $result = $stmt->get_result();
        $row = $result ? $result->fetch_assoc() : null;
        $stmt->close();
        return $row;
    }

    public function getByPersona($id, $rol = 'Maestros') {
        $this->ensureColumns();
        $col = 'id_maestro';
        if ($rol === 'Deportistas' || $rol === 'Estudiantes' || $rol === Roles::ESTUDIANTE) {
            $col = 'id_estudiante';
        } elseif ($rol === 'Administracion' || $rol === Roles::ADMINISTRADOR) {
            $col = 'id_administrador';
        }

        $stmt = $this->db->prepare("SELECT * FROM galeria_multimedia WHERE {$col} = ? AND tipo = 'instagram' LIMIT
        $stmt->bind_param("i", $id);
        $stmt->execute();
        $result = $stmt->get_result();
        $row = $result ? $result->fetch_assoc()
```

View 1 edited file | custom.css Alt+L

15.2 Categoria.php y Nivel.php

- **Propósito:** Catálogos normalizados para clasificar técnicas por tipología (formas, combate) y dificultad.

modelos > Nivel.php

```php
<?php

class Nivel extends Model {

    public function getAll() {
        $result = $this->db->query("SELECT id_grado AS id, nombre FROM grados ORDER BY id_grado ASC");
        return $result ? $result->fetch_all(MYSQLI_ASSOC) : [];
    }
}

```

modelos > Categoria.php

```php
<?php

class Categoria extends Model {

    public function getAll() {
        $result = $this->db->query("SELECT id_categoria AS id, nombre, descripcion FROM categorias ORDER BY id_cate
        return $result ? $result->fetch_all(MYSQLI_ASSOC) : [];
    }
}

```

16. Catálogo de Vistas y Plantillas de Interfaz (vistas/)

A continuación se presentan las capturas correspondientes a las vistas de usuario y componentes gráficos del proyecto:

16.1 Layouts y Cabeceras Modulares (vistas/layout/)

- **Layout Administración (cabecera y pie):** Plantilla envolvente para los módulos de dirección.

vistas > layout > administracion_cabecera.php

```php
<?php

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

$nombre_usuario = $_SESSION['nombre'] ?? $_SESSION['usuario']['nombre'] ?? 'Admin';
$foto_perfil = null;

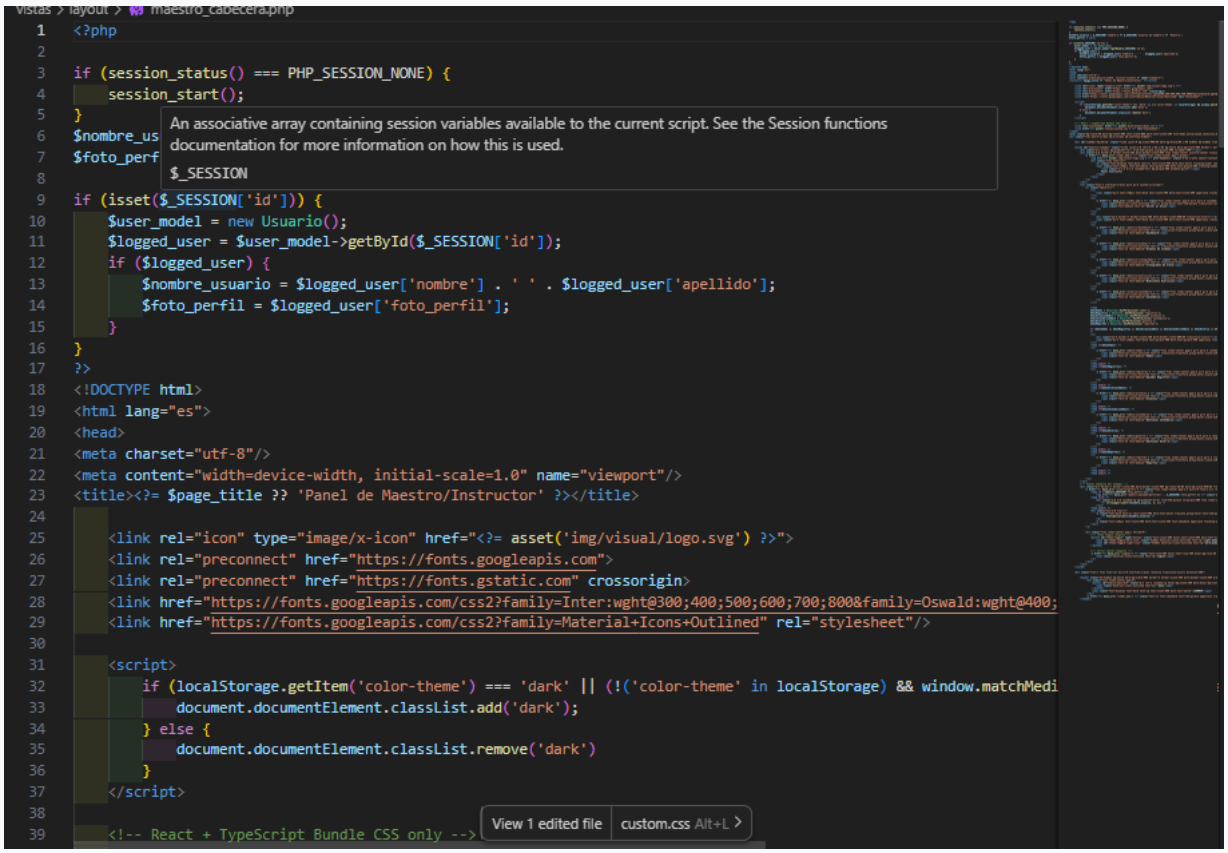
if (isset($_SESSION['id'])) {
    $user_model = new Usuario();
    $logged_user = $user_model->getById($_SESSION['id']);
    if ($logged_user) {
        $nombre_usuario = $logged_user['nombre'] . ' ' . $logged_user['apellido'];
        $foto_perfil = $logged_user['foto_perfil'];
    }
}
?>
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8"/>
<meta content="width=device-width, initial-scale=1.0" name="viewport"/>
<title><?= $page_title ?? 'Panel de Administración' ?></title>

    <link rel="icon" type="image/x-icon" href="<?= asset('img/visual/logo.svg') ?>">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&family=Oswald:wght@400;
    <link href="https://fonts.googleapis.com/css2?family=Material+Icons+Outlined" rel="stylesheet"/>

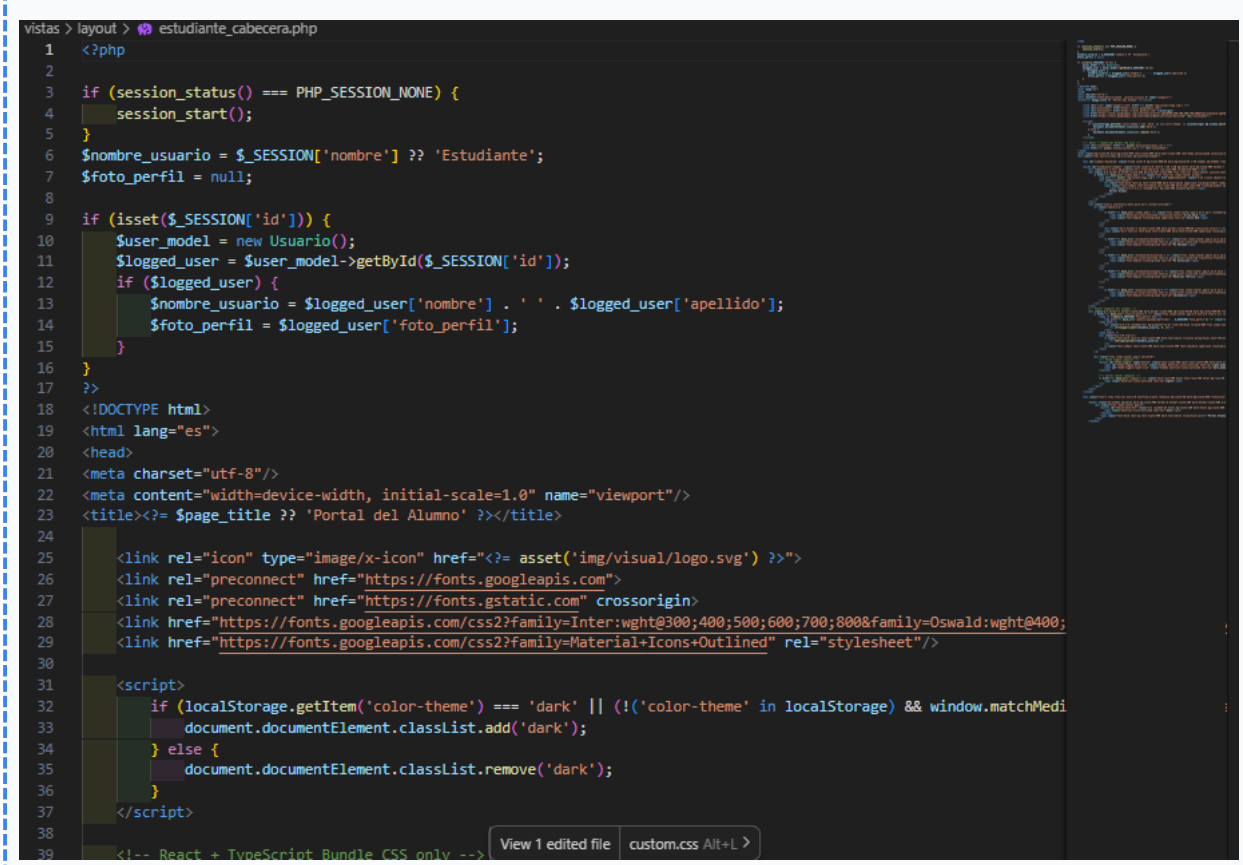
    <script>
        if (localStorage.getItem('color-theme') === 'dark' || (!('color-theme' in localStorage) && window.matchMedi
            document.documentElement.classList.add('dark');
        } else {
            document.documentElement.classList.remove('dark')
        }
    </script>
```

View 1 edited file | custom.css Alt+L

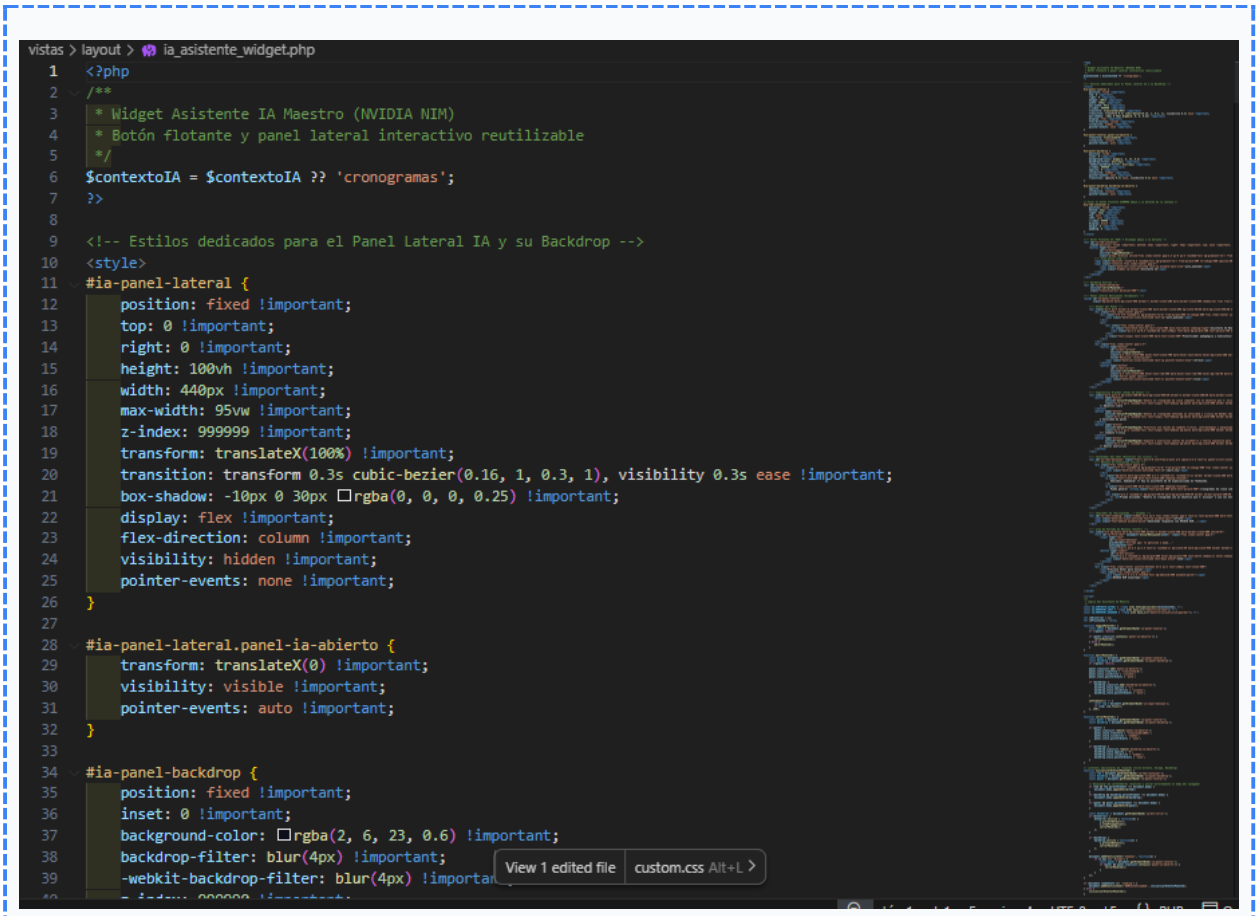
- **Layout Maestro (cabecera y pie):** Estructura visual adaptada para profesores e instructores de dojang.



- **Layout Estudiante (cabecera y pie):** Estructura simplificada orientada al deportista.

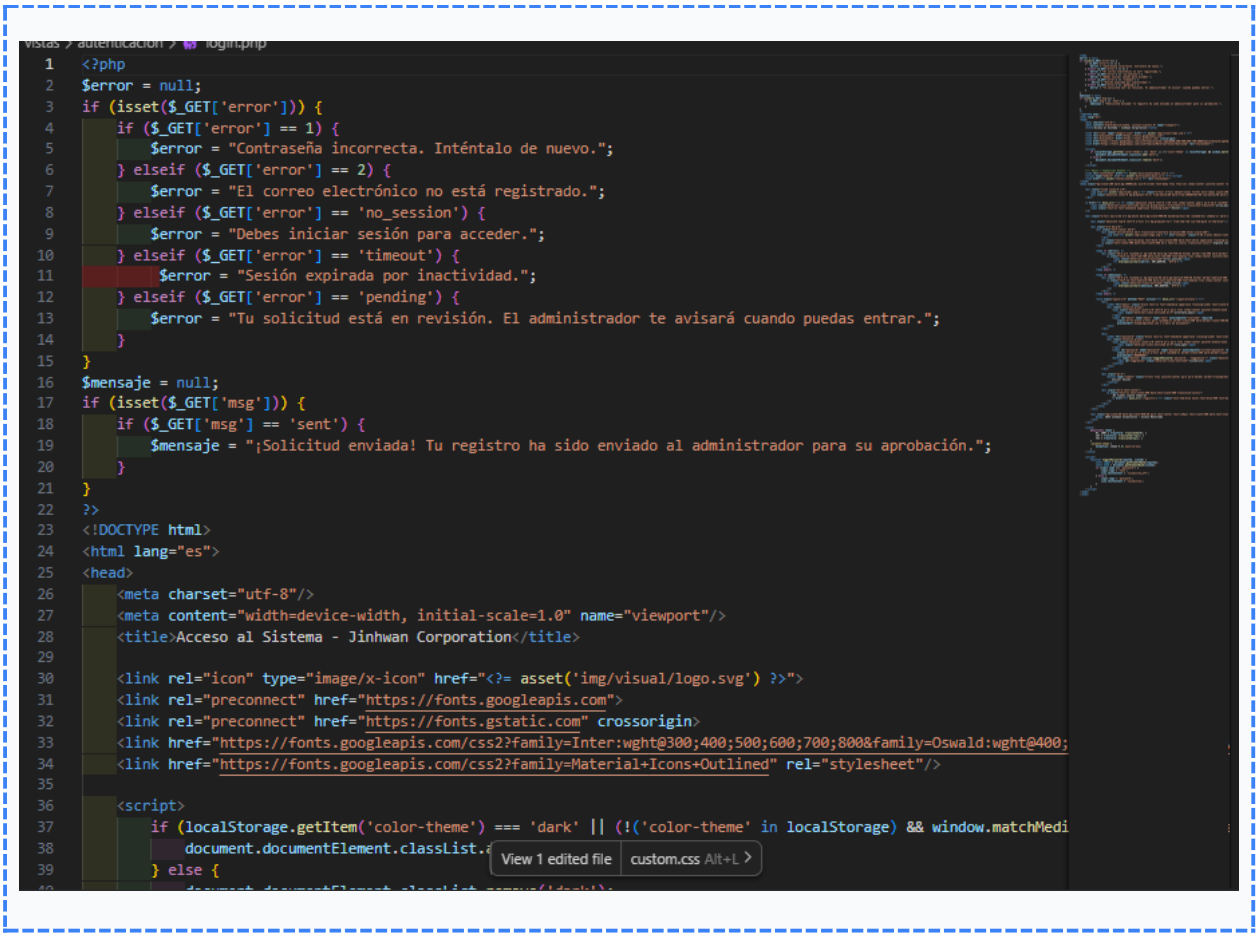


- **Widget Asistente IA (vistas/layout/ia_asistente_widget.php):** Componente interactivo flotante para asistencia marcial con inteligencia artificial.

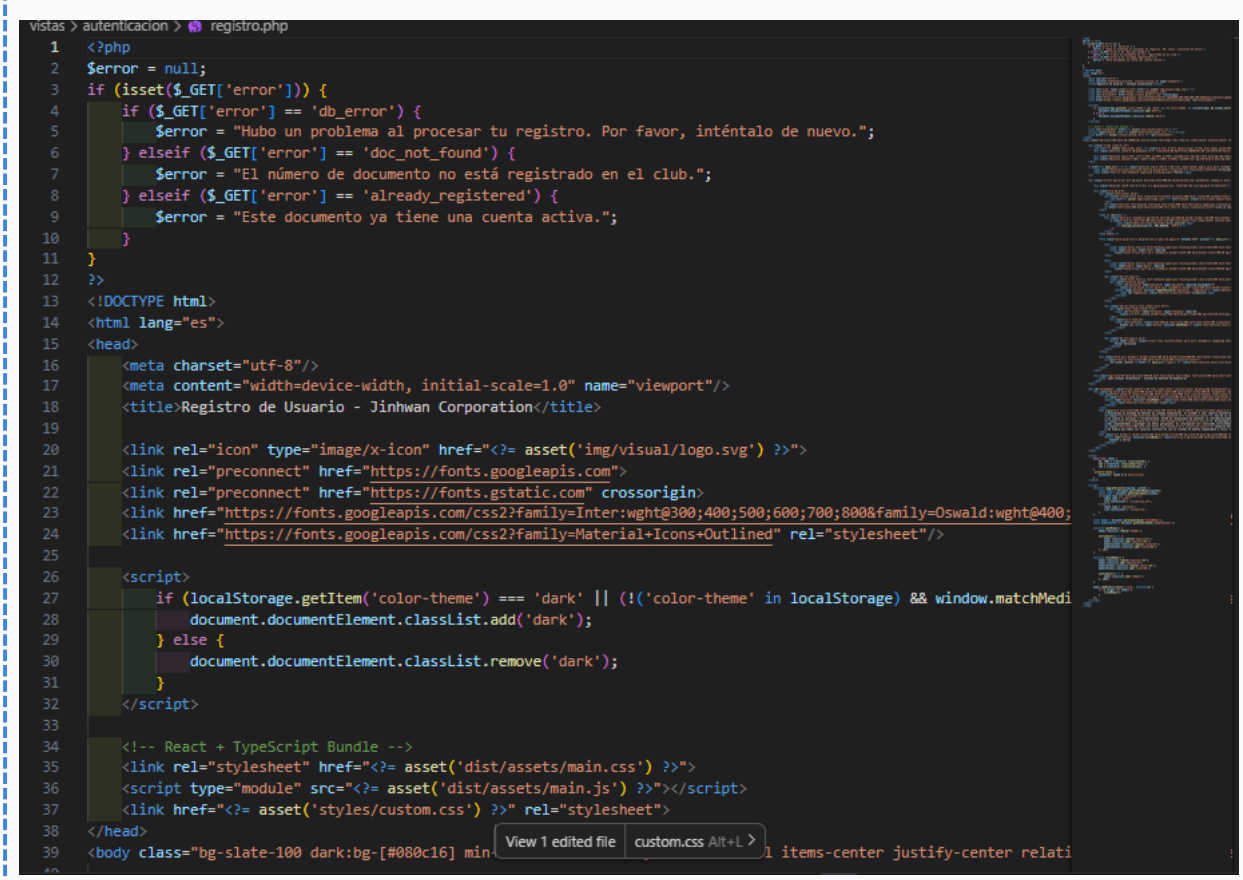


16.2 Vistas del Módulo de Autenticación

- **vistas/autenticacion/login.php:** Formulario de inicio de sesión responsivo con Tailwind CSS.

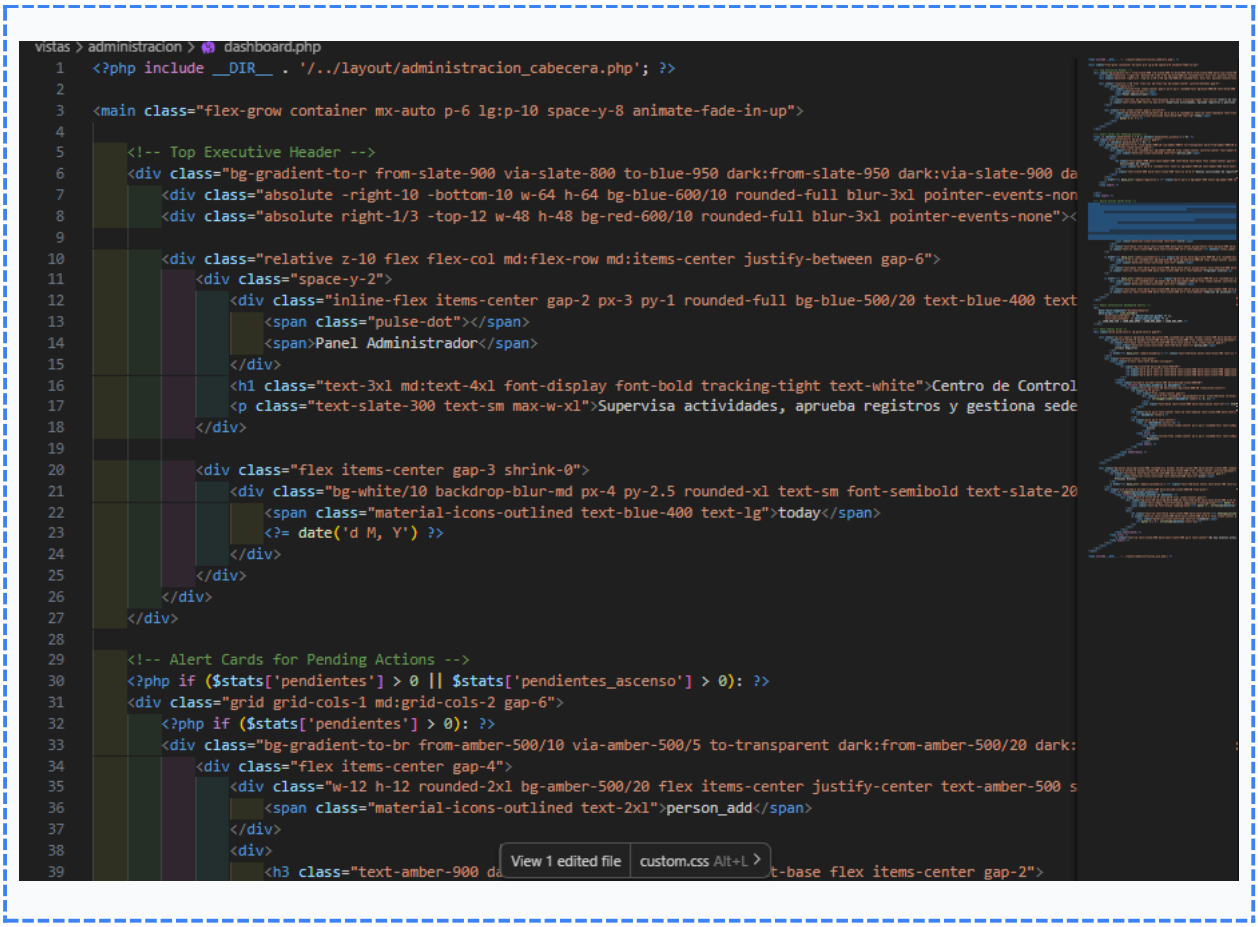


- **vistas/autenticacion/registro.php:** Formulario público para solicitud de ingreso de nuevos practicantes.

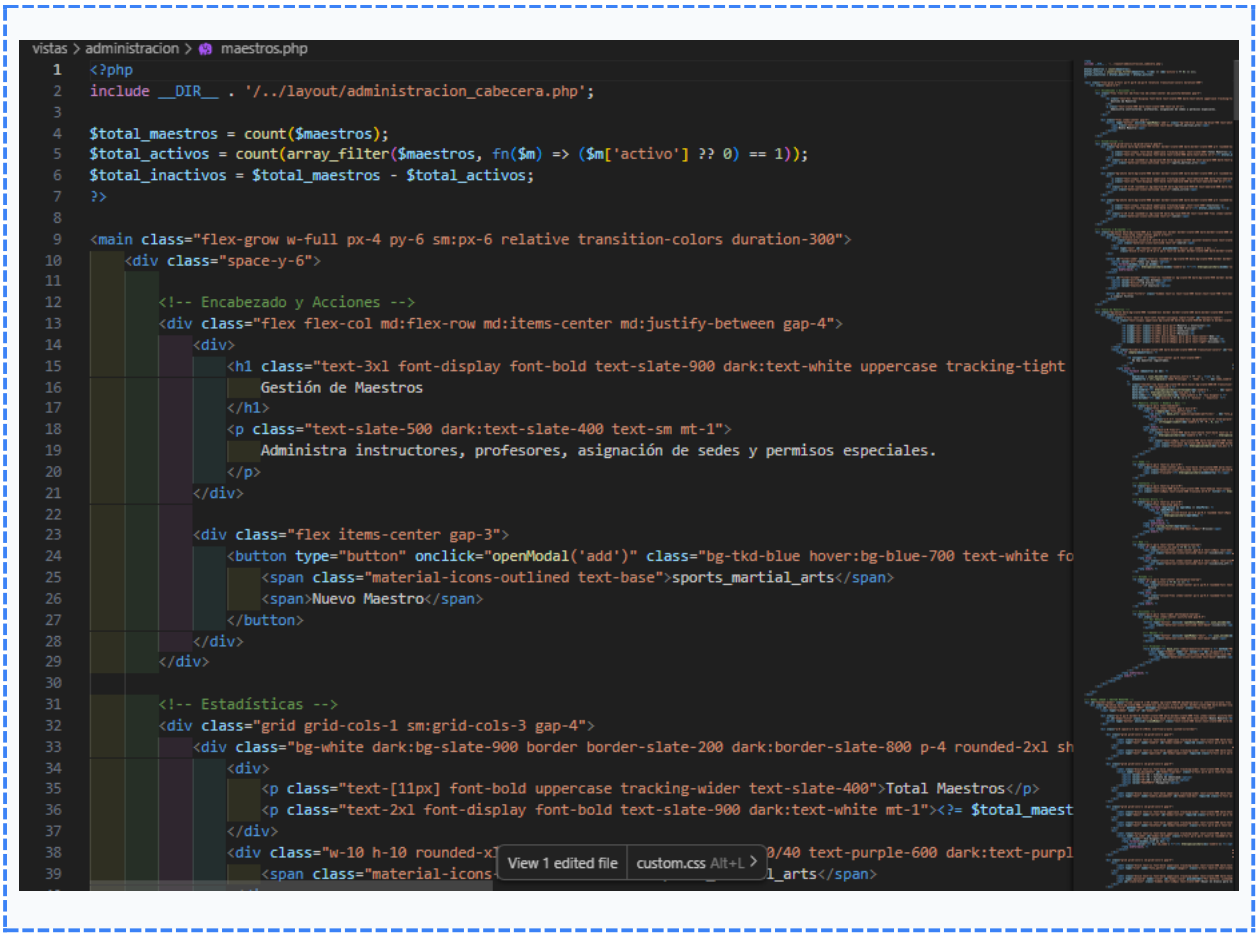


16.3 Vistas de Gestión Administrativa

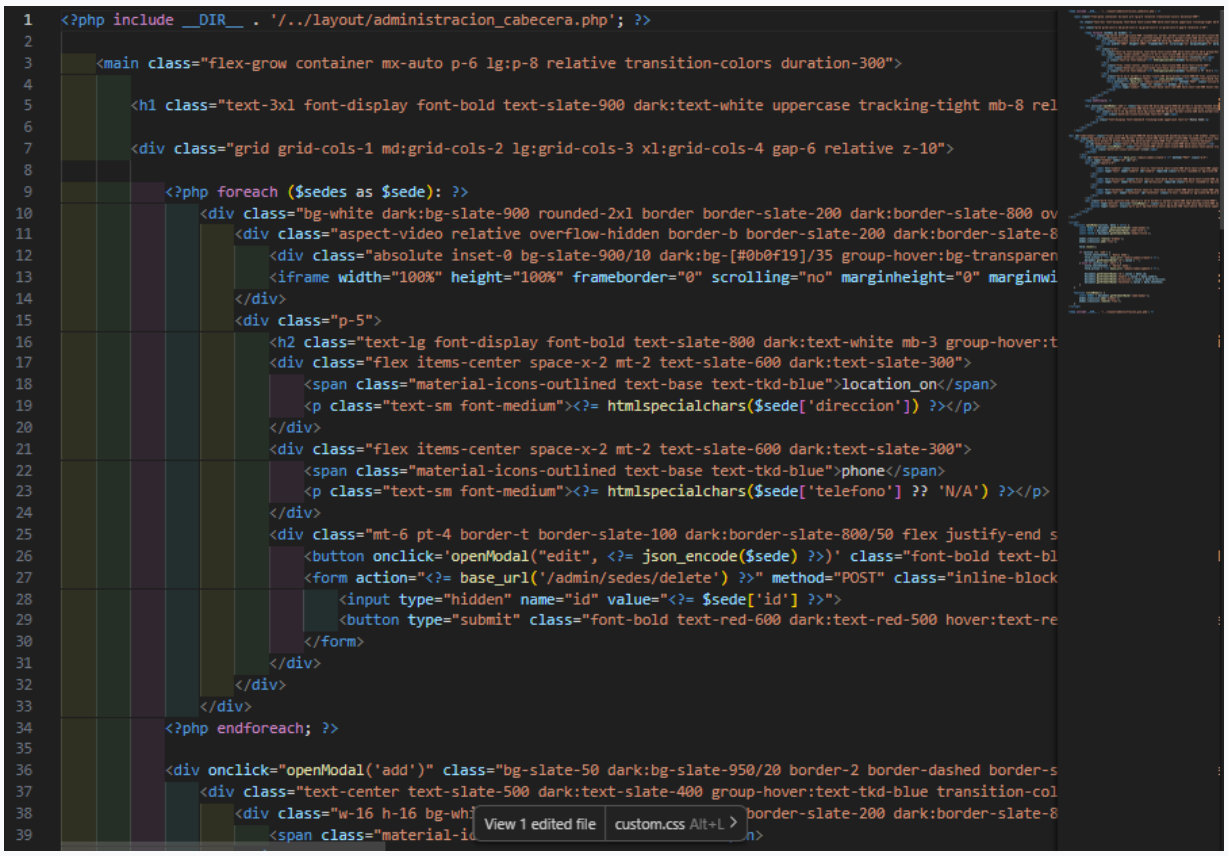
- **vistas/administracion/dashboard.php:** Panel directivo con tarjetas de indicadores y gráficos de rendimiento.



- **vistas/administracion/maestros.php:** Tabla de instructores con filtros y conmutador de estado activo/inactivo.



- **vistas/administracion/sedes.php y grupos.php:** Interfaces para alta y mantenimiento de escuelas y turnos de clase.



```php
<?php include __DIR__ . '/../layout/administracion_cabecera.php'; ?>

    <main class="flex-grow container mx-auto p-6 lg:p-8 relative transition-colors duration-300">

        <h1 class="text-3xl font-display font-bold text-slate-900 dark:text-white uppercase tracking-tight mb-8 rel

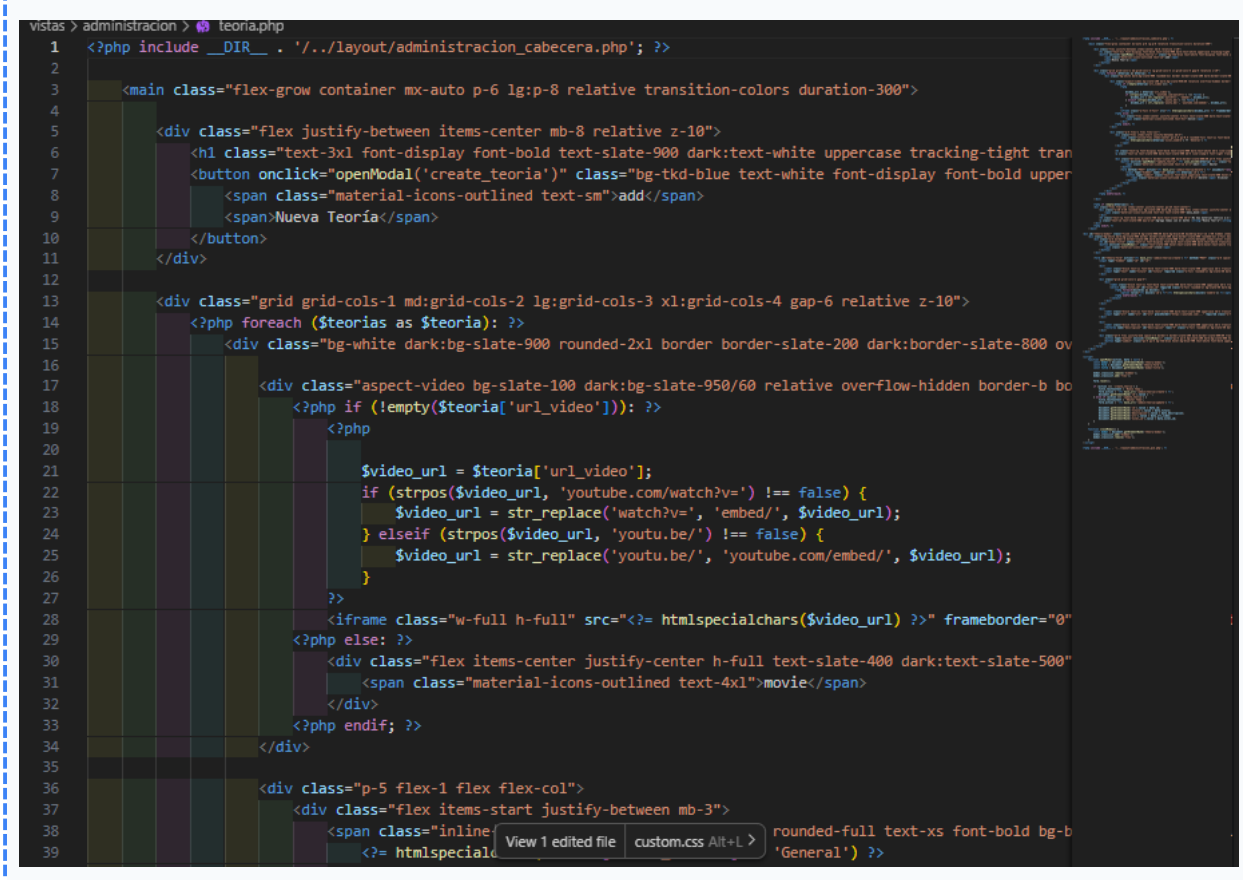
        <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6 relative z-10">

            <?php foreach ($sedes as $sede): ?>
                <div class="bg-white dark:bg-slate-900 rounded-2xl border border-slate-200 dark:border-slate-800 ov
                    <div class="aspect-video relative overflow-hidden border-b border-slate-200 dark:border-slate-8
                        <div class="absolute inset-0 bg-slate-900/10 dark:bg-[#0b0f19]/35 group-hover:bg-transparen
                        <iframe width="100%" height="100%" frameborder="0" scrolling="no" marginheight="0" marginwi
                    </div>
                    <div class="p-5">
                        <h2 class="text-lg font-display font-bold text-slate-800 dark:text-white mb-3 group-hover:t
                        <div class="flex items-center space-x-2 mt-2 text-slate-600 dark:text-slate-300">
                            <span class="material-icons-outlined text-base text-tkd-blue">location_on</span>
                            <p class="text-sm font-medium"><?= htmlspecialchars($sede['direccion']) ?></p>
                        </div>
                        <div class="flex items-center space-x-2 mt-2 text-slate-600 dark:text-slate-300">
                            <span class="material-icons-outlined text-base text-tkd-blue">phone</span>
                            <p class="text-sm font-medium"><?= htmlspecialchars($sede['telefono'] ?? 'N/A') ?></p>
                        </div>
                        <div class="mt-6 pt-4 border-t border-slate-100 dark:border-slate-800/50 flex justify-end s
                            <button onclick='openModal("edit", <?= json_encode($sede) ?>)' class="font-bold text-bl
                            <form action="<?= base_url('/admin/sedes/delete') ?>" method="POST" class="inline-block
                                <input type="hidden" name="id" value="<?= $sede['id'] ?>">
                                <button type="submit" class="font-bold text-red-600 dark:text-red-500 hover:text-re
                            </form>
                        </div>
                    </div>
                </div>
            <?php endforeach; ?>

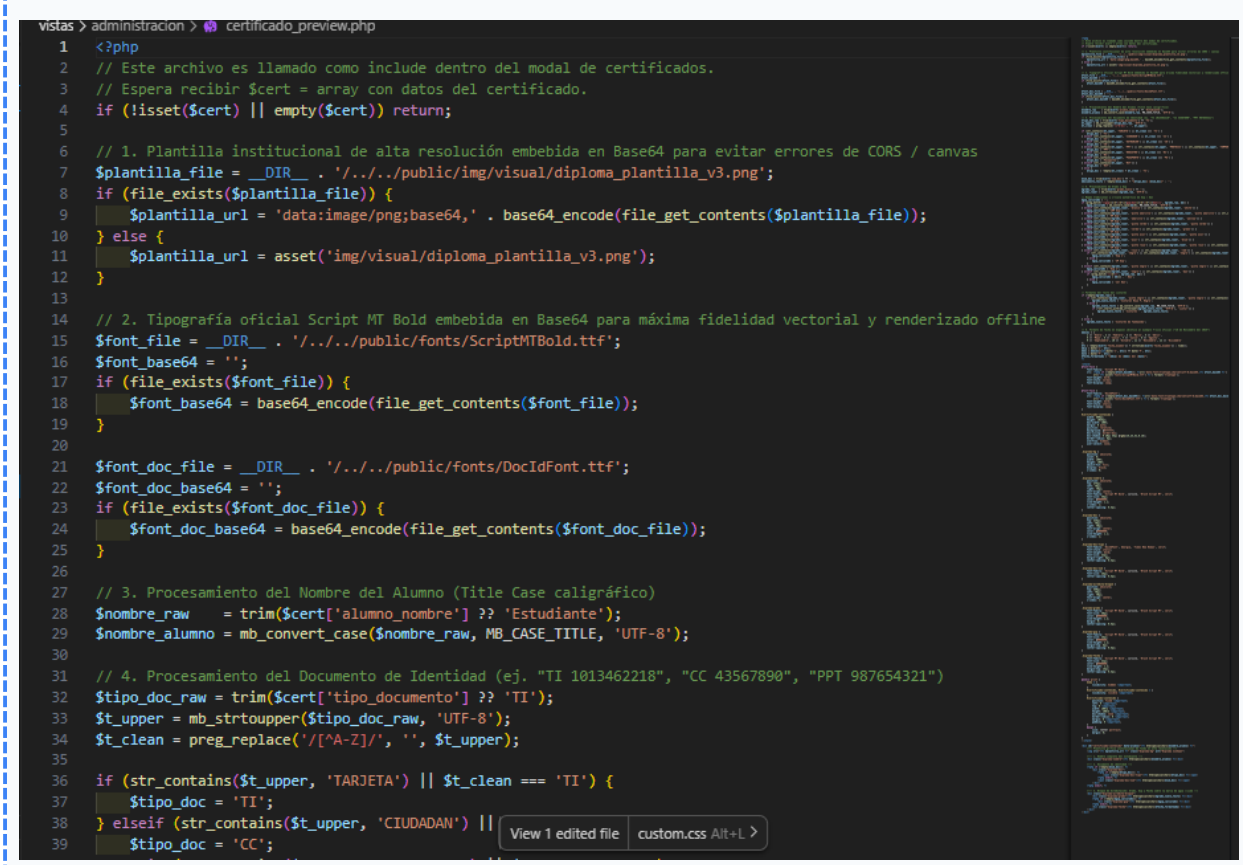
            <div onclick="openModal('add')" class="bg-slate-50 dark:bg-slate-950/20 border-2 border-dashed border-s
                <div class="text-center text-slate-500 dark:text-slate-400 group-hover:text-tkd-blue transition-col
                    <div class="w-16 h-16 bg-wh                                    border-slate-200 dark:border-slate-8
                        <span class="material-ic                                    n>

View 1 edited file  custom.css Alt+L
```

- **vistas/administracion/teoria.php:** Módulo de gestión de artículos teóricos y vocabulario marcial.



- **vistas/administracion/certificado_preview.php:** Vista previa del diploma oficial antes de su emisión definitiva.



16.4 Vistas del Maestro

- **vistas/maestro/dashboard.php:** Panel con las próximas sesiones del día y accesos rápidos.

vistas > maestro > dashboard.php

```php
<?php include __DIR__ . '/../layout/maestro_cabecera.php'; ?>

<main class="flex-grow container mx-auto p-6 lg:p-10 space-y-8 animate-fade-in-up">

    <!-- Instructor Hero Banner -->
    <div class="bg-gradient-to-r from-purple-950 via-slate-900 to-indigo-950 p-6 md:p-8 rounded-3xl text-white shad
        <div class="absolute -right-10 -bottom-10 w-64 h-64 bg-purple-600/15 rounded-full blur-3xl pointer-events-n
        <div class="absolute right-1/3 -top-12 w-48 h-48 bg-amber-500/10 rounded-full blur-3xl pointer-events-none"

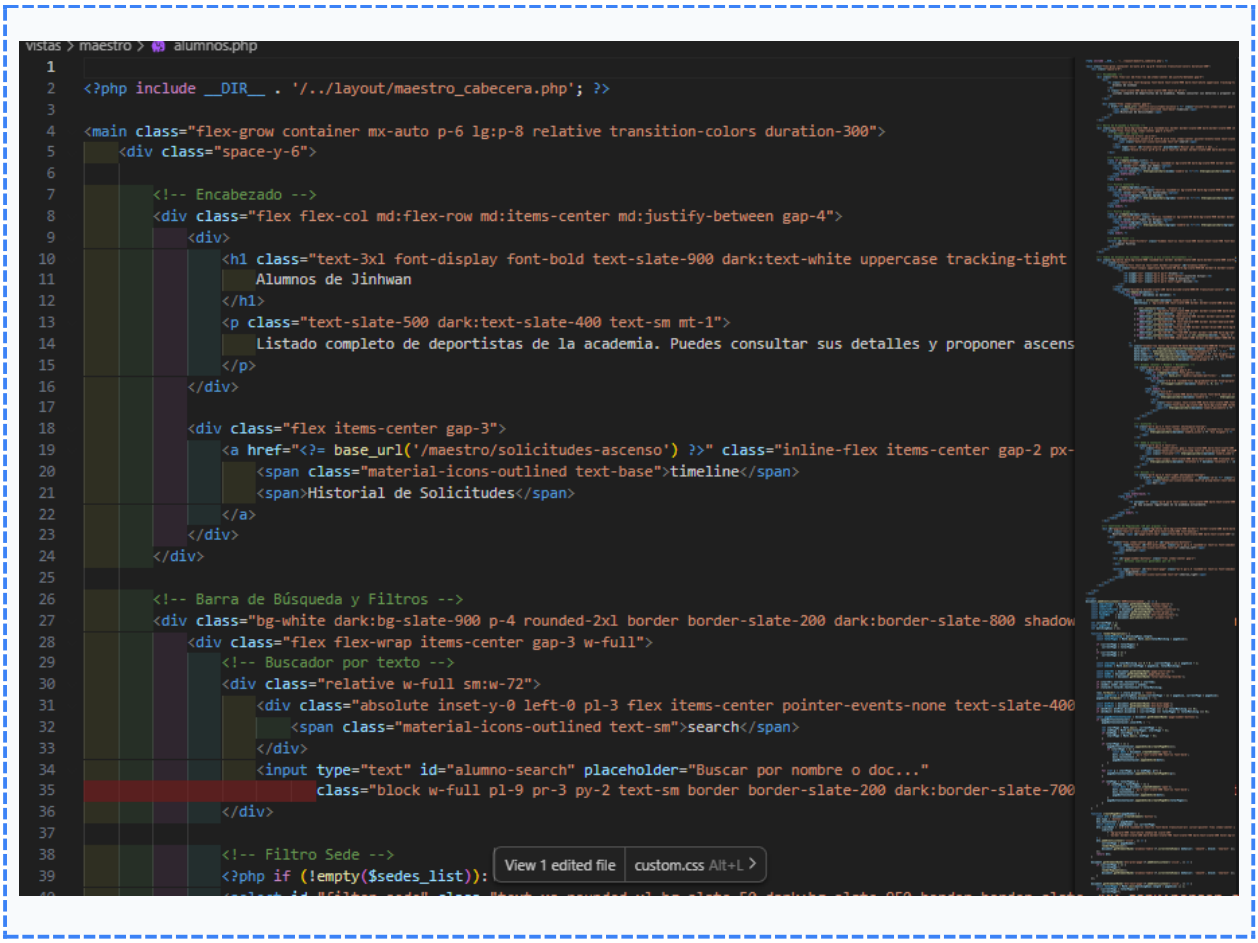
        <div class="relative z-10 flex flex-col md:flex-row md:items-center justify-between gap-6">
            <div class="space-y-2">
                <div class="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-purple-500/20 text-purple-300
                    <span class="pulse-dot bg-purple-400"></span>
                    <span>Panel de Instructor / Maestro</span>
                </div>
                <h1 class="text-3xl md:text-4xl font-display font-bold tracking-tight text-white">¡Hola, Sabonim!</
                <p class="text-slate-300 text-sm max-w-xl">Supervisa el avance de tus alumnos, propone grados de as
            </div>

            <div class="flex items-center gap-3 shrink-0">
                <div class="bg-white/10 backdrop-blur-md px-4 py-2.5 rounded-xl text-sm font-semibold text-slate-20
                    <span class="material-icons-outlined text-purple-400 text-lg">calendar_today</span>
                    <?= date('d M, Y') ?>
                </div>
            </div>
        </div>
    </div>

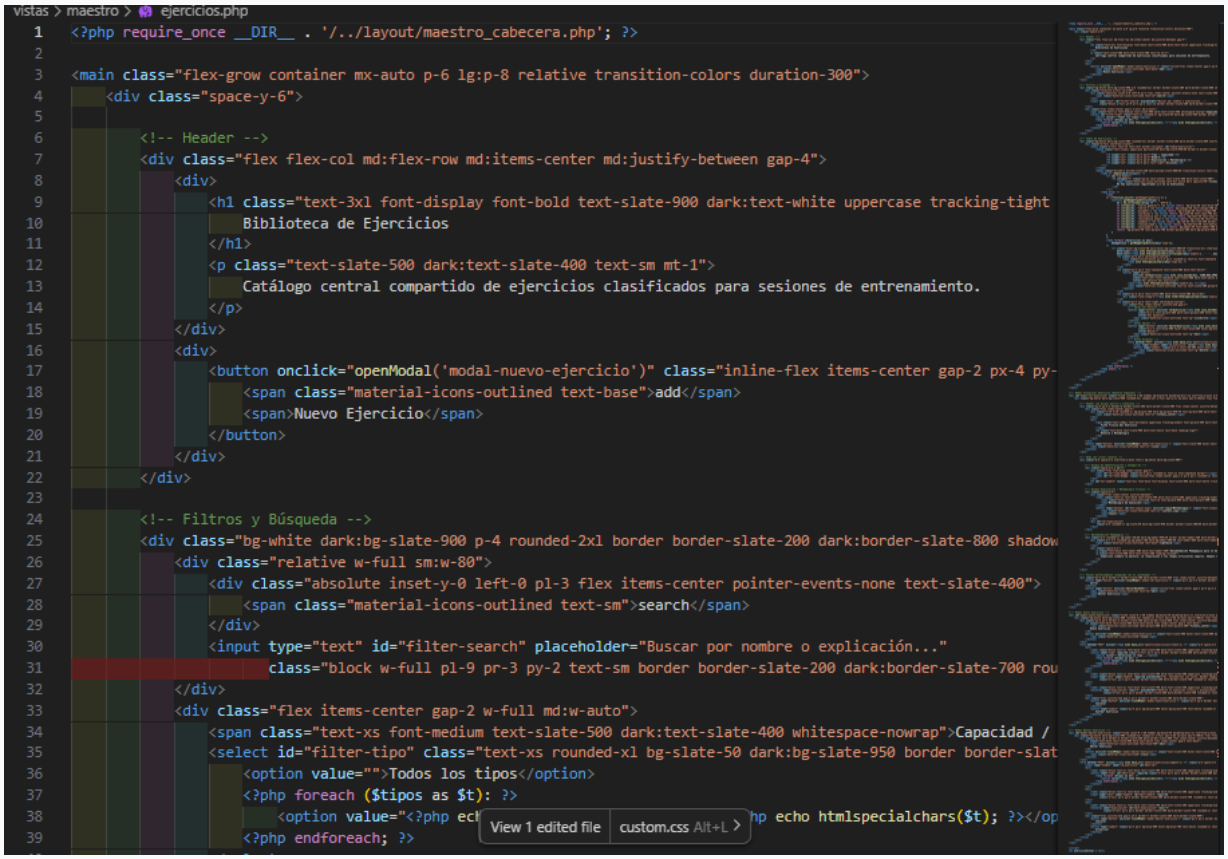
    <!-- Metric Cards Grid -->
    <div class="grid grid-cols-1 sm:grid-cols-3 gap-6">

        <div class="bg-white dark:bg-slate-900 p-6 rounded-2xl border border-slate-200 dark:border-slate-800 flex i
            <div class="w-14 h-14 rounded-2xl bg-purple-50 dark:bg-purple-500/10 flex items-center justify-center t
                <span class="material-icons-outlined text-3xl">people</span>
            </div>
            <div>
                <p class="text-xs font-bold text-slate-400 dark:text-slate-500 uppercase tracking-widest mb-1">Tota
                <h3 class="text-3xl font-black t                View 1 edited file  custom.css Alt+L  <?= $stats['total_alumnos'] ?></h3>
            </div>
```

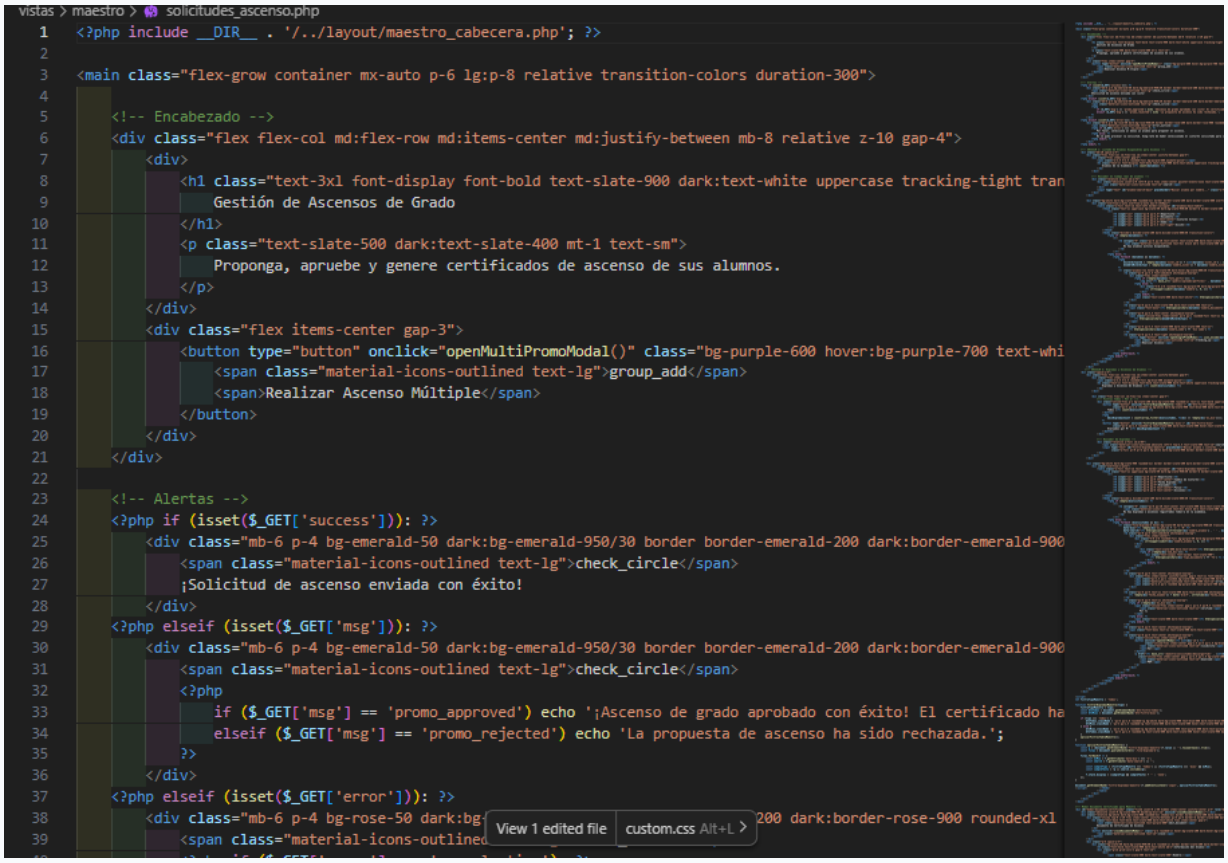
- **vistas/maestro/alumnos.php y alumno_detalle.php:** Listado y ficha técnica de los alumnos a su cargo.



- **vistas/maestro/ejercicios.php:** Catálogo técnico con tarjetas de ejercicios clasificados por categoría.

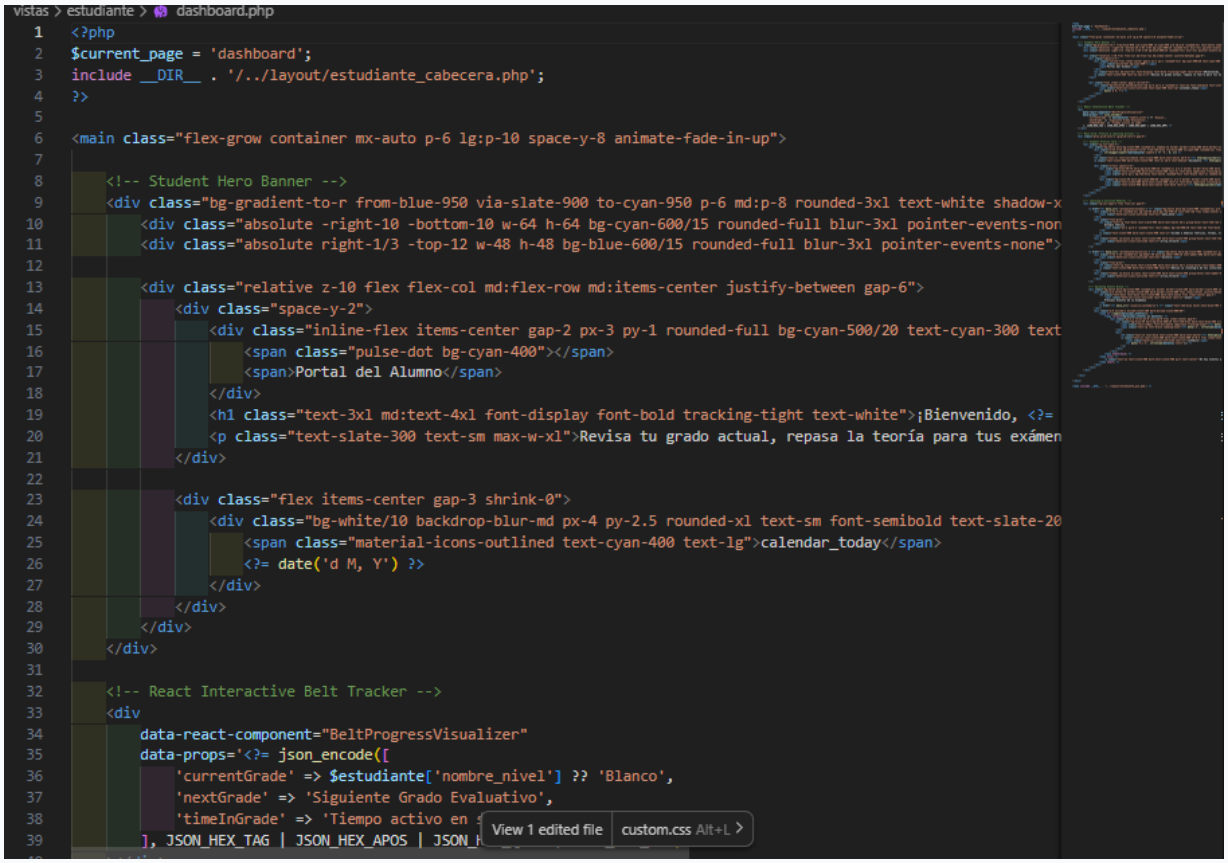


- **vistas/maestro/solicitudes_ascenso.php:** Bandeja de postulaciones a examen de grado enviadas a administración.

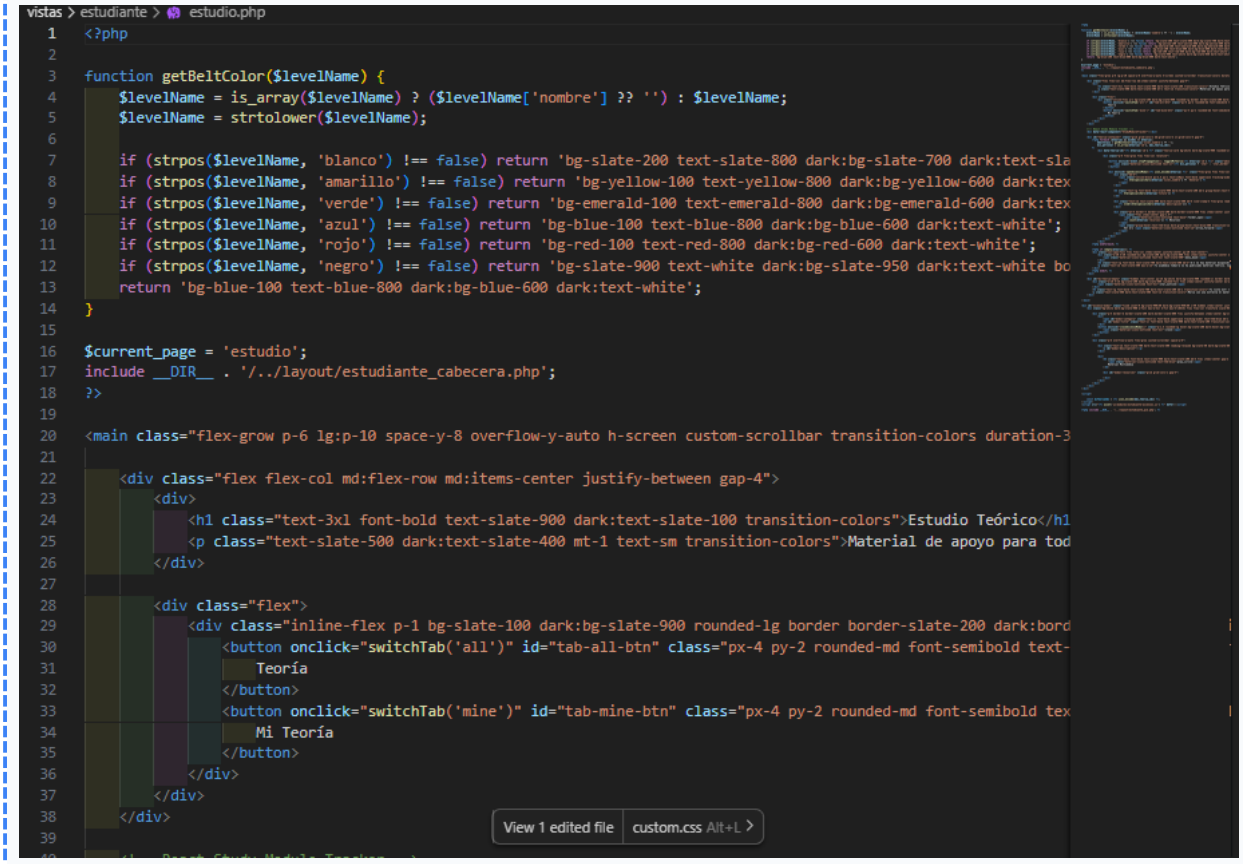


16.5 Vistas del Estudiante

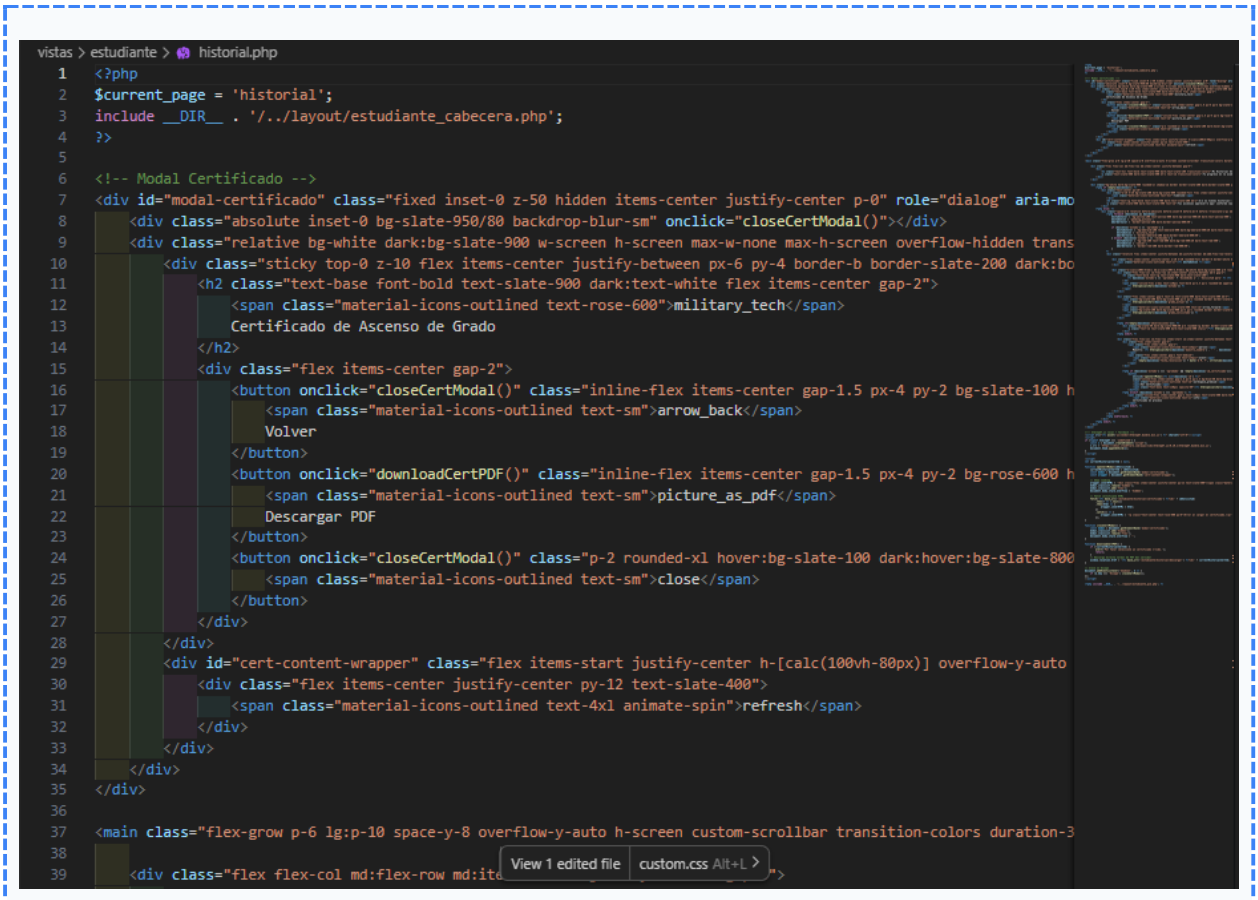
- **vistas/estudiante/dashboard.php:** Panel con cinturón actual, porcentaje de asistencia y próximos torneos.



- **vistas/estudiante/estudio.php:** Biblioteca interactiva de guías de estudio organizadas por grado.

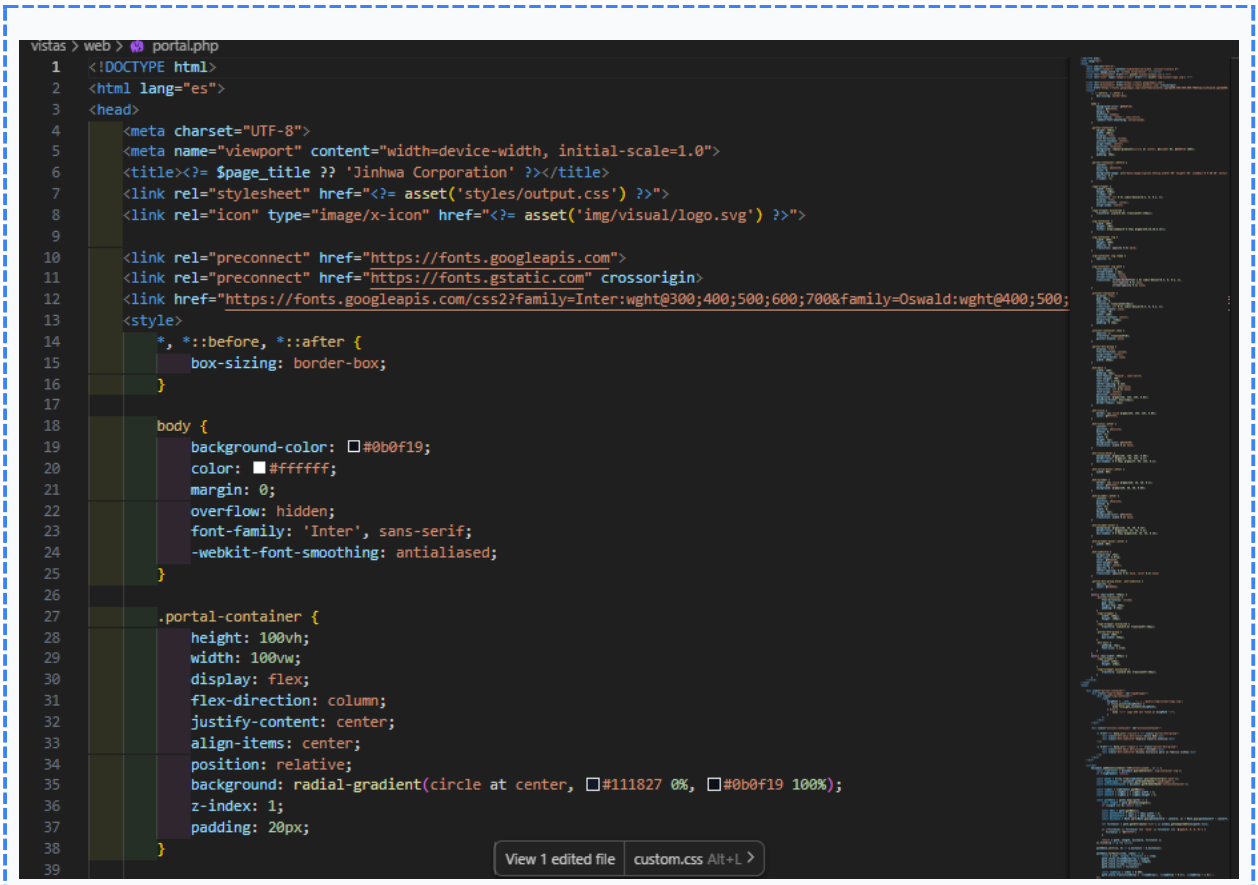


- **vistas/estudiante/historial.php:** Línea de tiempo con los exámenes aprobados y botón para ver certificado.

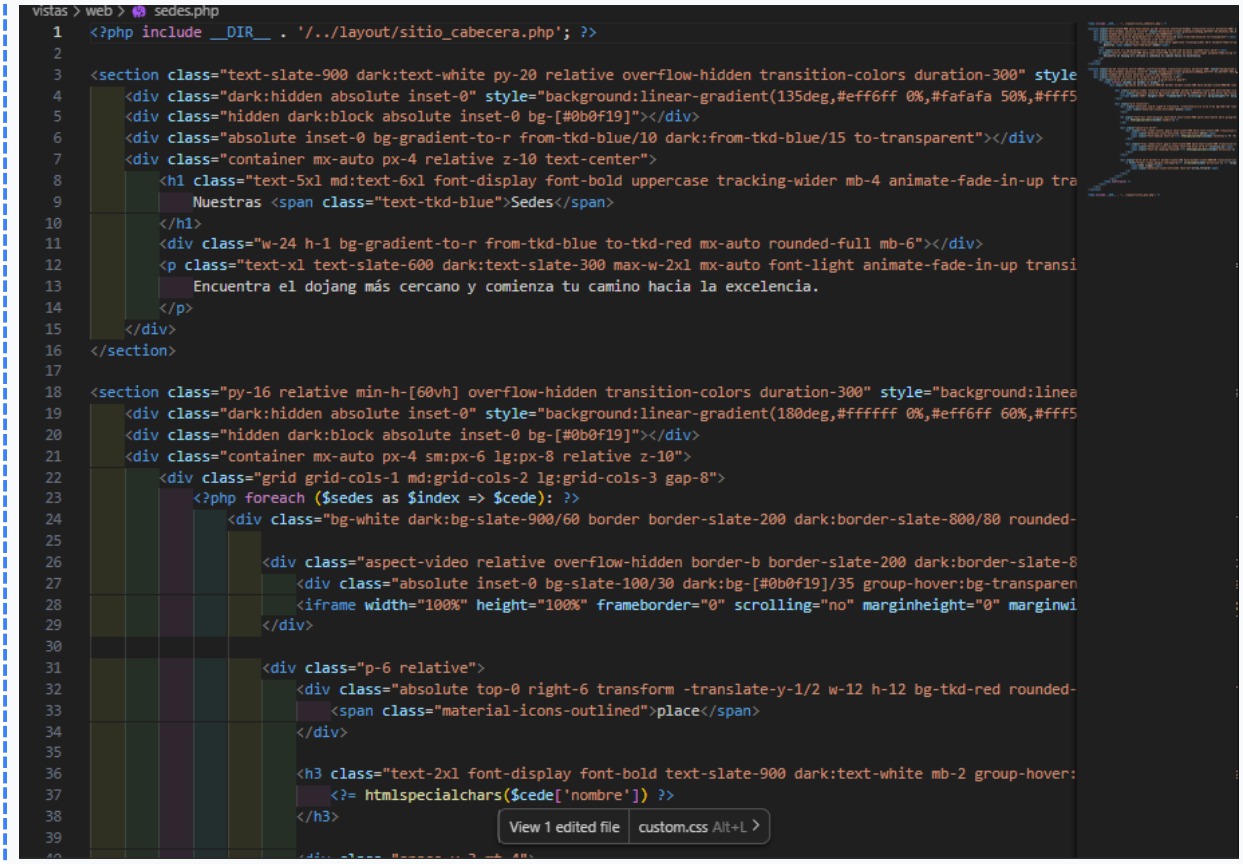


16.6 Vistas del Portal Público Institucional (vistas/web/)

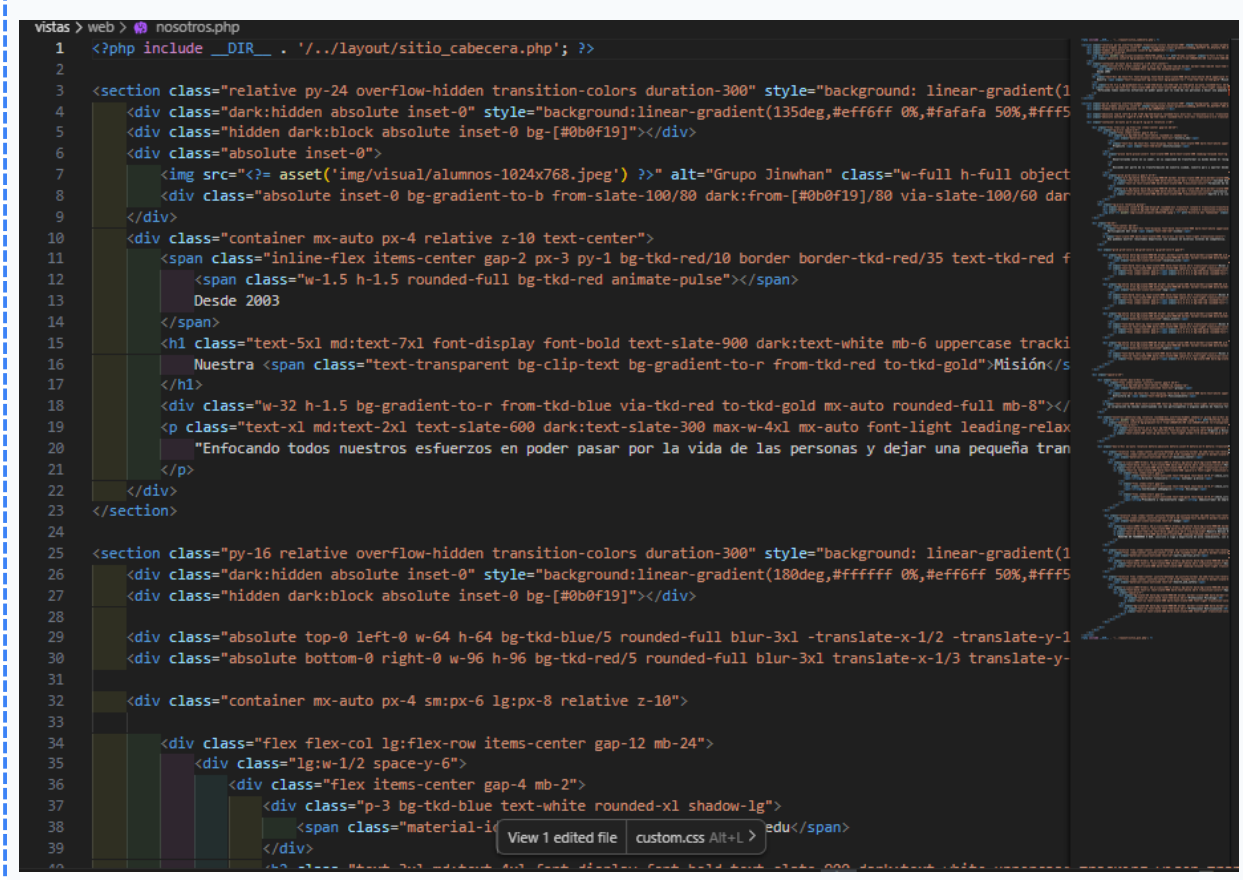
- **vistas/web/portal.php y vistas/web/inicio.php:** Página de bienvenida con presentación marcial de la corporación.



- **vistas/web/sedes.php y vistas/web/grupos.php:** Catálogo público de sucursales, ubicaciones y horarios.



- **vistas/web/nosotros.php y vistas/web/miembros.php:** Historia de la corporación y directorio del cuerpo docente.



GITHUB DE LA CORPORACION: https://github.com/wilouzet22/Jinhwan-Corporation